

TEACHER MANAGEMENT SYSTEM (GENIUSAQILOS)

Brendan Chia Yan Fei	A23CS0211
Chew Chiu Xian	A23CS0061
Elijah She Yu Sheng	A23CS0073
Neo Li Xin	A23CS0253
Ng Yu Hin	A23CS0148



UNIVERSITI TEKNOLOGI MALAYSIA

DECLARATION OF THESIS/PROJECT REPORT

Author's Full Name :
Student's Matric No. : Academic Session :
Date : UTM Email :
Thesis/Project Report Title :

I declare that this thesis is classified as : Please tick one (/)

- | | | |
|--------------------------|---------------------|--|
| ☐ | OPEN ACCESS | I agree that my thesis/project report to be published as hard copy or online open access. |
| ☐ | RESTRICTED | Contain restricted information as specified by the organization/institution where research was done. (The Library will block access for up to five (5) years). |
| ☐ | CONFIDENTIAL | Contain confidential information under Official Secret Act 1972. |

(If none of the options are selected, the first option will be carried out by default)

I acknowledge the intellectual property in the thesis/project report belonged to Universiti Teknologi Malaysia and I agree to allow this thesis/project report to be placed at the library under the following terms:

1. This thesis/project report is the property of Universiti Teknologi Malaysia.
2. The library of Universiti Teknologi Malaysia has the right to make copies for educational purposes only.
3. The library of Universiti Teknologi Malaysia is allowed to make copies of this thesis for academic exchange.

Signature of Student :

Full Name :

Date :

Approved by Supervisor :

Signature of University Coach :

Signature of Industry Coach :

Full Name of University Coach:

Full Name of Industry Coach:

Date :

Date:

NOTES: If the thesis is CONFIDENTIAL or RESTRICTED, please attach with the letter from the organization with period and reasons for confidentiality or restriction.

“I hereby declare that we have read this thesis and in my
opinion this thesis is sufficient in term of scope and quality for the
award of the degree of Bachelor of Computer Science (Data Engineering)”

Signature :
Industry Coach :
Date :

Signature :
University Coach :
Date :

VALIDATION OF COLLABORATION

It is verified that this project is achieved through the collaboration between

XXX (MALAYSIA) SDN. BHD.

and

UNIVERSITI TEKNOLOGI MALAYSIA

Verified by (Industry):

Signature:

Date:

Name:

Affiliation: XX (MALAYSIA) SDN. BHD.

Official stamp:

Verified by (University):

Signature:

Name:

Date:

Affiliation: UNIVERSITI TEKNOLOGI MALAYSIA

Official stamp:

TEACHER MANAGEMENT SYSTEM (GENIUSAQILOS)

Brendan Chia Yan Fei	A23CS0211
Chew Chiu Xian	A23CS0061
Elijah She Yu Sheng	A23CS0073
Neo Li Xin	A23CS0253
Ng Yu Hin	A23CS0148

A thesis submitted in fulfilment of the
requirements for the award of the degree of

Bachelor of Computer Science (Data Engineering)

Faculty of Computing
Universiti Teknologi Malaysia

JULY 2026

DECLARATION

I declare that this thesis entitled “*title* “ is the result of my own research except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature :

Name :

Date :

DEDICATION

This thesis is dedicated to my father, who taught me that the best kind of knowledge to have is that which is learned for its own sake. It is also dedicated to my mother, who taught me that even the largest task can be accomplished if it is done one step at a time.

ACKNOWLEDGEMENT

In the acknowledgements section, the author expresses gratitude to those who have provided personal and professional support throughout the thesis writing process. It is acceptable to use a more informal style and first-person pronouns than is typically permitted in academic writing. Acknowledgments are not considered part of the academic work itself; rather, they are an opportunity for you to express yourself in a more personal manner. The thesis acknowledgements should not exceed one page.

ABSTRACT

Teacher management is an essential administrative function in kindergarten operations because it involves staff records, leave applications, duty assignments, training progress, performance evaluation, and internal reporting. Tadika Aqil Miqail currently faces operational challenges due to manual and fragmented teacher management processes, where teacher information, documents, leave records, duties, training evidence, and reports are not fully centralized in one system. This makes it difficult for the principal to retrieve updated information, monitor staff availability, distribute duties fairly, track teacher development, and follow up on operational issues efficiently. Therefore, the aim of this project is to develop GENIUSAQILOS, a centralized Teacher Management System designed specifically for Tadika Aqil Miqail.

The system supports three main user roles: Principal, Teacher, and Technician. A system development methodology was applied, involving requirement analysis, system design, implementation, and testing. The requirements were identified based on the kindergarten's operational needs, including teacher records, leave management, task and duty management, teacher training tracker, teacher performance tracker, and teacher reporting system. The system design includes role-based access control to ensure that each user can only access functions related to their responsibilities.

The principal can manage teacher records, approve leave applications, assign duties, monitor training, evaluate performance, and review reports. Teachers can update their profiles, upload documents, apply for leave, view assigned duties, submit training records, and report issues. Technicians can handle assigned operational reports related to facility, safety, equipment, or system problems.

The outcome of this project shows that a centralized and role-based teacher management system can reduce administrative redundancy, improve information accessibility, support clearer task monitoring, and strengthen follow-up actions within the kindergarten. GENIUSAQILOS contributes practically by helping Tadika Aqil Miqail manage teacher-related operations in a more organized, secure, and efficient manner. The system also provides a foundation for future improvement, such as advanced notification features, dashboard analytics, and wider integration with other kindergarten management functions.

ABSTRAK

Pengurusan guru merupakan fungsi pentadbiran yang penting dalam operasi tadika kerana ia melibatkan rekod staf, permohonan cuti, tugas harian, perkembangan latihan, penilaian prestasi, dan pelaporan dalaman. Tadika Aqil Miqail kini menghadapi cabaran operasi kerana proses pengurusan guru masih dilakukan secara manual dan berasingan, di mana maklumat guru, dokumen, rekod cuti, tugas, bukti latihan, dan laporan tidak disimpan sepenuhnya dalam satu sistem berpusat. Keadaan ini menyukarkan pengetua untuk mendapatkan maklumat terkini, memantau kehadiran dan ketersediaan guru, mengagihkan tugas secara adil, menjejak perkembangan latihan, serta mengambil tindakan susulan terhadap isu operasi dengan cekap. Oleh itu, tujuan projek ini adalah untuk membangunkan GENIUSAQILOS, iaitu Sistem Pengurusan Guru berpusat yang direka khusus untuk Tadika Aqil Miqail.

Sistem ini menyokong tiga peranan pengguna utama, iaitu Pengetua atau Admin, Guru atau Pekerja, dan Juruteknik atau Admin Operasi. Metodologi pembangunan sistem digunakan dalam projek ini, merangkumi analisis keperluan, reka bentuk sistem, pelaksanaan, dan pengujian. Keperluan sistem dikenal pasti berdasarkan operasi sebenar tadika, termasuk rekod guru, pengurusan cuti, pengurusan tugas dan duty, penjejak latihan guru, penjejak prestasi guru, dan sistem pelaporan guru. Reka bentuk sistem ini menggunakan kawalan akses berasaskan peranan bagi memastikan setiap pengguna hanya boleh mengakses fungsi yang berkaitan dengan tanggungjawab masing-masing.

Pengetua boleh mengurus rekod guru, meluluskan permohonan cuti, menetapkan tugas, memantau latihan, menilai prestasi, dan menyemak laporan. Guru pula boleh mengemas kini profil, memuat naik dokumen, memohon cuti, melihat tugas, menghantar rekod latihan, dan membuat laporan. Juruteknik boleh mengendalikan laporan operasi berkaitan kemudahan, keselamatan, peralatan, atau masalah sistem.

Hasil projek ini menunjukkan bahawa sistem pengurusan guru yang berpusat dan berasaskan peranan dapat mengurangkan kerja pentadbiran berulang, meningkatkan capaian maklumat, menyokong pemantauan tugas yang lebih jelas, serta mengukuhkan tindakan susulan dalam tadika. GENIUSAQILOS memberi sumbangan praktikal dengan membantu Tadika Aqil Miqail mengurus operasi berkaitan guru dengan lebih tersusun, selamat, dan cekap.

TABLE OF CONTENTS

DECLARATION	ii
DEDICATION	iii
ABSTRACT.....	v
ABSTRAK.....	vi
TABLE OF CONTENTS	vii
CHAPTER 1	1
1.1 Introduction	1
1.2 Problem Background.....	1
1.3 Project Aim	3
1.4 Project Objectives	3
1.5 Project Scopes	3
1.6 Project Importance.....	5
1.7 Report Organization.....	6
CHAPTER 2	7
2.1 Introduction	7
2.2 Fundamental Theory and Concepts.....	7
2.2.1 Management Information System (MIS)	7
2.2.2 Human Resource Management System (HRMS)	8
2.2.3 Role-Based Access Control (RBAC)	9
2.2.4 Cloud Computing	10
2.2.5 Mobile Application Development	10
2.2.6 Software Development Life Cycle (SDLC) - Agile Model	11
2.3 Related Previous Research and Existing Systems.....	12
2.3.1 Human Resource Management Systems in Educational Institutions	12
2.3.2 Teacher Management System	13
2.3.3 School Management System	13
2.3.4 Teacher Performance Evaluation Systems	14
2.3.5 Leave Management System	15
2.3.6 Comparison of Existing Systems	16
2.3.7 Research Gap	17
2.4 Technology Used.....	18
2.4.1 Flutter	18
2.4.2 Dart	18
2.4.3 Firebase Authentication	19
2.4.4 Cloud Firestore	19
2.4.5 Firebase Storage	19
2.4.6 GitHub	20
2.5 Chapter Summary.....	20
CHAPTER 3	22
3.1 Introduction	22

3.2	Methodology Choice and Justification	22
3.3	Phases of the Chosen Methodology	24
3.3.1	Requirements Phase	24
3.3.2	Design Phase	25
3.3.3	Implementation Phase	26
3.3.4	Testing Phase	28
3.3.5	Deployment Phase	28
3.4	Technology Used Description	29
3.5	System Requirement Analysis: Hardware and Software.....	30
3.5.1	Hardware Requirements	30
3.5.2	Software Requirements	31
3.6	Chapter Summary.....	32
CHAPTER 4		34
4.1	Introduction	34
4.2	System Analysis	34
4.2.1	Case Study.....	34
4.2.2	Company Organization Structure	35
4.2.3	System Requirements Gathering Technique	36
4.3	System Requirements	37
4.3.1	Functional Requirements.....	37
4.3.2	Non-Functional Requirements	40
4.4	Current System Analysis (Manual/any improvement)	42
4.5	System Design.....	43
4.5.1	System Architecture	43
4.5.2	Explain each component in system architecture	44
4.5.3	System Design.....	46
4.5.3.1	Use Case Diagram	46
4.5.3.2	Activity Diagram	71
4.5.3.3	Sequence Diagram	88
4.5.4	Database Design.....	101
4.5.5	Interface Design	102
4.5.5.1	Overall Interface Design List	102
4.5.5.2	Registration Module	104
4.5.5.3	Manage User Module	106
4.5.5.4	Teacher Records Module	108
4.5.5.5	Leave Management Module	110
4.5.5.6	Task & Duty Manager Module	115
4.5.5.7	Teacher Training Tracker Module	119
4.5.5.8	Teacher Performance Tracker Module	125
4.5.5.9	Teacher Reporting System Module	126
4.5.6	Chapter Summary.....	127

CHAPTER 5	128
5.1 Introduction	128
5.2 System development	128
5.2.1 Development Environment and Framework	129
5.2.2 Backend and Database Development	129
5.2.3 Authentication and Role-Based Access Control Development.....	130
5.2.4 Main Module Development.....	130
5.2.5 Interface Development	131
5.2.6 Version Control and Team Collaboration	132
5.3 Coding of system's main functions.....	132
5.4 Essential interfaces that show system's results and achievements.....	132
5.5 Testing.....	136
5.5.1 Black box testing.....	137
5.5.2 White box testing	137
5.5.3 User testing.....	137
5.6 Chapter Summary.....	137
CHAPTER 6	143
6.1 Introduction	143
6.2 System Contribution/Achievement	143
6.3 System Constraint	143
6.4 Future Suggestion.....	143
6.5 Chapter Summary.....	143
6.2 Achievement of Project Objectives.....	145
APPENDIX	147

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 2.1		
Table 2.2		
Table 2.3		
Table A.1		

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
Figure 2.1		
Figure 3.1		
Table 2.3		
Table A.1		

LIST OF ABBREVIATIONS

[illegible]

LIST OF SYMBOLS

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
Appendix A		
Appendix B		
Appendix C		
Appendix D		

CHAPTER 1

INTRODUCTION

1.1 Introduction

Teacher management is an important part of kindergarten administration because teachers are directly involved in daily teaching, classroom supervision, student care, duty execution, documentation, and communication with parents. In Tadika Aqil Miqail, the principal needs to manage several teacher-related operations, including teacher records, leave applications, daily duty assignments, training records, performance evaluation, and internal reports. These operations require accurate information, proper tracking, and quick access to updated records.

Currently, many of these administrative activities are still handled manually or through scattered documents. This makes the management process time-consuming and less efficient, especially when the principal needs to retrieve teacher information, check document completeness, monitor leave balances, assign duties fairly, or review teacher performance. As the kindergarten grows, relying on manual records may increase the risk of missing documents, repeated work, unclear task responsibility, and delayed decision-making.

Therefore, this project proposes the development of GENIUSAQILOS, a Teacher Management System designed for Tadika Aqil Miqail. The system aims to centralize teacher-related administrative workflows into one digital platform. It supports three main user roles, namely Principal or Admin, Teacher or Employee, and Technician or Operational Admin. Through this system, the kindergarten can manage teacher data more systematically, improve administrative efficiency, and support better daily operation monitoring.

1.2 Problem Background

Tadika Aqil Miqail currently faces administrative challenges due to the absence of a centralized teacher management system. Although the kindergarten has a manageable number of teachers, the principal still needs to handle many operational tasks, such as

storing teacher profiles, checking required documents, approving leave applications, assigning daily duties, monitoring teacher training, reviewing performance, and handling internal reports. When these processes are managed manually or separately, the principal needs more time to search for information and update records.

The first problem is fragmented teacher information. Teacher personal details, employment-related information, and important documents such as MyKad copy, passport photo, resume, academic certificates, medical check-up report, and bank statement header need to be stored properly. Without a centralized system, it becomes difficult to ensure that all teacher records are complete, updated, and easy to retrieve.

The second problem is inefficient leave and duty management. Teachers have different leave types, such as annual leave, medical leave, unpaid leave, maternity leave, marriage leave, compassionate leave, Umrah leave, Haji leave, birthday leave, and half-day leave. Manual tracking of this leave balances may lead to confusion or inaccurate records. At the same time, daily duties such as cleaning duty, arrival duty, dismissal duty, half-full day transition duty, and assembly duty need to be assigned and monitored clearly so that every teacher understands their responsibilities.

The third problem is the difficulty of monitoring teacher training and performance. Teachers are required to attend training and record training details such as course title, category, organizer, date, duration, mode, venue, and reflection. The principal also needs to evaluate teachers using performance categories such as attendance, classroom management, teaching performance, documentation, communication, duty responsibility, creativity, self-development, and SOP compliance. Without a proper system, training progress and performance records may be hard to track consistently.

The fourth problem is the lack of a structured internal reporting process. Teachers may need to report issues related to safety hazards, facility maintenance, teaching material shortages, IT problems, SOP violations, workload stress, staff conflict, or misconduct. If reports are submitted informally, it may be difficult for the principal or technician to track the report status and take follow-up action.

Based on these problems, GENIUSAQILOS is proposed as a system development project to improve the current manual and fragmented process. The system will provide a structured platform for managing teacher records, leave, duties, training,

performance, and reporting in one integrated environment.

1.3 Project Aim

The aim of this project is to develop GENIUSAQILOS, a centralized Teacher Management System for Tadika Aqil Miqail to improve teacher record management, leave management, duty tracking, training monitoring, performance evaluation, and internal reporting.

1.4 Project Objectives

The objectives of this project are:

- i. To analyse the current teacher management process at Tadika Aqil Miqail and identify the main administrative problems related to teacher records, leave, duties, training, performance, and reporting.
- ii. To design a role-based Teacher Management System that supports the main users of the system, including Principal, Teacher, and Technician.
- iii. To develop the GENIUSAQILOS system with core modules such as login, registration, manage user, teacher records, leave management, task and duty manager, teacher training tracker, teacher performance tracker, and teacher reporting system.
- iv. To test the developed system based on its functionality, usability, role-based access control, and ability to support the daily administrative needs of Tadika Aqil Miqail.

1.5 Project Scopes

The scope of this project focuses on the development of a Teacher Management System for Tadika Aqil Miqail. The system is designed to support the daily administrative work related to teacher management and operational monitoring.

- (a) The first scope is user account and access management. The system includes login, registration, and managing user modules. These modules allow users to

register, log in securely, and access the system based on their assigned roles. The principal can manage user accounts, update roles, deactivate accounts, and control user permissions. Teachers can access their own information, while technicians can only access assigned operational issue functions.

- (b) The second scope is teacher record management. The system allows teacher profiles to be stored digitally, including personal details, contact information, emergency contact details, and required documents. Teachers can update personal information and upload documents, while the principal can view teacher records and verify document status.
- (c) The third scope is leave management. Teachers can submit leave applications through the system, while the principal can review, approve, or reject applications. The system also helps monitor leave balances and leave history according to the leave types provided by Tadika Aqil Miqail.
- (d) The fourth scope is task and duty management. The system supports the assignment and tracking of teacher duties such as cleaning duty, arrival duty, dismissal duty, half-full day transition duty, and assembly duty. Teachers can view their assigned duties and update completion status, while the principal can monitor duty progress.
- (e) The fifth scope is training and performance management. Teachers can submit training records, upload related evidence, and track their own training progress. The principal can review training submissions and evaluate teacher performance based on the required KPI categories.
- (f) The sixth scope is teacher reporting. The system allows teachers to submit reports related to school operations, safety, facility maintenance, teaching materials, staff issues, or system problems. Technicians can handle assigned operational reports, while the principal can monitor report status and follow-up actions.

This project does not include student academic management, parent communication system, payroll calculation, full financial accounting, biometric attendance integration, or advanced artificial intelligence prediction. The project focuses only on teacher management and internal operational reporting required by Tadika Aqil Miqail.

1.6 Project Importance

This project is important because it helps Tadika Aqil Miqail move from manual and scattered teacher management processes to a more organized digital system. By centralizing teacher records, the principal can access updated information faster and reduce the risk of missing or incomplete documents. This supports better administrative accuracy and saves time in daily operations.

The system is also important for improving leave and duty management. Digital leave application and duty tracking can reduce confusion, make responsibilities clearer, and help the principal monitor teacher availability more effectively. This is especially useful in a kindergarten environment where teacher presence and duty coverage directly affect daily student supervision and school operations.

In addition, GENIUSAQILOS supports better teacher development and performance monitoring. Training records and performance evaluation can be stored systematically, allowing the principal to review teacher progress and provide more structured feedback. This can help improve teacher professionalism, accountability, and continuous development.

Finally, the reporting module improves communication and follow-up for internal issues. Teachers can report problems through a structured system, while the principal and technician can track the status of each issue. This helps the kindergarten respond faster to maintenance, safety, teaching material, and staff-related concerns. Overall, this project contributes to better efficiency, transparency, and long-term administrative management for Tadika Aqil Miqail.

1.7 Report Organization

This report is organized into six chapters. Chapter 1 presents the introduction of the project, including the project background, problem background, project aim, objectives, scope, importance, and report organization. Chapter 2 discusses the literature review related to teacher management systems, digital record management, leave management, task management, training tracking, performance evaluation, and reporting systems. Chapter 3 explains the methodology used to develop the system, including the chosen development approach, project phases, tools, and system requirements. Chapter 4 presents the analysis and design of the system, including user requirements, functional and non-functional requirements, system architecture, database design, and interface design. Chapter 5 describes the implementation and testing of the system, including the developed modules, testing activities, and test results. Chapter 6 concludes the project by summarizing the achievement of objectives, project limitations, and suggestions for future improvement.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter presents a review of the literature related to teacher management systems and the technologies used in modern information systems. The review covers fundamental theories and concepts, previous research and existing systems, as well as technologies commonly adopted in mobile application development. By examining scholarly literature, the strengths and limitations of existing studies and systems are identified, allowing research gaps to be established. The findings from this chapter provide the theoretical and technological foundation for the design and development of the proposed system presented in the subsequent chapters.

2.2 Fundamental Theory and Concepts

This section discusses the fundamental theories and concepts related to information systems and software development. The topics include Management Information Systems (MIS), Human Resource Management Systems (HRMS), Role-Based Access Control (RBAC), Cloud Computing, Mobile Application Development, and the Software Development Life Cycle (SDLC) using the Agile model. Understanding these concepts provides the theoretical foundation necessary for analysing existing systems and supporting the development of modern information systems.

2.2.1 Management Information System (MIS)

Management Information System (MIS) is an integrated system that combines people, processes, and technology to collect, process, store, and distribute information for organizational decision-making. According to Laudon and Laudon (2022), an MIS enables organizations to transform raw data into meaningful information, allowing managers to make informed decisions while improving operational efficiency. By centralizing information, an MIS reduces data redundancy, minimizes manual work, and improves communication across different departments.

In educational institutions, MIS plays an important role in managing administrative

activities such as employee records, attendance, leave management, and performance evaluation. Traditional paper-based record keeping is often inefficient because information can be difficult to retrieve, update, and maintain. A computerized management system provides centralized data storage, allowing authorized users to access accurate and up-to-date information while improving productivity and reducing administrative errors.

The proposed GENIUSAQILOS Teacher Management System applies the principles of MIS by integrating multiple teacher management functions into a single mobile application. Instead of managing teacher records, leave applications, training history, performance evaluations, and reporting separately, the system centralizes these administrative processes within one platform. This enables the principal and teachers to manage information more efficiently while supporting faster and more accurate decision-making.

2.2.2 Human Resource Management System (HRMS)

A Human Resource Management System (HRMS) is an integrated information system that supports the management of an organization's human resources by digitalizing core administrative processes such as employee record management, leave administration, performance evaluation, training management, and reporting. HRMS combines human resource management practices with information technology to improve operational efficiency, reduce manual paperwork, and centralize employee information within a single platform. Modern HRMS solutions also provide employee self-service functions, enabling staff to access and update relevant information while reducing the administrative workload of management.

The implementation of an HRMS offers several benefits to organizations. By storing employee information in a centralized database, organizations can improve data accuracy, simplify information retrieval, and enhance communication between management and employees. In addition, HRMS supports better workforce planning through functions such as leave tracking, performance monitoring, training records, and report generation. These features allow managers to make informed decisions while improving productivity and ensuring that employee information remains organized and up to date.

Although HRMS is commonly implemented in business organizations, its concepts can also be applied within educational institutions. In schools and kindergartens, teachers represent the

primary human resources responsible for delivering educational services. Therefore, functions such as maintaining teacher profiles, monitoring professional development, evaluating performance, managing leave applications, and assigning daily responsibilities can be effectively managed through an HRMS. The proposed GENIUSAQILOS Teacher Management System adopts these HRMS principles by integrating teacher records, leave management, task scheduling, training tracking, performance evaluation, and reporting into a centralized mobile application. This integration helps improve administrative efficiency while providing the principal and teachers with a more systematic and accessible management platform.

2.2.3 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) is a widely adopted security model that restricts system access based on predefined user roles rather than assigning permissions to individual users. According to Sandhu et al. (1996), RBAC simplifies access management by grouping users according to their responsibilities and assigning permissions to each role. This approach enhances system security by ensuring that users can only access information and perform tasks relevant to their responsibilities, thereby reducing the risk of unauthorized access and accidental modification of sensitive data.

RBAC has become an essential security mechanism in information systems because it supports efficient permission management while maintaining data confidentiality and integrity. Instead of manually configuring permissions for every user, administrators only need to assign users to appropriate roles, making the system easier to maintain as the number of users increases. Furthermore, RBAC follows the principle of least privilege, where users are granted only the minimum permissions necessary to perform their duties, reducing potential security risks (Ferraiolo et al., 2001).

In the proposed GENIUSAQILOS Teacher Management System, RBAC is implemented to differentiate system access among three user roles: Principal, Teacher, and Technician. The Principal is responsible for managing teacher records, approving leave applications, monitoring performance, and generating reports. Teachers are limited to managing their personal information, submitting leave applications, viewing assigned tasks, and updating training records. Meanwhile, the Technician is responsible for managing user accounts and maintaining the system. By implementing RBAC, the proposed system ensures that sensitive information is protected while providing each user with access only to the functions required for their respective roles.

2.2.4 Cloud Computing

Cloud computing is a computing model that enables users to access computing resources such as servers, databases, storage, and software over the Internet without the need to maintain physical infrastructure. According to Mell and Grance (2011), cloud computing provides convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort. This model has become increasingly popular due to its scalability, flexibility, cost-effectiveness, and ease of deployment.

One of the key advantages of cloud computing is its ability to support real-time data access and synchronization across multiple devices. Cloud-based services allow users to store and retrieve information anytime and anywhere, provided that an Internet connection is available. In addition, cloud platforms offer automatic data backup, enhanced security features, and high system availability, reducing the risks associated with traditional local storage systems. These benefits make cloud computing particularly suitable for information systems that require multiple users to access shared data simultaneously.

The proposed GENIUSAQILOS Teacher Management System utilizes cloud computing to provide a centralized platform for managing teacher information and administrative processes. By leveraging cloud services, teachers and administrators can securely access records, submit leave applications, upload training certificates, and monitor performance in real time. Furthermore, cloud-based storage ensures that important documents and system data are securely maintained while supporting efficient collaboration and information sharing among authorized users.

2.2.5 Mobile Application Development

Mobile application development refers to the process of designing, developing, testing, and deploying software applications for mobile devices such as smartphones and tablets. With the widespread adoption of mobile technology, organizations increasingly utilize mobile applications to improve accessibility, communication, and service

delivery. Mobile applications enable users to access information and perform tasks anytime and anywhere, making them an effective solution for managing daily operations in various industries, including education.

Compared to traditional desktop applications, mobile applications offer greater convenience and flexibility. They provide features such as real-time notifications, intuitive user interfaces, and instant access to cloud-based services. According to Pressman and Maxim (2020), modern mobile applications emphasize usability, responsiveness, and user-centered design to ensure a positive user experience. Furthermore, cross-platform development frameworks have enabled developers to build applications that operate consistently across multiple operating systems while reducing development time and maintenance costs.

The proposed GENIUSAQILOS Teacher Management System is developed as a mobile application to improve the efficiency of administrative tasks within Tadika Aqil Miqail. Through a mobile platform, teachers can conveniently update their personal information, submit leave applications, upload training certificates, and view assigned duties, while the principal can monitor teacher performance and manage administrative activities remotely. This enhances accessibility and enables users to perform essential tasks more efficiently regardless of their location.

2.2.6 Software Development Life Cycle (SDLC) - Agile Model

The Software Development Life Cycle (SDLC) is a systematic process used to develop high-quality software through a series of structured phases, including planning, requirements analysis, system design, implementation, testing, deployment, and maintenance. SDLC provides developers with a clear framework for managing software projects, ensuring that user requirements are fulfilled while maintaining software quality throughout the development process. Different SDLC models, such as Waterfall, Spiral, and Agile, have been widely adopted depending on the nature and requirements of a project.

Among these models, the Agile development model has gained significant popularity due to its iterative and flexible approach. Unlike the traditional Waterfall model, Agile divides software development into small iterations, allowing developers to

continuously implement, test, and improve the system based on user feedback. According to Sommerville (2020), Agile promotes collaboration between developers and stakeholders, enabling project requirements to evolve throughout the development process while ensuring that working software is delivered incrementally.

The proposed GENIUSAQILOS Teacher Management System adopts the Agile model because the project is developed collaboratively by multiple team members, with each member responsible for different system modules. Agile supports continuous integration, frequent testing, and regular discussions with the project supervisor and industry client. This approach enables the team to identify issues early, incorporate feedback efficiently, and ensure that the final system meets the operational requirements of Tadika Aqil Miqail.

2.3 Related Previous Research and Existing Systems

2.3.1 Human Resource Management Systems in Educational Institutions

Human Resource Management Systems (HRMS) are widely used to digitalize employee management processes such as personnel records, leave administration, performance evaluation, and training management. In educational institutions, HRMS helps school administrators manage teacher information more efficiently by replacing manual record-keeping with centralized digital systems. Studies have shown that implementing HRMS improves administrative efficiency, enhances data accuracy, and supports better decision-making.

Runhaar (2017) highlighted that effective human resource management in schools should integrate teacher records, professional development, and performance evaluation to improve educational quality. More recently, Shaikh (2024) found that Electronic Human Resource Management (E-HRM) systems enhance teachers' performance by providing centralized management of employee information, training activities, and performance monitoring. These studies demonstrate that digital HR systems can significantly reduce administrative workload while improving organizational efficiency.

Although existing HRMS solutions provide comprehensive employee management

functions, most are developed for general organizations or higher education institutions rather than kindergartens. Therefore, the proposed GENIUSAQILOS Teacher Management System adapts HRMS concepts to meet the specific operational needs of Tadika Aqil Miqail by integrating teacher records, leave management, task scheduling, training tracking, performance evaluation, and reporting within a single mobile application.

2.3.2 Teacher Management System

Teacher Management Systems (TMS) are developed to support the administration and management of teachers within educational institutions. Unlike general Human Resource Management Systems, TMS focuses on teacher-specific functions such as managing teacher profiles, monitoring attendance, assigning duties, tracking professional development, and evaluating teaching performance. By integrating these functions into a centralized platform, TMS enables school administrators to reduce administrative workload while improving the accuracy and accessibility of teacher information.

Shafie et al. (2023) developed a web-based Teacher Management System to digitalize teacher records, attendance management, and leave applications. The study reported that the system improved administrative efficiency by reducing manual documentation and simplifying access to teacher information. Similarly, Al-Hassan et al. (2022) highlighted that centralized teacher management systems enhance communication between school administrators and teachers while providing better monitoring of staff performance and professional development activities.

Although these systems successfully digitalize teacher administration, most focus only on core functions such as teacher records, attendance, and leave management. Features including task scheduling, teacher training tracking, performance evaluation, and incident reporting are often implemented as separate systems or are not included. To overcome these limitations, the proposed GENIUSAQILOS Teacher Management System integrates these functions into a single mobile application, providing a more comprehensive solution for managing teachers at Tadika Aqil Miqail.

2.3.3 School Management System

School Management Systems (SMS) are integrated information systems designed to support the administration and daily operations of educational institutions. These systems commonly manage student information, teacher records, attendance, academic schedules, examinations, and administrative reporting through a centralized database. By digitalizing school operations, SMS improves information accessibility, enhances communication among stakeholders, and reduces the reliance on manual record-keeping.

Recent studies have shown that School Management Systems significantly improve administrative efficiency and decision-making. Grepon et al. (2021) emphasized that centralized school management systems simplify data storage, processing, and retrieval, enabling school administrators to manage information more effectively. Similarly, Damayanto et al. (2025) concluded that school management automation through Management Information Systems reduces administrative workload, improves operational efficiency, and supports better educational management.

Despite these advantages, most School Management Systems are designed to manage both academic and administrative functions, with greater emphasis on student-related operations such as enrolment, examinations, and academic records. Comparatively, less attention is given to comprehensive teacher management features such as professional training, KPI evaluation, task scheduling, and internal reporting. Therefore, the proposed GENIUSAQILOS Teacher Management System complements existing school management solutions by focusing specifically on teacher administration and integrating essential teacher management functions into a centralized mobile application tailored for Tadika Aqil Miqail.

2.3.4 Teacher Performance Evaluation Systems

Teacher Performance Evaluation Systems are developed to assess teachers' effectiveness based on predefined criteria such as teaching quality, classroom management, professional development, and administrative responsibilities. These systems assist school administrators in monitoring teacher performance objectively while identifying areas for improvement. By utilizing Key Performance Indicators (KPIs) and structured evaluation frameworks, performance evaluation systems support continuous professional development and improve the overall quality of education.

Almubarak et al. (2024) proposed an AI-based teacher performance evaluation system that integrates Key Performance Indicators (KPIs) to provide a more objective and comprehensive assessment of teachers. The study found that digital evaluation systems improve the consistency and reliability of performance assessments while assisting administrators in identifying teachers' strengths and areas requiring further development. Similarly, Kardianto et al. (2022) developed a KPI-based teacher performance assessment model and concluded that standardized performance indicators provide a fair and systematic approach for measuring teacher performance across different evaluation periods.

Although these performance evaluation systems provide structured assessment methods, they primarily focus on performance measurement and do not integrate other administrative functions such as teacher records, leave management, task scheduling, training management, and reporting. Therefore, the proposed GENIUSAQILOS Teacher Management System extends beyond performance evaluation by incorporating KPI-based teacher assessment as one component within a centralized mobile application that supports comprehensive teacher management.

2.3.5 Leave Management System

Leave Management Systems (LMS) are designed to automate the process of applying for, approving, and tracking employee leave. Compared to traditional paper-based applications, digital leave management systems reduce administrative workload, improve record accuracy, and enable employees and administrators to monitor leave balances in real time. These systems also streamline approval workflows, reducing processing time and minimizing errors associated with manual documentation.

Research by Ibrahim et al. (2023) found that implementing an online leave management system significantly improved the efficiency of leave processing by providing automated application submission, approval notifications, and centralized leave records. Similarly, Ali et al. (2022) reported that digital leave management systems enhance transparency and communication between employees and administrators while reducing paperwork and ensuring that leave records remain accurate and easily accessible.

Although existing leave management systems effectively digitalize leave applications and approvals, they are generally developed as standalone applications focusing only on leave administration. They often lack integration with other teacher management functions such as performance evaluation, task scheduling, training records, and incident reporting. The proposed GENIUSAQILOS Teacher Management System addresses this limitation by incorporating leave management as one of several integrated modules within a centralized mobile application, allowing administrators to manage all teacher-related information through a single platform.

2.3.6 Comparison of Existing Systems

Table 2.1 compares the major features of the existing systems discussed in the previous sections. Although these systems successfully digitalize various administrative functions, each focuses on a limited set of features. Most Human Resource Management Systems emphasize employee information management, while Teacher Management Systems mainly concentrate on teacher records and attendance. Similarly, School Management Systems are generally designed to support academic administration, and Teacher Performance Evaluation Systems primarily focus on performance assessment. Leave Management Systems, on the other hand, are often implemented as standalone applications for processing leave requests.

In contrast, the proposed GENIUSAQILOS Teacher Management System integrates these essential functions into a single mobile application. Besides managing teacher records and leave applications, the system also includes task scheduling, teacher training tracking, performance evaluation, and reporting features. This integrated approach minimizes the need for multiple independent systems and provides a more efficient solution for managing teachers at Tadika Aqil Miqail.

Table 2.1 Comparison of Existing Systems

Features	HRMS	Teacher Management System	School Management System	Teacher Performance Evaluation System	Leave Management System	GENIUS AQILOS
Teacher Records	✓	✓	✓	✗	✗	✓

Leave Management	✓	✓	X	X	✓	✓
Task Scheduling	X	Limited	X	X	X	✓
Teacher Training Tracker	✓	Limited	X	X	X	✓
Performance Evaluation	✓	Limited	X	✓	X	✓
Reporting System	Limited	X	✓	Limited	X	✓
Mobile Accessibility	Limited	Limited	Limited	Limited	✓	✓

2.3.7 Research Gap

The literature review indicates that existing Human Resource Management Systems, Teacher Management Systems, School Management Systems, Teacher Performance Evaluation Systems, and Leave Management Systems have significantly improved administrative efficiency by digitalizing various management processes. However, most of these systems focus only on specific functions, such as employee records, leave management, performance evaluation, or academic administration, rather than providing a comprehensive solution for teacher management.

Furthermore, many existing systems are designed for large organizations or general educational institutions, making them less suitable for the operational requirements of small kindergartens. Features such as task scheduling, teacher training tracking, performance evaluation, and incident reporting are often implemented separately or are not available within a single platform. As a result, administrators may still need to manage multiple systems, leading to duplicated information and increased administrative complexity.

To address these limitations, the proposed GENIUSAQILOS Teacher Management System integrates teacher records, leave management, task scheduling, training tracking, performance evaluation, and reporting into a centralized mobile application.

By consolidating these functions into a single platform, the system aims to improve administrative efficiency, enhance information accessibility, and provide a practical teacher management solution specifically tailored to the operational needs of Tadika Aqil Miqail.

2.4 Technology Used

2.4.1 Flutter

Flutter is an open-source UI software development kit (SDK) developed by Google for building cross-platform applications from a single codebase. It enables developers to create applications for Android, iOS, web, Windows, macOS, and Linux while maintaining consistent performance and user interface design. Flutter uses widget-based architecture, allowing developers to build responsive and visually appealing user interfaces efficiently.

One of Flutter's major advantages is its ability to compile applications into native machine code, resulting in high performance comparable to native applications. In addition, its Hot Reload feature enables developers to instantly view code changes without restarting the application, significantly reducing development time. Previous studies have reported that Flutter improves development productivity while maintaining application quality, making it one of the most widely adopted cross-platform development frameworks.

2.4.2 Dart

Dart is an object-oriented programming language developed by Google specifically for building modern applications. It is the primary programming language used by Flutter and supports both just-in-time (JIT) compilation during development and ahead-of-time (AOT) compilation for production deployment. These compilation methods enable faster development while maintaining high application performance.

Dart also supports asynchronous programming through features such as Futures and Streams, allowing applications to perform multiple operations efficiently without affecting user experience. Its simple syntax, strong typing, and rich standard libraries

make Dart suitable for developing scalable and maintainable mobile applications.

2.4.3 Firebase Authentication

Firebase Authentication is a cloud-based authentication service provided by Google that enables developers to implement secure user authentication without developing complex authentication mechanisms from scratch. It supports multiple authentication methods, including email and password, phone authentication, and third-party providers such as Google and Facebook.

Firebase Authentication provides built-in security features, user identity management, and integration with other Firebase services. These capabilities simplify application development while improving user account security and reducing the effort required to manage authentication processes.

2.4.4 Cloud Firestore

Cloud Firestore is a cloud-hosted NoSQL database provided by Firebase for storing and synchronizing application data in real time. Data is organized into collections and documents, providing a flexible database structure suitable for modern mobile applications. Unlike traditional relational databases, Firestore supports automatic synchronization between multiple devices, ensuring users always access the latest information.

Cloud Firestore also offers automatic scaling, offline data persistence, and strong security through configurable access rules. These features have made Firestore a popular database solution for cloud-based mobile applications that require reliable real-time data management.

2.4.5 Firebase Storage

Firebase Storage is a cloud storage service that enables applications to securely store and retrieve user-generated files such as images, documents, videos, and other multimedia content. It is designed to handle large file uploads efficiently while

maintaining high availability and secure access through Firebase Security Rules. Firebase Storage integrates seamlessly with other Firebase services and automatically manages file storage infrastructure, reducing the complexity of backend development. Its scalability and reliability make it suitable for applications that require secure cloud-based file management.

2.4.6 GitHub

GitHub is a web-based platform built on Git version control that supports collaborative software development. It enables developers to manage source code through repositories while tracking changes using commits, branches, and pull requests. These version control features facilitate collaboration among multiple developers and reduce conflicts during software development.

In addition to version control, GitHub provides project management features such as issue tracking, documentation, and workflow automation through GitHub Actions. These capabilities have made GitHub one of the most widely used platforms for collaborative software engineering projects in both academia and industry.

2.5 Chapter Summary

This chapter reviewed the fundamental theories, previous studies, and technologies relevant to the development of a Teacher Management System. The discussion began with the theoretical foundations, including Management Information Systems (MIS), Human Resource Management Systems (HRMS), Role-Based Access Control (RBAC), Cloud Computing, Mobile Application Development, and the Software Development Life Cycle (SDLC) using the Agile model. These concepts provide the theoretical basis for understanding modern information systems and software development practices.

The chapter also examined previous research and existing systems related to teacher management, school management, performance evaluation, and leave management. Through the comparison of these systems, several limitations were identified, particularly the lack of integration among essential teacher management functions.

These findings established the research gap that motivates the development of a more comprehensive teacher management solution.

Finally, the technologies adopted in this project, including Flutter, Dart, Firebase Authentication, Cloud Firestore, Firebase Storage, and GitHub, were reviewed based on their features and capabilities. Overall, the literature reviewed in this chapter provides a strong theoretical and technological foundation for the system design and implementation presented in the following chapter.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter describes the methodology used to develop GeniusAqilOS, a mobile-based Teacher Management System for Tadika Aqil Miqail. The methodology covers the overall development approach chosen, the justification for that choice, and the activities carried out within each phase of the project. It also describes the tools and technologies used and specifies the hardware and software requirements for both development and deployment.

GeniusAqilOS addresses the centre's reliance on paper-based and fragmented teacher administration by digitalising six core administrative processes: user registration and account management (Registration and Manage User modules), teacher personal records and documents (Teacher Records module), leave applications and entitlement tracking (Leave Management module), duty assignment and completion tracking (Task & Duty Manager module), teacher professional development tracking (Teacher Training Tracker module), teacher KPI evaluation (Teacher Performance Tracker module), and operational issue reporting (Teacher Reporting System module). The methodology adopted was selected to suit the iterative, stakeholder-driven nature of this project, given that the system supports three distinct user roles, namely Principal/Admin, Teacher/Employee, and Technician/Operational Admin, each requiring role-based access to a different subset of these modules.

3.2 Methodology Choice and Justification

Three common SDLC approaches were considered for this project: Waterfall, Prototyping, and Agile Scrum. The Waterfall model was ruled out early because it requires requirements to be fully defined and stable at the outset. For GeniusAqilOS, many details, such as the exact leave entitlement rules, KPI evaluation categories, and document handling workflows, were only confirmed progressively through meetings with the industry partner, making a rigid linear process impractical. Prototyping was considered useful for user interface design but insufficient as a standalone approach, as it lacks the structure needed to manage backend engineering, role-based security configuration, and the integration of seven interdependent modules.

Agile Scrum was selected as the most appropriate methodology for the following reasons:

- Requirements evolved throughout the project and needed to be refined sprint by sprint based on industry partner feedback, which Agile accommodates by design.
- The system's modules, namely Registration and Manage User, Teacher Records, Leave Management, Task & Duty Manager, Teacher Training Tracker, Teacher Performance Tracker, and Teacher Reporting System, mapped naturally to sprint deliverables, allowing each module to be built, tested, and demonstrated as a self-contained increment.
- Weekly check-ins and sprint reviews with Dr. Norhanisah Abdullah and Tadika Aqil Miqail provided the structured, frequent feedback loops that Agile relies on.
- Testing was integrated within each sprint rather than deferred to the end, ensuring defects were caught and resolved close to the point of introduction.
- Work could be distributed across the five team members by module, with each sprint producing an independently demonstrable module while still integrating with the shared Firebase backend.

The team operated on sprint cycles of approximately three to four weeks. Each sprint began with planning, proceeded through daily asynchronous stand-ups, and concluded with a review session where a working application increment was demonstrated to the industry partner, followed by a retrospective.

Table 3.1: Summary Comparison of SDLC Approaches

Criteria	Waterfall	Prototyping	Agile Scrum (Selected)
Flexibility to changing requirements	Low	Medium	High
Stakeholder engagement	Front-loaded only	During prototype reviews	Continuous via sprint reviews
Testing integration	End-stage only	Minimal	Within every sprint
Modular development support (7 modules)	Low	Low	High

Suitability GeniusAqilOS	for	Not suitable	Partially suitable	Most suitable — selected
-----------------------------	-----	--------------	--------------------	--------------------------

3.3 Phases of the Chosen Methodology

GeniusAqilOS was developed across five phases: Requirements, Design, Implementation, Testing, and Deployment. In keeping with Agile Scrum, these phases were not strictly sequential; design, implementation, and testing activities occurred iteratively within each sprint. The sections below describe the activities and processes carried out within each phase of this project.

3.3.1 Requirements Phase

The requirements phase focused on understanding the operational context of Tadika Aqil Miqail and establishing a precise, agreed-upon set of system requirements before development began. The process involved a project kick-off meeting with the principal, Dr. Norhanisah Abdullah, followed by structured and unstructured interviews conducted for each of the system's seven modules, supported by a review of existing physical documents such as leave request forms, medical certificates, cleaning schedules, and training records.

From these sessions, the team documented functional requirements for each module. For the Registration and Manage User modules, requirements covered teacher self-registration with principal approval, technician-created accounts, and role assignment, search, deactivation, and removal of user records. For the Teacher Records module, requirements covered personal and employment data fields together with document categories such as MyKad copy, passport photograph, resume, academic certificates, medical check-up report, and bank statement header. For the Leave Management module, requirements covered ten leave types, including annual, medical, unpaid, maternity, marriage, compassionate, Umrah, Haji, birthday, and half-day leave, together with automatic balance calculation and an approval workflow. For the Task & Duty Manager module, requirements covered duty types such as cleaning duty, arrival duty, dismissal duty, half-full day transition duty, and assembly duty, together with checklist tracking, proof-photo upload, and duty switch requests. For the Teacher

Training Tracker module, requirements covered training record fields such as course title, category, organiser, date, duration, mode, venue, and reflection, together with evidence upload. For the Teacher Performance Tracker module, requirements covered KPI categories such as attendance, classroom management, teaching performance, documentation, communication, duty responsibility, creativity, self-development, and SOP compliance, together with draft saving and feedback release. For the Teacher Reporting System module, requirements covered issue categories such as safety hazards, facility maintenance, teaching material shortages, IT problems, SOP violations, workload stress, staff conflict, and misconduct, together with anonymous submission and technician assignment.

Non-functional requirements were also captured, including compatibility across mobile devices, secure role-based authentication, real-time data synchronisation within two seconds, and support for concurrent multi-user access by the Principal, Teachers, and the Technician. All requirements were written as user stories and prioritised in a product backlog by business value and technical dependency. The product backlog served as the primary reference for all subsequent sprint planning sessions.

Outputs: Product Backlog, Requirements Specification Document, Functional and Non-Functional Requirements Tables all presented in full in Chapter 4.

3.3.2 Design Phase

The design phase translated the gathered requirements into blueprints guiding implementation. Activities were distributed across system architecture, data modelling, and interface design. The detailed design artefacts produced in this phase, including the system architecture diagram, use case diagrams, activity diagrams, sequence diagrams, database design, and interface designs for all seven modules, are presented in full in Chapter 4.

In this phase, the team agreed on the overall three-tier system architecture: a Flutter presentation layer, an application logic layer handling authentication and role-based access control through Firebase Authentication, and a Cloud Firestore and Firebase Storage data layer. User interface designs were produced for each of the three role-based dashboards (Principal/Admin, Teacher/Employee, and Technician/Operational

Admin) and for every module screen, from the login and registration interfaces through to the teacher reporting and technician resolution interfaces. A project Gantt chart was developed to schedule all development activities across the project timeline, mapping tasks against calendar weeks. Figure 3.1 presents the Gantt chart.

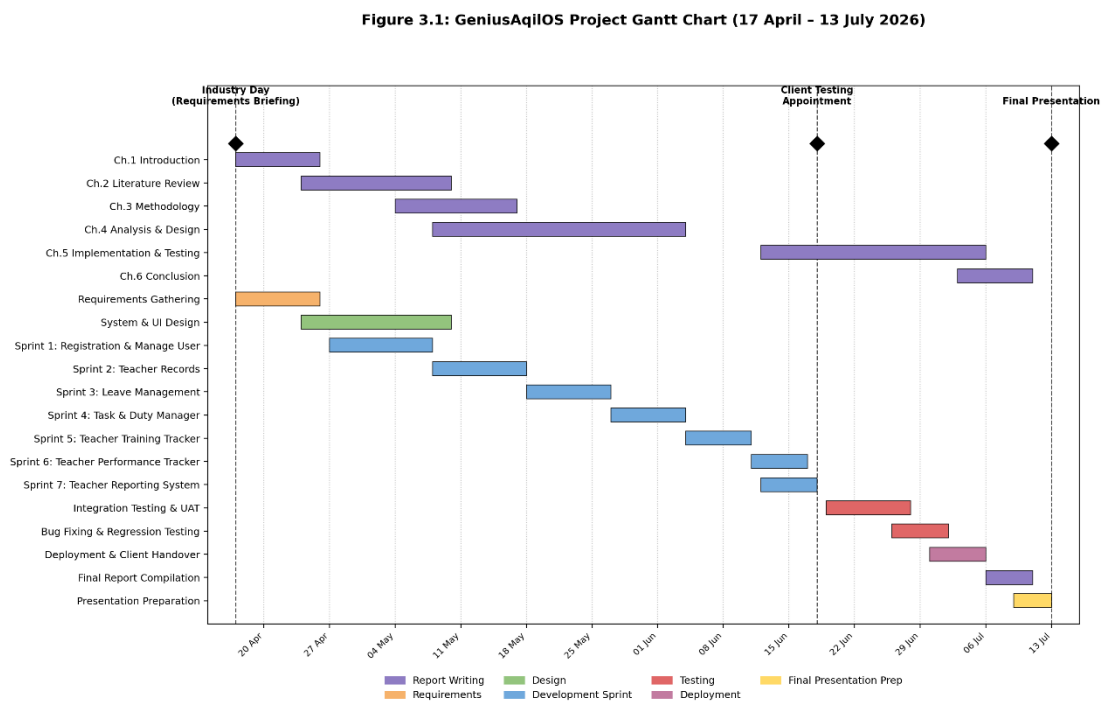


Figure 3.1: GeniusAqilOS Project Gantt Chart

Outputs: System Architecture Diagram, Use Case and Activity Diagrams, Sequence Diagrams, Database Design, Interface Designs, Gantt Chart. (Full artefacts in Chapter 4.)

3.3.3 Implementation Phase

Implementation was carried out across sprints, with each sprint delivering one or two functional modules according to their scope and interdependency. Development used Flutter and Dart for the mobile front end and Firebase for all cloud back-end services. Source code was managed on GitHub using a branching strategy consisting of a stable “main” branch, a “development” integration branch, and individual “feature/[name]” branches merged via pull request after peer review.

Sprint 1 delivered the Registration and Manage User modules, covering Firebase Authentication setup for the three user roles, the teacher self-registration and principal-approval workflow, technician account creation, and the administrative interface for

viewing, searching, filtering, updating the role of, deactivating, and removing user accounts, with all actions recorded in an audit trail.

Sprint 2 delivered the Teacher Records module, covering teacher profile creation and editing with all required personal and employment data fields, document upload to Firebase Storage across all six document categories, and a document completeness indicator for the principal.

Sprint 3 delivered the Leave Management module, including a dynamic leave application form supporting all ten leave types, automatic leave balance calculation and decrement on approval, a principal approval and rejection workflow, and leave report generation for administrative monitoring.

Sprint 4 delivered the Task & Duty Manager module, covering duty creation and assignment by the principal, an interactive duty calendar, a teacher-facing checklist and proof-photo upload interface, duty completion marking, and a duty switch request and approval workflow between teachers.

Sprint 5 delivered the Teacher Training Tracker module, covering training record submission and evidence upload by teachers, and search, filtering, and summary monitoring of training records by the principal.

Sprint 6 delivered the Teacher Performance Tracker module, covering KPI-based evaluation creation, scoring, and draft saving by the principal, together with evaluation release and feedback viewing by teachers.

Sprint 7 delivered the Teacher Reporting System module, covering normal and anonymous report submission with evidence upload by teachers, report review, status update, and technician assignment by the principal, and assigned-report handling and resolution by the technician.

Outputs: Flutter/Dart source code that includes all seven modules, Firebase Firestore and Storage configuration, Firebase Security Rules, GitHub repository.

3.3.4 Testing Phase

Testing was integrated throughout each sprint rather than conducted only at the end. Within each sprint, individual features were tested immediately after implementation before being merged into the development branch. A formal testing effort was also conducted after all seven implementation sprints were complete.

Unit testing covered critical business logic functions across all modules, such as leave balance calculation, document upload validation, KPI score aggregation, and duty checklist completion. Integration testing verified cross-module interactions, such as a leave approval updating a teacher's balance, a resolved duty switch request updating both teachers' schedules, and a principal-assigned report becoming visible in the technician's task queue. User Acceptance Testing (UAT) was conducted at Tadika Aqil Miqail with the principal and volunteer teachers, following a structured test script covering the primary user journeys of every module. Feedback was categorised and all critical defects were resolved before deployment. Regression testing was then performed across all modules to confirm that fixes did not introduce new failures.

Outputs: Unit Test Results, Integration Test Report, UAT Script and Findings, Bug Log, Regression Test Checklist.

3.3.5 Deployment Phase

The application was compiled into a release-mode Android package, with code obfuscation applied. Firebase Firestore security rules were hardened for the production environment, enforcing strict role-based access permissions across the Principal, Teacher, and Technician roles for all seven modules. Administrator and technician accounts were provisioned for Tadika Aqil Miqail's staff. An onboarding session was held at the centre where existing teacher data was entered into the system and staff were walked through each module. Handover documentation, including a Principal (Admin) User Manual, a Teacher User Guide, a Technician User Guide, and a System Configuration Document, was delivered to the industry partner.

Outputs: Release application package, Firebase Production Configuration, User Manuals, System Configuration Document.

3.4 Technology Used Description

The following tools and technologies were used in the development of GeniusAqilOS. Each was chosen for its technical suitability, compatibility with other components in the stack, and the team's familiarity.

Table 3.2: Tools and Technologies Used in GeniusAqilOS

No.	Tool / Technology	Version	Role
1	Flutter	3.x (stable)	Cross-platform mobile application framework; primary front-end layer for all seven modules.
2	Dart	3.x	Programming language for all Flutter application logic, form validation, and UI.
3	Firebase Firestore	Cloud (NoSQL)	Real-time NoSQL database storing teacher profiles, leave applications, duty assignments, training records, performance evaluations, and reports.
4	Firebase Storage	Cloud	Cloud object storage for uploaded documents, training evidence, report evidence, and duty proof photos.
5	Firebase Authentication	Cloud	User identity management and role-based access control for Principal/Admin, Teacher/Employee, and Technician roles.
6	GitHub	Web-based	Version control and collaborative source code management across feature branches for all modules.
7	Figma	Web-based	UI/UX wireframing and high-fidelity interface design for all module screens.
8	Visual Studio Code	1.x	Primary IDE for Flutter/Dart development with integrated debugging.
9	Android Studio	Latest stable	Android emulator management and release

			build generation.
--	--	--	-------------------

3.5 System Requirement Analysis: Hardware and Software

This section specifies the hardware and software requirements for the development environment and for the operational environment at Tadika Aqil Miqail. Requirements are listed and justified based on the demands of the development tools, the Firebase platform, and the target mobile devices used by the Principal, Teachers, and Technician.

3.5.1 Hardware Requirements

Table 3.3: Hardware Requirements for Development Environment

No.	Component	Minimum Specification	Justification
1	Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5, 64-bit	Flutter build compilation and Android emulation across seven modules are CPU-intensive and require a multi-core 64-bit processor at this tier for acceptable performance.
2	RAM	8 GB minimum; 16 GB recommended	Running VS Code, Android Studio, and an Android emulator concurrently requires at least 8 GB. 16 GB is recommended for stable multi-process performance.
3	Storage	20 GB free space (SSD recommended)	The Flutter SDK, Android SDK, emulator system images, and project dependencies require significant space. SSD reduces build and emulator load times.
4	Internet Connection	Broadband, minimum 10 Mbps	Continuous interaction with Firebase cloud services, package downloads, and GitHub operations all require a stable internet connection.

5	Test Device	Mid-range Android or iOS smartphone	A physical mobile device is required for user acceptance testing of role-specific workflows across all seven modules.
---	-------------	-------------------------------------	---

Table 3.4: Hardware Requirements for Operational Environment (Tadika Aqil Miqail)

No.	Component	Minimum Specification	Justification
1	Mobile Device	Modern Android or iOS smartphone	The Principal, Teachers, and Technician each require a mobile device to access their respective role-based dashboards and modules.
2	Internet Connection	Stable mobile data or Wi-Fi	All application operations require internet access for real-time communication with Cloud Firestore and Firebase Storage across every module.

3.5.2 Software Requirements

Table 3.5: Software Requirements for Development Environment

No.	Software	Version	Justification
1	Flutter SDK	3.x stable	Core framework for building GeniusAqilOS. The stable channel ensures a production-ready, tested build environment.
2	Dart SDK	3.x (bundled with Flutter)	Programming language runtime; bundled with Flutter and listed separately for clarity.
3	Android SDK	API Level 26–34	Required to compile, emulate, and run the application across supported Android versions.

4	Visual Studio Code	1.x latest stable	Primary IDE with Flutter and Dart extensions for language support, debugging, and hot-reload.
5	Android Studio	Latest stable	Required for Android emulator creation and release build generation.
6	Firebase CLI	Latest stable	Used to manage Firebase project configuration and deploy Firestore security rules from the development machine.
7	Git	2.x	Local version control system enabling commits, branching, and push/pull with the GitHub repository.

Table 3.6: Software Requirements for Operational Environment (End User)

No.	Software	Specification	Justification
1	Mobile Operating System	Current supported Android/iOS version	The application must be installed on a compatible operating system version to support all seven modules correctly.
2	GeniusAqilOS Application	Current release version	The application package must be installed on each user's device according to their assigned role.
3	Google Play Services / equivalent	Current version	Required for Firebase Authentication session management. Present on standard mobile devices by default.

3.6 Chapter Summary

This chapter presented the system development methodology for GeniusAqilOS. Agile Scrum was selected over Waterfall and Prototyping because it best accommodated the project's evolving requirements, the seven-module architecture spanning Registration

and Manage User, Teacher Records, Leave Management, Task & Duty Manager, Teacher Training Tracker, Teacher Performance Tracker, and Teacher Reporting System, and the frequent stakeholder feedback sessions with Tadika Aqil Miqail and Dr. Norhanisah Abdullah. Development proceeded across five phases, namely Requirements, Design, Implementation, Testing, and Deployment, executed iteratively through sprints that each delivered one or more functional modules. The technology stack (Flutter, Dart, Firebase, GitHub, and Figma) and the hardware and software requirements for both development and deployment were also documented. The detailed design artefacts produced during the design phase, consisting of the system architecture, use case, activity, and sequence diagrams, database design, and interface designs for all seven modules, are presented in Chapter 4.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter focuses on the requirements analysis and design phase of the GENIUSAQILOS teacher management system. The main objective of this phase is to transform the identified operational inefficiencies at the Tadika Akil Michael Kindergarten into a structured and technical blueprint. This section provides a detailed description of the case study context, organizational structure, requirements elicitation methods, and clearly defines both functional and non-functional requirements. Additionally, it defines the system architecture, component decomposition, and data management mechanism required to support the digital transformation of kindergarten management.

4.2 System Analysis

4.2.1 Case Study

The case study centers on the operational ecosystem of Tadika Aqil Miqail, a professional early childhood education center managing teachers under the direct leadership of its principal. Although the human resource scale is specialized, the administrative workload still heavily relies on manual operations.

Daily activities including detailed teacher profiles, physical qualification documents, leave requests, daily operational duty rosters, and annual professional development metrics are all recorded through isolated paper-based files or localized spreadsheets. The lack of a centralized data repository has led to several vulnerabilities:

- **Search Delay:** The principal must manually search physical folders to audit teacher files or professional certifications.
- **Operational Transparency:** The leave balances are updated manually, increasing the possibility of tracking discrepancies and computational errors.
- **Uneven Task Distribution:** The class assignments and activity lists lack historical records, often leading to unbalanced workloads.

To address these issues, the proposed Geniusaqilos platform replaces manual

execution loops with an end-to-end digital pipeline. This platform operates on an elastic cloud infrastructure and offers features such as structured data integrity, real-time document search, and real-time monitoring.

4.2.2 Company Organization Structure

The operational hierarchy of Tadika Akil Mikelle operates through a structured chain of command, coordinating leadership, teaching implementation, and administrative management.

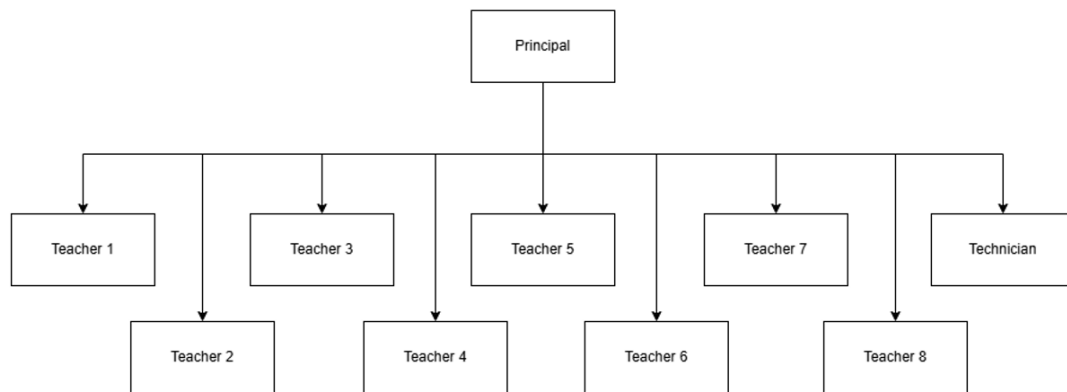


Figure 4.1: Company Organization Structure Diagram

As shown in Figure 4.1, the principal holds the core management position, overseeing the institution's strategic planning, quality control, personnel performance evaluation, and final leave approval. Below him, eight teachers act as the daily managers of the executive level, responsible for teaching implementation, student safety, and classroom management.

To support the migration of these manual workflow processes to the cloud environment, the organizational structure has added a third level of independent stakeholders: technician. Although these technical personnel are not part of the regular daily teaching staff of the kindergarten, they act as external operational roles, responsible for managing the digital infrastructure, adjusting the system permissions of high-level back-end users, and resolving the bottlenecks of the upgraded software.

In addition, the technical personnel play a key role in the teacher reporting system module of the platform. They are directly assigned by the principal and are responsible for investigating and recording the handling situations of facility structure abnormalities, safety hazards, and equipment failures reported by teaching staff. The Geniusaqilos platform implements role-based access control (RBAC) to fully map this

organizational structure, ensuring that the management processes naturally fit this verticalized reporting and maintenance structure.

4.2.3 System Requirements Gathering Technique

To compile a complete system specifications list, a structured requirements gathering process was implemented. This process focused on accurately identifying the users' pain points and converting them into technical data components, ensuring that each function of the new system could directly address real and existing practical needs, rather than being based on assumptions.

Structured and Unstructured Interviews

The main engineering tools used for obtaining main knowledge and business logic rules are a series of consultations conducted directly with the principal stakeholders of Tadika Aqil Miqail, Dr. Norhanisah Abdullah. These meetings combined a structured agenda with an unstructured exploration cycle.

The structured part includes a set of pre-written technical questions focused on strict organizational indicators, including compliance with employee historical records, exact annual leave allowances, mathematical parameters of key performance indicators (KPIs), and the current training hours target for each teacher. While the unstructured part allows stakeholders to express complex operational realities, such as the qualitative impact of uneven workload distribution on teacher morale, and the communication delay problems caused by manual message transmission.

The insights gained from these meetings enabled the development team to define precise access control permissions for the platform's user roles and determined the necessary conditional routing paths for the automated leave authorization system.

Document and Artifact Analysis

To ensure that the Geniusaqilos platform aligned with organizational standards, the team reviewed physical documents such as leave requests, medical certificates, cleaning schedules, and training records. This on-site review helped the developers understand the existing workflow and identify data patterns, thereby providing a basis for system design.

Each item was broken down into separate data entities to define the architecture

requirements for cloud storage. Firstly, the leave request revealed validation rules for fields such as date, phone number, and description, and encoded them into the mobile input interface. Besides that, analysis of the duty schedule helped to clarify the logical relationships in the automated scheduling engine, linking teacher IDs with calendar dates and facility locations.

Through this analysis, the team able to accurately converted paper documents into a structured JSON format suitable for Cloud Firestore. Each form field became a database entry, and each cross-reference became an association relationship, preserving the existing workflow while eliminating the inefficiencies brought about by paper documents, thus achieving a faster and more reliable management experience.

4.3 System Requirements

The data collected during the requirement engineering phase was sorted into definitive software engineering matrices. These are presented below as explicit Functional Requirements (FR) and Non-Functional Requirements (NFR).

4.3.1 Functional Requirements

Functional requirements define the core operational capabilities, actions, and calculation loops that the *Geniusaqilos* application code must execute to support individual user actions.

Table 4.3.1: Functional Requirements Table

Requirement ID	Module Name	Primary Actor	Functional Description & Technical Specification
FR-M1-01	Authentication	All Users	The system must authenticate the user session by using the email and password parameters through the Firebase authentication endpoint.
FR-M1-02	Access Routing	All Users	The system must evaluate the user role token (Principal, Teacher, Technician) when the user logs in, and route the

			session to the corresponding dashboard based on the user's role.
FR-M1-03	Profile Management	Teacher	The system must allow teachers to edit their personal contact information and upload their qualification documents directly to the cloud.
FR-M1-04	Profile Control	Principal	The system must grant the principal the authority to create, update, search and partially delete the individual teacher profile records.
FR-M2-01	Leave Submission	Teacher	The system must provide a data entry form to fill for leave dates, categories, text descriptions, and binary file uploads for medical certificates.
FR-M2-02	Balance Evaluation	Teacher	The system must perform cloud-side calculation loops to display an accurate summary of available leave balances prior to submission.
FR-M2-03	Leave Processing	Principal	The system must provide the principal with interface controls to instantly approve or reject pending leave entries via real-time data synchronization.

FR-M3-01	Duty Allocation	Principal	The system must provide an interactive calendar matrix so that the principal can assign daily tasks and activity responsibilities to the staff.
FR-M3-02	Roster Viewing	Teacher	The system must extract active data strings from the database in order to provide teachers with personalized task schedules.
FR-M4-01	Evaluation Entry	Principal	The system must provide data fields for the principal to submit numeric metrics and qualitative feedback based on teacher KPIs.
FR-M4-02	Performance Review	Teacher	The system must securely present the summary of each teacher's assessment results while blocking access of the records from other teachers at the same level.
FR-M5-01	Development Logging	Teacher	The system must provide input screens for teachers to log completed workshops, course hours, and training credentials.
FR-M5-02	Training Analytics	Principal	The system must compile the accumulated training hours across the workforce and present them through an administrative analytics panel.

FR-M6-01	Facility Logging	Teacher	The system must allow teachers to submit technical, safety, or facility maintenance tickets directly into the platform tracking queue.
FR-M6-02	Issue Resolution	Technician	The system must provide the technician with database update tools to change the status, notes, and completion parameters of reported logs.

4.3.2 Non-Functional Requirements

Non-functional requirements specify the operational constraints, performance thresholds, and security features necessary to maintain system stability and provide a reliable user experience.

Table 4.3.2: Non-Functional Requirements Table

Requirement ID	Quality Attribute	Target Metric / System Constraint	Detailed Specification & Implementation Parameters
NFR-SEC-01	Security	Cryptographic Protection	All user passwords must be hashed and managed through the server-side mechanism of Firebase Auth. Direct storage in plain text is prohibited.

NFR-SEC-02	Security	Rule Enforcement	The database access paths in Cloud Firestore must be controlled by backend security rules to limit data isolation boundaries through roles.
NFR-PER-01	Performance	Query Latency	The read and write operations performed on the Firestore collection must be completed and updated in the interface within 2.0 seconds.
NFR-PER-02	Performance	Stream Refresh	The real-time listeners that are deployed in the Flutter UI layer must update the data display within 1000 milliseconds after the backend has made the changes.
NFR-AVA-01	Availability	Uptime Threshold	System-dependent backend web services and hosting clusters must maintain an availability of 99.9% during their operational period.
NFR-USA-01	Usability	Cross-Platform Sync	The user experience and component styles must be adapted to different mobile layouts and must follow the Google Material 3 UI design

			guidelines.
NFR-MAI-01	Maintainability	Logic Decoupling	The application code library must separate the modifications to the core state from the visual components to ensure the easy scalability of future modules.

4.4 Current System Analysis (Manual/any improvement)

Prior to the development of GENIUSAQILOS, Tadika Aqil Miqail managed all teacher-related administrative tasks manually or through scattered, non-integrated documents such as physical files, spreadsheets, and printed forms. Teacher personal information, employment documents, leave records, duty schedules, training records, performance evaluations, and internal reports were each maintained separately, often by different staff members and in different formats. This fragmented approach required the principal to manually cross-reference multiple sources whenever a decision needed to be made, such as verifying a teacher's leave balance before approving a new application or confirming that all mandatory documents had been submitted.

The manual system presented several operational weaknesses. First, retrieving a specific teacher's record required searching through physical folders or multiple spreadsheet files, which was time-consuming and prone to human error. Second, leave balances were updated manually after each approval, which increased the risk of inconsistent or outdated records and made it difficult to detect discrepancies. Third, daily duty assignments were communicated informally, often through verbal instructions or paper notices, which sometimes resulted in unclear responsibility or missed duties. Fourth, training attendance and performance evaluation records were not

centrally stored, making it difficult for the principal to track a teacher's development over time or generate consolidated reports. Finally, internal reports submitted by teachers, such as facility issues or IT problems, lacked a formal tracking mechanism, so the principal or technician could not easily monitor the status of a reported issue or ensure timely follow-up.

These limitations directly informed the design of GENIUSAQILOS. By replacing the manual and fragmented workflow with a centralized, role-based digital platform built on Flutter and Firebase, the proposed system addresses each of the weaknesses identified above. Teacher records, leave applications, duty assignments, training logs, performance evaluations, and internal reports are consolidated into a single Cloud Firestore database, allowing real-time updates, automatic leave balance calculation, structured duty tracking, and status-based report monitoring. This improvement reduces administrative workload, minimizes the risk of data loss or inconsistency, and provides the principal with faster and more reliable access to teacher information.

4.5 System Design

4.5.1 System Architecture

GENIUSAQILOS adopts a three-tier client-server architecture consisting of the presentation layer, the application logic layer, and the data layer, which together support real-time synchronization across the three user roles: Principal/Admin, Teacher/Employee, and Technician/Operational Admin. The overall structure of this architecture is illustrated in Figure 4.5.1.1.

The presentation layer is developed using Flutter, allowing the system to be deployed as a cross-platform application that runs consistently across mobile and web devices from a single Dart codebase. This layer is responsible for rendering the user interface, capturing user input, and displaying real-time data updates through Flutter's widget-based rendering system.

The application logic layer sits between the presentation layer and the data layer. It handles authentication and session management through Firebase Authentication, enforces role-based access control so that each user type can only access the modules

relevant to their role, and manages the business logic required for features such as leave balance calculation, duty scheduling, and report status tracking.

The data layer is built on Cloud Firestore, a NoSQL, document-based cloud database that stores all structured data, including teacher profiles, leave records, duty assignments, training records, performance evaluations, and internal reports. Firebase Storage complements Firestore by storing unstructured file data, such as MyKad copies, passport photographs, resumes, academic certificates, medical check-up reports, and bank statement headers. Firestore's real-time listener capability ensures that any change made by one user, such as a leave approval by the principal, is immediately reflected on the relevant teacher's interface without requiring a manual refresh.

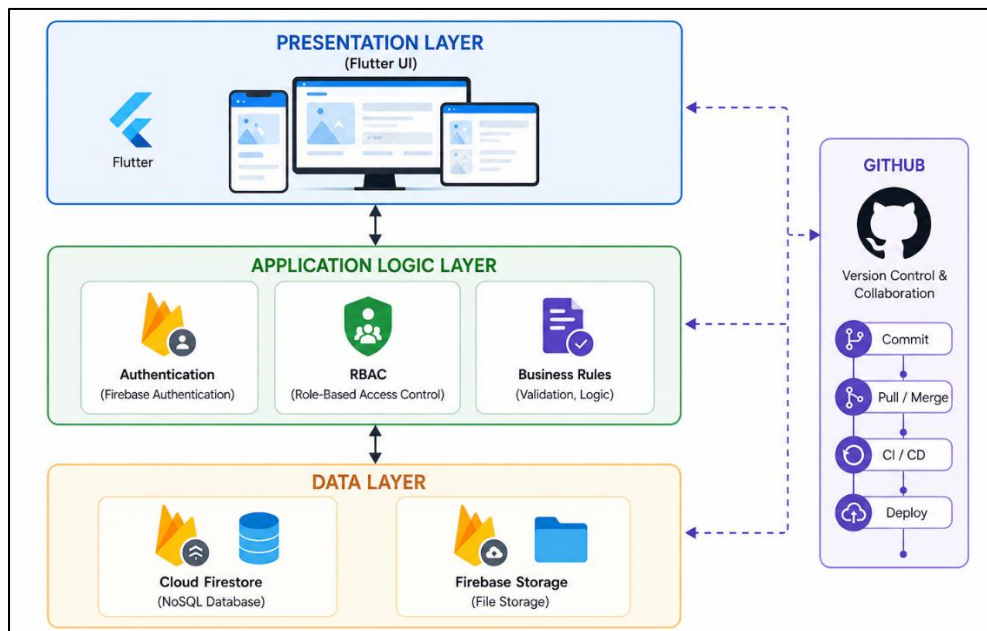


Figure 4.5.1.1: System Architecture Diagram

4.5.2 Explain each component in system architecture

Flutter serves as the front-end framework responsible for building the user interface across all three user roles in the presentation layer. It renders role-specific dashboards, forms for data entry such as leave applications and training records, and real-time views of teacher records, duty rosters, and report statuses. Flutter's widget-based structure allows user interface components to be reused across modules, thereby ensuring visual consistency throughout the system.

Dart is the programming language underlying the Flutter application. It manages client-side logic such as form validation, state management, and the construction of data objects before they are sent to Cloud Firestore, ensuring that data submitted by users conforms to the expected structure.

Within the application logic layer, Firebase Authentication manages user identity verification during login and registration. It issues secure authentication tokens that are used to determine a user's role, whether Principal/Admin, Teacher/Employee, or Technician/Operational Admin, and to restrict access to modules and data that are not relevant to that role.

Within the application logic layer, Role-Based Access Control ensures that each authenticated user is only permitted to view or modify the data and functions assigned to their role. For example, only the Principal/Admin is able to approve leave applications, whereas Teachers/Employees can only submit their own leave requests and view their own records.

Cloud Firestore forms the central component of the data layer and functions as the NoSQL database that stores all structured records, including teacher profiles, leave applications and balances, duty assignments, training logs, performance evaluations, and internal reports. Its real-time listener feature allows any update to be propagated instantly to all connected clients, which supports the system's requirement for immediate data synchronization.

Firebase Storage complements Cloud Firestore within the data layer by storing unstructured files that accompany teacher records, such as identification documents, certificates, and photographs. These files are referenced within their corresponding Firestore documents through secure download URLs, allowing structured metadata and unstructured files to remain logically linked.

Finally, GitHub provides version control for the development of GENIUSAQILOS, allowing the development team to track code changes, manage feature branches, and maintain a reliable history of the application's codebase throughout the development lifecycle.

4.5.3 System Design

4.5.3.1 Use Case Diagram

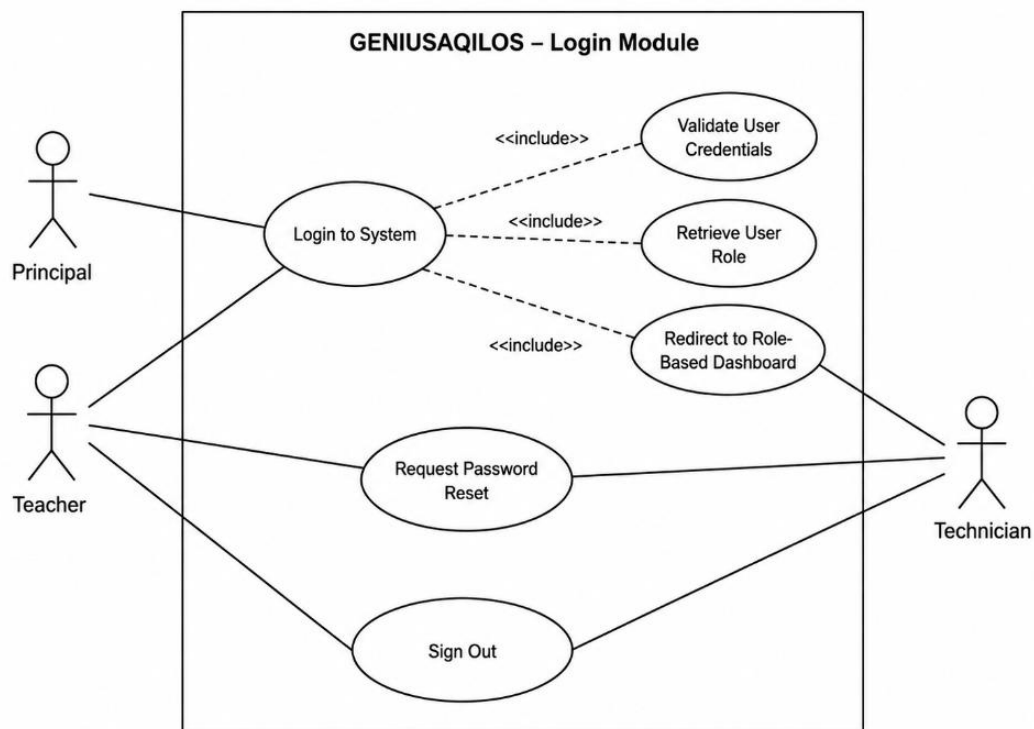


Figure 4.5.3.1.1 Use Case Diagram for Login Module

Table 4.5.3.1.1.1 Login and Authentication Use Case Description

- Login to System, Validate User Credential, Role-Based Dashboard Redirect, Forgot Password, Sign Out

Item	Description
Use Case ID	UC-LM-01
Use Case Name	Login and Authentication
Actor	Principal, Teacher, Technician
Description	This case describes how authorized users access the GENIUSAQILOS system by logging in with valid credentials. The system validates the user credentials, retrieves the user role, redirects the user to the correct role-based dashboard, allows password reset

	requests, and supports signing out from the system.
Trigger	The user opens the GENIUSAQILOS application and selects the login function.
Pre-condition	The user must have a registered account in the system. The system must be connected to the authentication database.
Main flow	1. The user enters email and password. 2. The system validates the user credentials. 3. The system retrieves the user role after successful authentication. 4. The system redirects the user to the correct role-based dashboard. 5. The user may continue using the system or sign out after completing their task.
Alternative flow	1. If the user enters invalid credentials, the system displays an error message and asks the user to try again. 2. If the user forgets the password, the user can request a password reset. 3. If the system cannot retrieve the user role, access is denied until the account role is verified.
Post-condition	The user is successfully authenticated and redirected to the correct dashboard, or the login attempt is rejected. If the user signs out, the session is terminated securely.

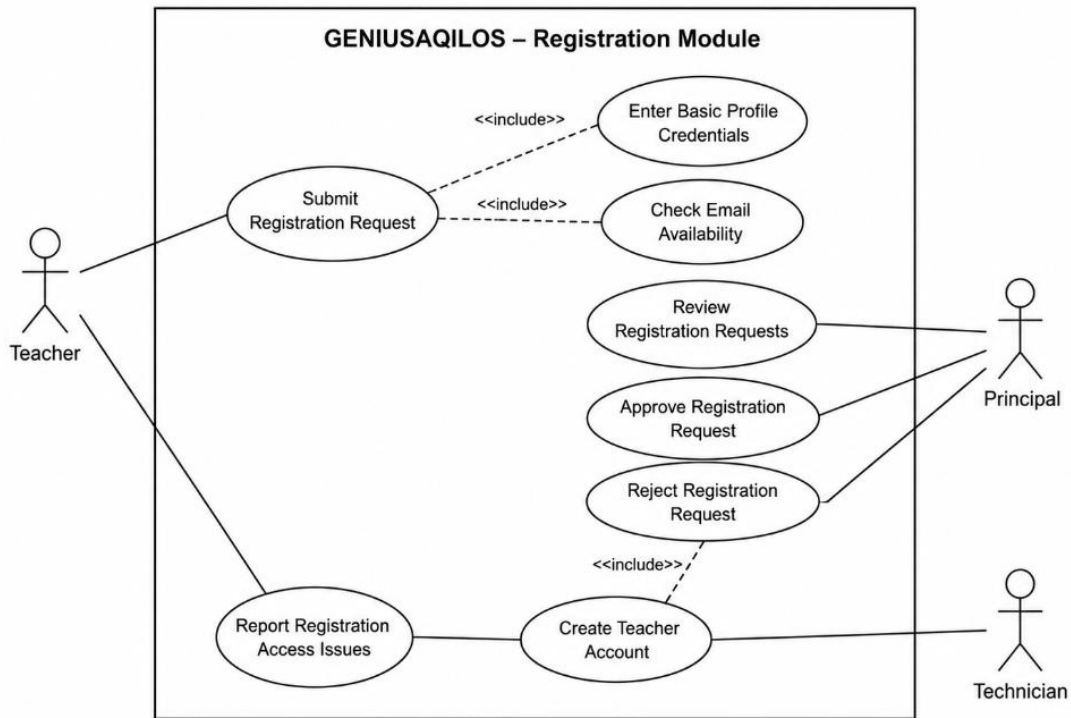


Figure 4.5.3.1.2 Use Case Diagram for Registration Module

Table 4.5.3.1.2.1 User Registration Use Case Description

- Submit Registration Request, Validate Registration Information, Review Registration Request, Approve Registration, Reject Registration

Item	Description
Use Case ID	UC-RM-02
Use Case Name	User Registration
Actor	Teacher, Principal, Technician
Description	This case describes the process of registering a new teacher account in the GENIUSAQILOS system. A teacher submits a registration request by entering basic profile credentials. The system checks email availability and forwards the registration request for review. The principal reviews the request and decides whether to approve or reject it. If approved, the technician creates the teacher's account. If issues occur during the process, the teacher may report registration access issues.

Trigger	A teacher selects the registration function to request access to the system.
Pre-condition	The teacher does not yet have an account in the system. The registration function and database must be available. The principal and technician must have access to manage registration requests.
Main flow	1. The teacher submits a registration request. 2. The teacher enters basic profile credentials. 3. The system checks email availability and validates the registration information. 4. The registration request is sent to the principal for review. 5. The principal reviews the registration request. 6. The principal approves the registration request. 7. The technician creates the teacher account. 8. The teacher is informed that the registration has been successfully completed.
Alternative flow	1. If the entered email is already in use, the system displays an error and asks the teacher to enter another email. 2. If the registration information is incomplete or invalid, the system asks the teacher to correct and resubmit the request. 3. If the principal rejects the registration request, the request is marked as rejected and the teacher is notified. 4. If the teacher faces access problems during registration, the teacher may report registration access issues for further assistance.
Post-condition	The registration request is either approved and the teacher account is created or rejected and no account is created.

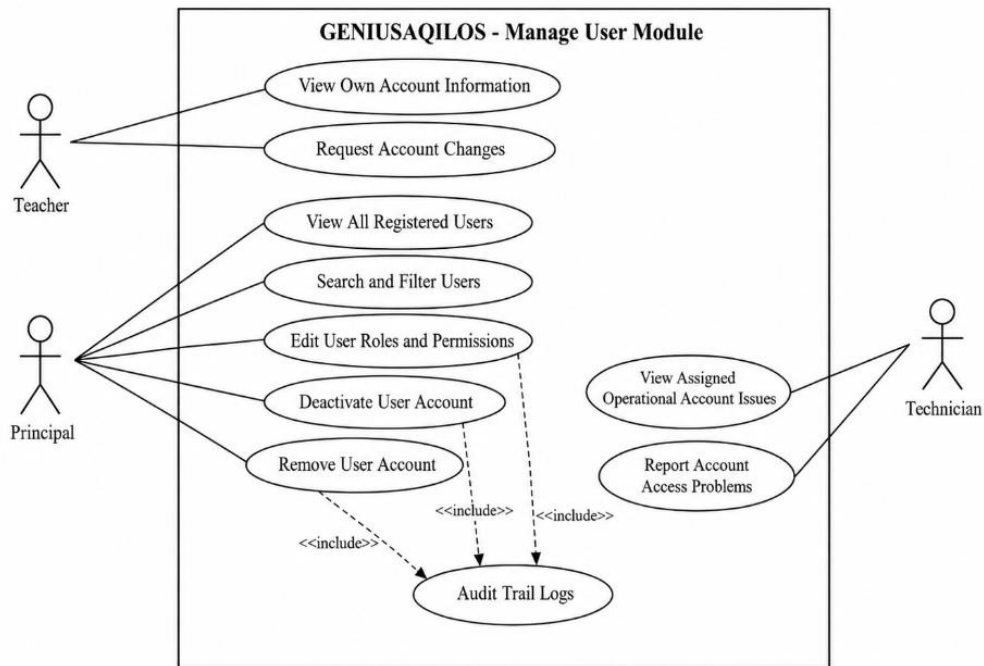


Figure 4.5.3.1.3 Use Case Diagram for Manage User Module

Table 4.5.3.1.3.1 Teacher Profile Management Use Case Description

- View All Registered Users, Search and Filter Users, View User Details, Update User Role, Deactivate User Account, Remove User Account

Item	Description
Use Case ID	UC-RM-03
Use Case Name	Manage User
Actor	Principal
Description	This case describes how the principal manages registered users in the GENIUSAQILOS system. The principal can view all registered users, search and filter user records, view user details, update user roles, deactivate user accounts, and remove user accounts when necessary. Selected actions are recorded in the audit trail logs for accountability and security purposes.
Trigger	The principal accesses the Manage User module to manage registered user accounts.
Pre-condition	The principal must be successfully logged into the system with authorized access rights. Registered user data must already exist in the

	system.
Main flow	1. The principal opens the Manage User module. 2. The system displays all registered users. 3. The principal may search and filter users based on relevant criteria. 4. The principal selects a specific user to view detailed information. 5. The principal may update the user's role and permissions if required. 6. The principal may deactivate a user account if the account should no longer be active. 7. The principal may remove a user account if it is no longer needed. 8. The system records the relevant actions in the audit trail logs.
Alternative flow	1. If no matching user is found, the system displays a "no record found" message. 2. If the selected user details cannot be retrieved, the system displays an error message. 3. If the role update is invalid or not permitted, the system rejects the update request. 4. If the account cannot be deactivated or removed due to system restrictions, the system notifies the principal.
Post-condition	User information is viewed successfully, or the selected user account is updated, deactivated, or removed. All important administrative actions are logged in the audit trail.

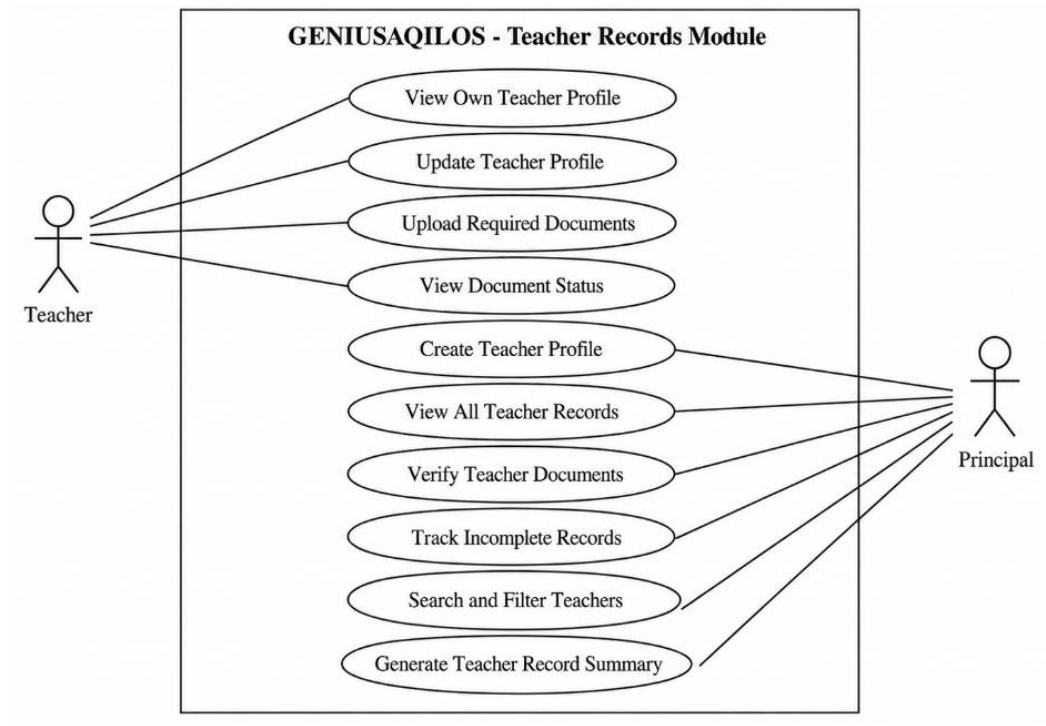


Figure 4.5.3.1.4 Use Case Diagram for Teacher Records Module

Table 4.5.3.1.4.1 Teacher Profile Management Use Case Description

- Create Teacher Profile, View All Teacher Records, View Own Teacher Profile, Update Teacher Profile

Item	Description
Use Case ID	UC-RM-04
Use Case Name	Teacher Profile Management
Actor	Teacher, Principal
Description	This case describes how teacher profile information is managed in the GENIUSAQILOS system. Teachers can view and update their own teacher profile, while the principal can create teacher profiles and view all teacher records in the system.
Trigger	A teacher or principal accesses the Teacher Records module to view, create, or update teacher profile information.
Pre-condition	The user must be logged into the system. The teacher must have an existing profile to view or update. The principal must have permission to create and view teacher records

Main flow	1. The teacher opens the Teacher Records module. 2. The system displays the teacher's own profile. 3. The teacher updates profile information if needed. 4. The system validates and saves the updated profile information. 5. The principal opens the Teacher Records module. 6. The principal creates a new teacher profile by entering the required information. 7. The principal views all teacher records stored in the system.
Alternative flow	1. If the teacher profile cannot be found, the system displays an error message. 2. If the updated information is incomplete or invalid, the system asks the teacher to correct it. 3. If a duplicate teacher profile already exists, the system rejects the creation request. 4. If the principal does not have permission, access to all teacher records is denied.
Post-condition	The teacher's profile is successfully viewed, updated, or created. The teacher records remain accurate and updated in the system.

Table 4.5.3.1.4.2 Teacher Document Management Use Case Description

- Upload Required Documents, View Document Status, Verify Teacher Documents, Track Incomplete Records

Item	Description
Use Case ID	UC-RM-05
Use Case Name	Teacher Document Management
Actor	Teacher, Principal
Description	This case describes how teacher documents are uploaded, viewed, verified, and monitored in the GENIUSAQILOS system. Teachers can upload required documents and view their document status, while the principal verifies submitted documents and tracks incomplete teacher records.

Trigger	A teacher uploads required documents, or the principal reviews teacher document submissions.
Pre-condition	The teacher must be logged into the system and have a teacher profile. The required document categories must be available in the system. The principal must have permission to verify teacher documents.
Main flow	1. The teacher opens the Teacher Records module. 2. The teacher uploads the required documents. 3. The system stores the uploaded documents and updates the document status. 4. The teacher views the current document status. 5. The principal reviews the submitted teacher documents. 6. The principal verifies whether the documents are complete and valid. 7. The principal tracks teacher records with missing or incomplete documents.
Alternative flow	1. If the uploaded document format is invalid, the system rejects the upload. 2. If required documents are missing, the system marks the record as incomplete. 3. If the document cannot be verified, the principal may keep the status as pending or request correction. 4. If no incomplete records are found, the system displays that all teacher records are complete.
Post-condition	Teacher documents are successfully uploaded, viewed, verified, or marked as incomplete. The document status is updated in the system.

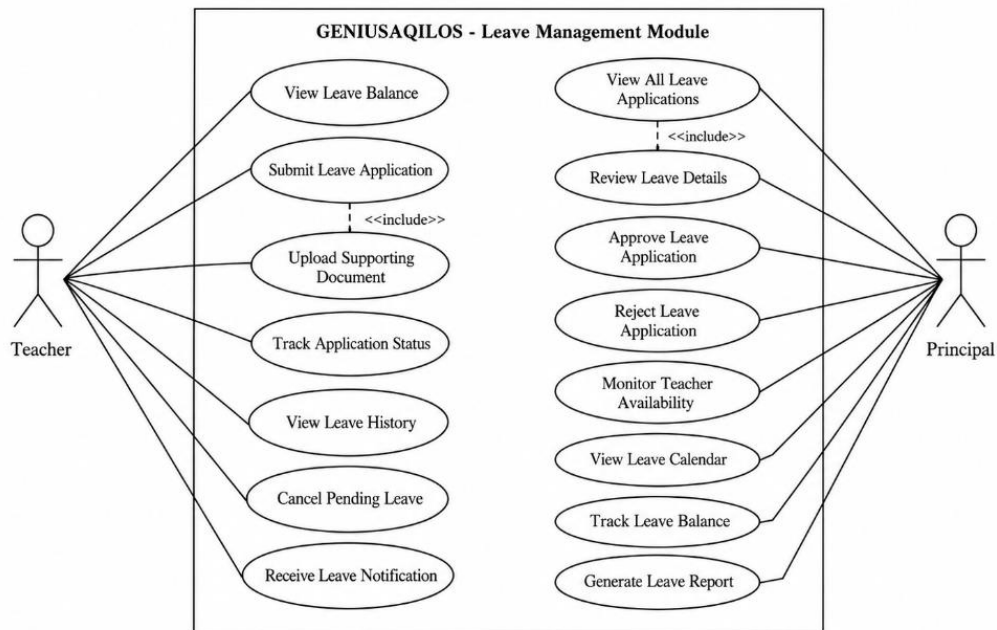


Figure 4.5.3.1.5 Use Case Diagram for Leave Management Module

Table 4.5.3.1.5.1 Leave Application Use Case Description

- View Leave Balance, Submit Leave Application, Upload Supporting Document, Track Application Status, Cancel Pending Leave

Item	Description
Use Case ID	UC-RM-06
Use Case Name	Leave Application Management
Actor	Teacher
Description	This case describes how teachers manage their leave applications in the GENIUSAQILOS system. Teachers can view their leave balance, submit leave applications, upload supporting documents, track the application status, and cancel pending leave applications when necessary.
Trigger	A teacher wants to apply for leave or manage an existing leave application.
Pre-condition	The teacher must be logged into the system. The teacher profile and leave balance must already exist in the system.

Main flow	1. The teacher opens the Leave Management module. 2. The teacher views the available leave balance. 3. The teacher submits a leave application by entering the required leave details. 4. The teacher uploads supporting documents if required. 5. The system stores the leave application and marks it as pending. 6. The teacher tracks the application status. 7. The teacher may cancel the leave application if it is still pending.
Alternative flow	1. If the teacher has insufficient leave balance, the system rejects the leave application. 2. If required leave details are incomplete, the system asks the teacher to complete the form. 3. If the supporting document is missing or invalid, the system asks the teacher to upload a valid document. 4. If the leave application has already been approved or rejected, the teacher cannot cancel it.
Post-condition	The leave application is submitted, tracked, or cancelled successfully. The application status and leave balance are updated in the system.

Table 4.5.3.1.5.2 Leave Approval and Reporting Use Case Description

- View All Leave Applications, Review Leave Details, Approve Leave, Reject Leave, Generate Leave Report

Item	Description
Use Case ID	UC-RM-07
Use Case Name	Leave Approval and Reporting
Actor	Principal
Description	This case describes how the principal manages teacher leave applications in the GENIUSAQILOS system. The principal can view all leave applications, review leave details, approve or reject leave requests, and generate leave reports for administrative monitoring.

Trigger	A teacher submits a leave application, or the principal accesses the Leave Management module to review and manage leave records.
Pre-condition	The principal must be logged into the system with authorized access. Leave applications must exist in the system for review, approval, rejection, or reporting.
Main flow	1. The principal opens the Leave Management module. 2. The system displays all leave applications. 3. The principal selects a leave application to review its details. 4. The principal checks the teacher's leave information and supporting documents. 5. The principal approves the leave application if it is valid. 6. The principal rejects the leave application if it is not valid or cannot be approved. 7. The system updates the leave application status. 8. The principal generates a leave report for record keeping and monitoring.
Alternative flow	1. If no leave applications are available, the system displays a "no record found" message. 2. If the leave details are incomplete, the principal may reject the application or keep it pending for correction. 3. If the supporting document is invalid, the leave application may be rejected. 4. If the report cannot be generated due to missing data, the system displays an error message.
Post-condition	The leave application is approved or rejected. The application status is updated, and the leave report is generated successfully when required.

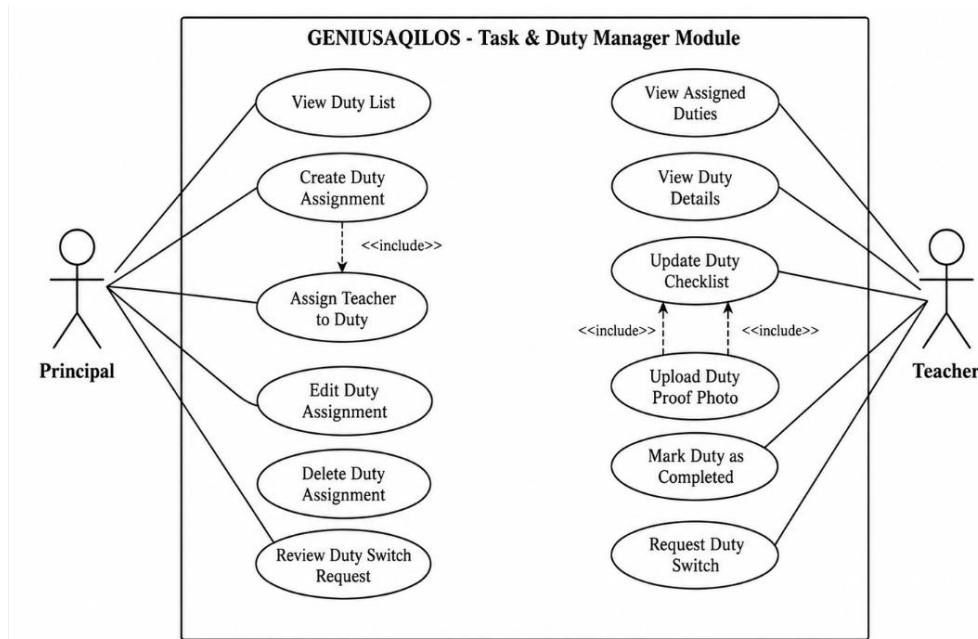


Figure 4.5.3.1.6 Use Case Diagram for Task & Duty Manager Module

Table 4.5.3.1.6.1 Duty Assignment Management Use Case Description

- View Duty List, Create Duty Assignment, Assign Teacher to Duty, Edit Duty Assignment, Delete Duty Assignment

Item	Description
Use Case ID	UC-RM-08
Use Case Name	Duty Assignment Management
Actor	Principal
Description	This use case describes how the principal manages duty assignments in the GENIUSAQILOS system. The principal can view the duty list, create new duty assignments, assign teachers to duties, edit duty details, and delete duty assignments when they are no longer needed.
Trigger	The principal accesses the Task & Duty Manager module to manage teacher duty assignments.
Pre-condition	The principal must be logged into the system with authorized access. Teacher records must exist in the system before duties can be assigned.

Main flow	1. The principal opens the Task & Duty Manager module. 2. The system displays the duty list. 3. The principal creates a new duty assignment. 4. The principal assigns a teacher to the duty. 5. The principal may edit the duty assignment if changes are needed. 6. The principal may delete a duty assignment if it is no longer required. 7. The system saves and updates the duty assignment records.
Alternative flow	1. If no duty records are available, the system displays an empty duty list. 2. If required duty details are incomplete, the system asks the principal to complete the missing information. 3. If the selected teacher is unavailable, the principal must choose another teacher. 4. If a duty assignment cannot be edited or deleted, the system displays an error message.
Post-condition	Duty assignments are successfully created, updated, deleted, or assigned to teachers in the system.

Table 4.5.3.1.6.2 Duty Completion Tracking Use Case Description

- View Assigned Duties, View Duty Details, Update Duty Checklist, Upload Duty Proof Photo, Mark Duty as Completed

Item	Description
Use Case ID	UC-RM-09
Use Case Name	Duty Completion Tracking
Actor	Teacher
Description	This case describes how teachers track and complete their assigned duties in the GENIUSAQILOS system. Teachers can view assigned duties, check duty details, update the duty checklist, upload proof photos, and mark duties as completed after finishing the assigned task.

Trigger	A teacher accesses the Task & Duty Manager module to view or complete assigned duties.
Pre-condition	The teacher must be logged into the system. At least one duty must be assigned to the teacher by the principal.
Main flow	1. The teacher opens the Task & Duty Manager module. 2. The system displays the teacher's assigned duties. 3. The teacher selects a duty to view its details. 4. The teacher updates the duty checklist based on the completed task. 5. The teacher uploads a duty proof photo if required. 6. The teacher marks the duty as completed. 7. The system updates the duty completion status.
Alternative flow	1. If no duty is assigned, the system displays a "no assigned duties" message. 2. If the duty details cannot be loaded, the system displays an error message. 3. If the proof photo is missing or invalid, the system asks the teacher to upload a valid photo. 4. If the checklist is incomplete, the system may prevent the teacher from marking the duty as completed.
Post-condition	The duty checklist, proof photo, and completion status are updated in the system.

Table 4.5.3.1.6.3 Duty Switch Request Use Case Description

- Request Duty Switch, View Switch Request, Approve Duty Switch, Reject Duty Switch

Item	Description
Use Case ID	UC-RM-10
Use Case Name	Duty Switch Request
Actor	Teacher, Principal
Description	This case describes how a teacher requests to switch an assigned duty and how the principal manages the request. The teacher can submit a duty switch request, while the principal can view the request and

	decide whether to approve or reject it.
Trigger	A teacher is unable to perform an assigned duty and submits a duty switch request.
Pre-condition	The teacher must be logged into the system and must have an assigned duty. The principal must have access to review duty switch requests.
Main flow	1. The teacher opens the assigned duty details. 2. The teacher submits a duty switch request with a reason. 3. The system records the request as pending. 4. The principal views the duty switch request. 5. The principal reviews the request details. 6. The principal approves the request if it is valid. 7. The system updates the duty assignment after approval.
Alternative flow	1. If the teacher does not provide a valid reason, the system asks the teacher to complete the request. 2. If the principal rejects the request, the original duty assignment remains unchanged. 3. If no suitable replacement teacher is available, the request may remain pending or be rejected.
Post-condition	The duty switch request is approved, rejected, or kept pending. If approved, the duty assignment is updated in the system.

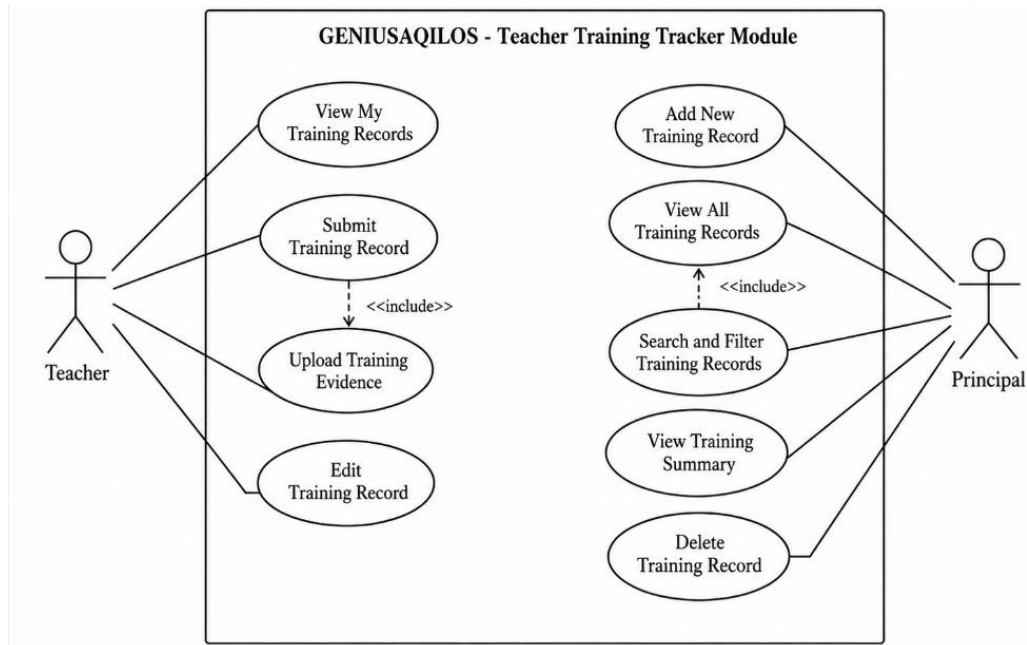


Figure 4.5.3.1.7 Use Case Diagram for Teacher Training Tracker Module

Table 4.5.3.1.7.1 Training Record Submission Use Case Description

- View My Training Records, Submit Training Record, Upload Training Evidence, Edit Training Record, Delete Training Record

Item	Description
Use Case ID	UC-RM-11
Use Case Name	Training Record Submission
Actor	Teacher
Description	This case describes how teachers manage their own training records in the GENIUSAQILOS system. Teachers can view their own training records, submit new training records, upload training evidence, and edit training record information when necessary.
Trigger	A teacher wants to record, or update completed training information in the system.
Pre-condition	The teacher must be logged into the system. The teacher must have valid training information or evidence to submit.
Main flow	1. The teacher opens the Teacher Training Tracker module. 2. The system displays the teacher's training records. 3. The teacher submits a new training record by entering the required

	training information. 4. The teacher uploads training evidence as supporting proof. 5. The teacher edits the training record if any information needs correction. 6. The system saves and updates the training record.
Alternative flow	1. If the training information is incomplete, the system asks the teacher to complete the missing fields. 2. If the uploaded evidence format is invalid, the system rejects the upload and asks for a valid file. 3. If the record cannot be edited due to system restrictions, the system displays an error message.
Post-condition	The teacher training record is successfully viewed, submitted, uploaded with evidence, or updated in the system.

Table 4.5.3.1.7.2 Training Monitoring Use Case Description

- View All Training Records, Search and Filter Training Records, View Training Summary

Item	Description
Use Case ID	UC-RM-12
Use Case Name	Training Monitoring
Actor	Principal
Description	This case describes how the principal monitors and manages teacher training records in the GENIUSAQILOS system. The principal can add new training records, view all training records, search and filter records, view training summaries, and delete training records when necessary.
Trigger	The principal accesses the Teacher Training Tracker module to monitor teacher training progress or manage training records.
Pre-condition	The principal must be logged into the system with authorized access. Training records must exist for viewing, searching, summarizing, or deleting.

Main flow	1. The principal opens the Teacher Training Tracker module. 2. The principal adds a new training record if required. 3. The system displays all teacher training records. 4. The principal searches and filters training records based on teacher name, training type, date, or status. 5. The principal views the training summary to monitor teacher training progress. 6. The principal deletes an incorrect or unnecessary training record when needed. 7. The system updates the training record list.
Alternative flow	1. If no training records are available, the system displays a “no record found” message. 2. If the search or filter criteria do not match any record, the system displays an empty result. 3. If required information is missing when adding a new training record, the system asks the principal to complete the form. 4. If the selected training record cannot be deleted, the system displays an error message.
Post-condition	Training records are successfully added, viewed, searched, summarized, or deleted. The training record database remains updated and accurate.

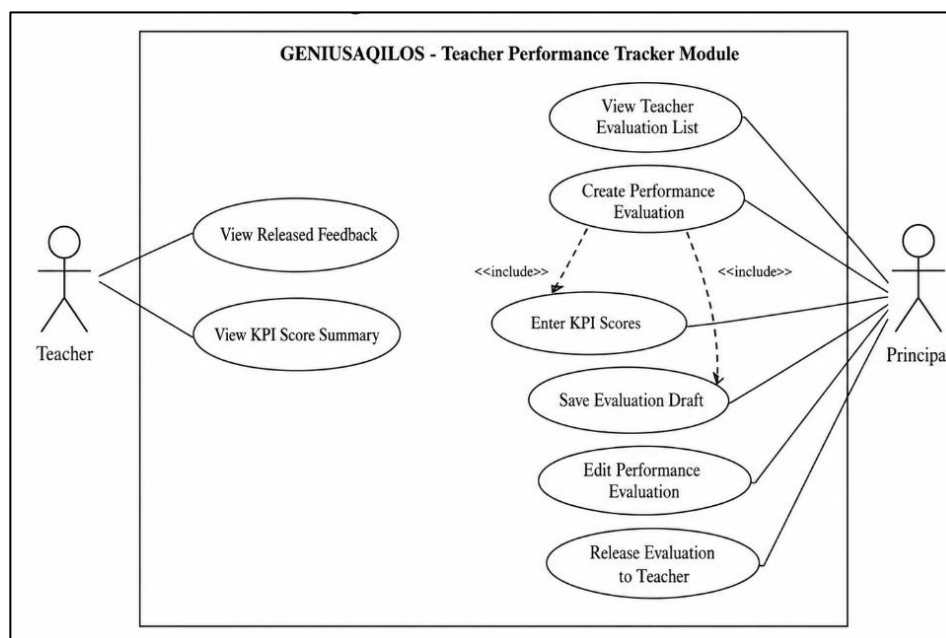


Figure 4.5.3.1.8 Use Case Diagram for Teacher Performance Tracker Module

Table 4.5.3.1.8.1 Performance Evaluation Management Use Case Description

- View Teacher Evaluation List, Create Performance Evaluation, Enter KPI

Scores, Save Evaluation Draft, Edit Performance Evaluation

Item	Description
Use Case ID	UC-RM-13
Use Case Name	Performance Evaluation Management
Actor	Principal
Description	This case describes how the principal manages teacher performance evaluations in the GENIUSAQILOS system. The principal can view the teacher evaluation list, create a new performance evaluation, enter KPI scores, save the evaluation as a draft, and edit the evaluation before releasing it to the teacher.
Trigger	The principal accesses the Teacher Performance Tracker module to create or manage teacher performance evaluations.
Pre-condition	The principal must be logged into the system with authorized access. Teacher records must exist in the system before performance evaluations can be created.
Main flow	1. The principal opens the Teacher Performance Tracker module. 2. The system displays the teacher evaluation list. 3. The principal selects a teacher for evaluation. 4. The principal creates a performance evaluation. 5. The principal enters KPI scores based on the teacher's performance. 6. The principal saves the evaluation as a draft if further editing is needed. 7. The principal edits the performance evaluation before final submission. 8. The system saves the updated evaluation record.
Alternative flow	1. If no teacher record is available, the system displays a "no record found" message. 2. If KPI scores are incomplete or invalid, the system asks the principal to correct the scores. 3. If the evaluation cannot be saved, the system displays an error message. 4. If the principal exits the interface before saving, unsaved changes may be lost.

Post-condition	The teacher performance evaluation is created, saved as a draft, or updated successfully in the system.
----------------	---

Table 4.5.3.1.8.2 Performance Feedback Use Case Description

- Release Evaluation to Teacher, View Released Feedback, View KPI Score Summary

Item	Description
Use Case ID	UC-RM-14
Use Case Name	Performance Feedback
Actor	Principal, Teacher
Description	This case describes how performance feedback is released and viewed in the GENIUSAQILOS system. The principal releases the completed evaluation to the teacher, while the teacher can view the released feedback and KPI score summary.
Trigger	The principal completes a teacher performance evaluation and releases it to the teacher.
Pre-condition	The principal must be logged into the system, and the performance evaluation must already be completed. The teacher must have access to view released feedback.
Main flow	1. The principal reviews completed performance evaluation. 2. The principal releases the evaluation to the teacher. 3. The system updates the evaluation status as released. 4. The teacher opens the Teacher Performance Tracker module. 5. The teacher views the released feedback. 6. The teacher views the KPI score summary to understand performance results.
Alternative flow	1. If the evaluation is still saved as a draft, it cannot be viewed by the teacher. 2. If the evaluation has not been released, the system does not display feedback to the teacher. 3. If the KPI score summary cannot be loaded, the system displays an

	error message.
Post-condition	The evaluation is successfully released to the teacher, and the teacher can view the feedback and KPI score summary.

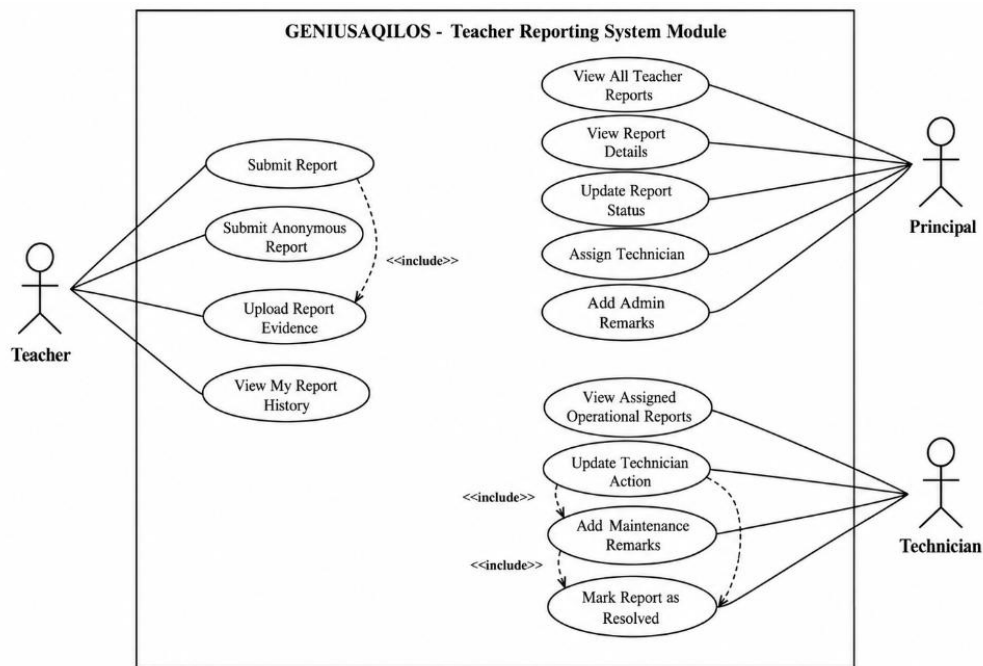


Figure 4.5.3.1.9 Use Case Diagram for Teacher Reporting System Module

Table 4.5.3.1.9.1 Report Submission Use Case Description

- Submit Report, Submit Anonymous Report, Upload Report Evidence, View My Report History

Item	Description
Use Case ID	UC-RM-15
Use Case Name	Report Submission
Actor	Teacher
Description	This use case describes how teachers submit reports through the GENIUSAQILOS Teacher Reporting System. Teachers can submit

	normal reports, submit anonymous reports, upload report evidence, and view their own report history to track previous submissions.
Trigger	A teacher wants to report an issue, incident, or operational problem through the system.
Pre-condition	The teacher must be logged into the system to submit a normal report or view report history. For anonymous reports, the system must allow identity-hidden submission.
Main flow	1. The teacher opens the Teacher Reporting System module. 2. The teacher selects the option to submit a report. 3. The teacher enters the report details such as issue type, description, and location. 4. The teacher uploads report evidence if available. 5. The teacher submits the report to the system. 6. The system records the report and assigns an initial status. 7. The teacher views report history to check submitted reports.
Alternative flow	1. If the teacher chooses anonymous reporting, the system hides the teacher's identity from the report record. 2. If required report details are incomplete, the system asks the teacher to complete the missing information. 3. If the uploaded evidence file is invalid, the system rejects the upload and requests a valid file. 4. If no report history exists, the system displays an empty history message.
Post-condition	The report is successfully submitted and stored in the system. The teacher can view the report status or report history when needed.

Table 4.5.3.1.9.2 Report Review and Assignment Use Case Description

- View All Teacher Reports, View Report Details, Update Report Status, Assign Technician, Add Admin Remarks

Item	Description
Use Case ID	UC-RM-16
Use Case Name	Report Review and Assignment

Actor	Principal
Description	This case describes how the principal reviews and manages teacher reports in the GENIUSAQILOS system. The principal can view all submitted teacher reports, open report details, update report status, assign a technician for operational handling, and add admin remarks for follow-up records.
Trigger	A teacher report is submitted, or the principal accesses the reporting module to review submitted reports.
Pre-condition	The principal must be logged into the system with authorized access. At least one teacher report must exist in the system for review or assignment.
Main flow	1. The principal opens the Teacher Reporting System module. 2. The system displays all teacher reports. 3. The principal selects a report to view full report details. 4. The principal reviews the report description, evidence, and status. 5. The principal updates the report status based on the review decision. 6. The principal assigns a technician if the report requires operational action. 7. The principal adds admin remarks for documentation and follow-up. 8. The system saves updated report information.
Alternative flow	1. If no reports are available, the system displays a “no report found” message. 2. If the report details cannot be retrieved, the system displays an error message. 3. If no technician is available, the report remains pending assignment. 4. If the report status update fails, the system keeps the previous status and notifies the principal.
Post-condition	The report is reviewed, updated, assigned to a technician if required, and recorded with admin remarks.

Table 4.5.3.1.9.3 Technician Report Handling Use Case Description

- View Assigned Operational Reports, Update Technician Action, Add Maintenance Remarks, Mark Report as Resolved

Item	Description
Use Case ID	UC-RM-17
Use Case Name	Technician Report Handling
Actor	Technician
Description	This case describes how technicians handle assigned operational reports in the GENIUSAQILOS system. The technician can view assigned reports, update the action taken, add maintenance remarks, and mark the report as resolved after completing the required work.
Trigger	A principal assigns an operational report to a technician for handling.
Pre-condition	The technician must be logged into the system. The report must already be assigned by the principal.
Main flow	1. The technician opens the Teacher Reporting System module. 2. The system displays assigned operational reports. 3. The technician selects a report to review the assigned issue. 4. The technician updates the action taken for the report. 5. The technician adds maintenance remarks to explain the work completed. 6. The technician marks the report as resolved after the issue is handled. 7. The system updates the report status and records the technician's action.
Alternative flow	1. If no reports are assigned, the system displays a "no assigned reports" message. 2. If the report details cannot be loaded, the system displays an error message. 3. If the technician has not completed the required action, the report cannot be marked as resolved. 4. If the maintenance remarks are missing, the system may request the technician to add remarks before resolution.
Post-condition	The assigned report is updated with technician action, maintenance remarks, and resolution status. The principal can review the updated report record.

4.5.3.2 Activity Diagram

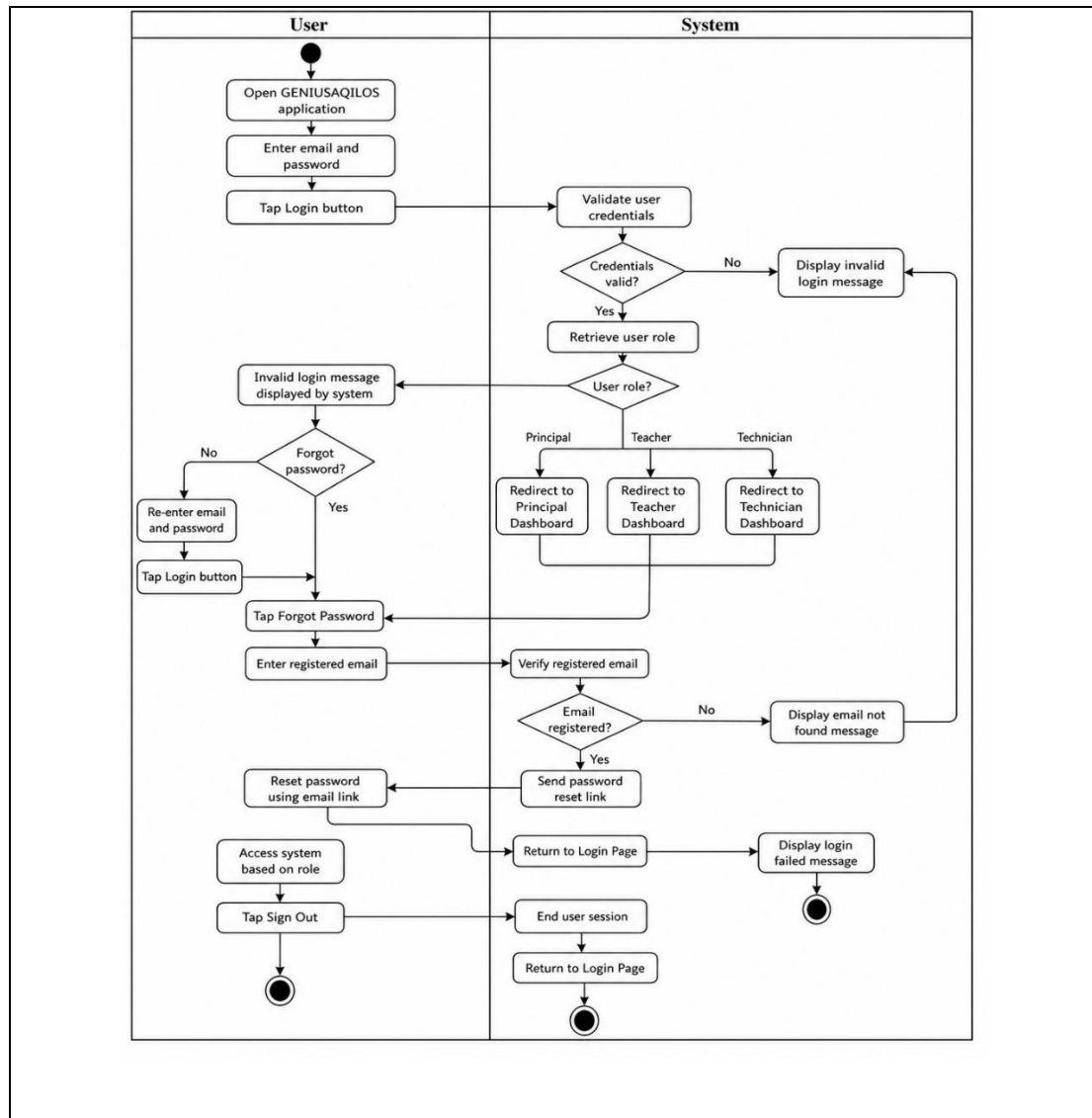


Figure 4.5.3.2.1 Activity Diagram for Login and Authentication

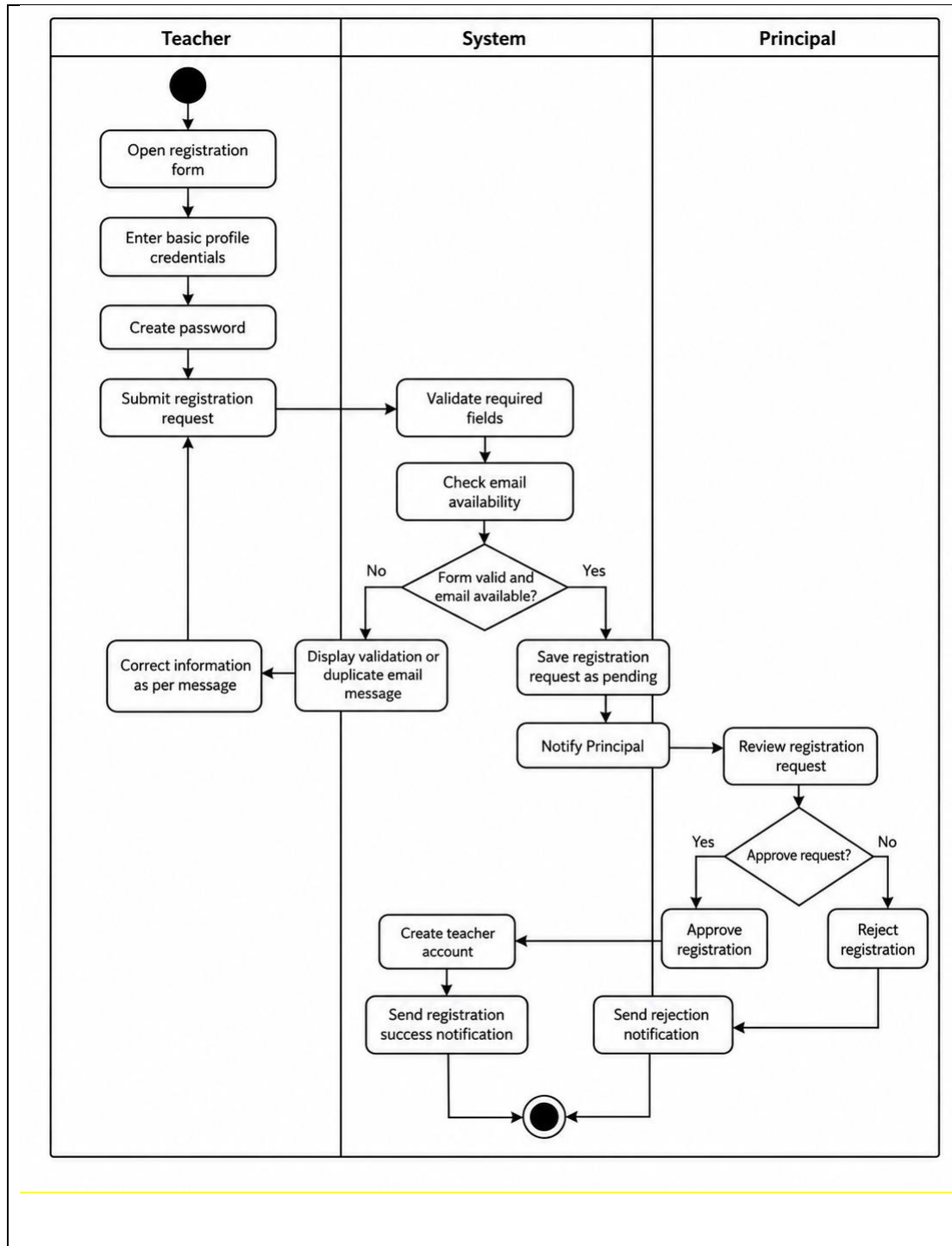


Figure 4.5.3.2.2 Activity Diagram for User Registration

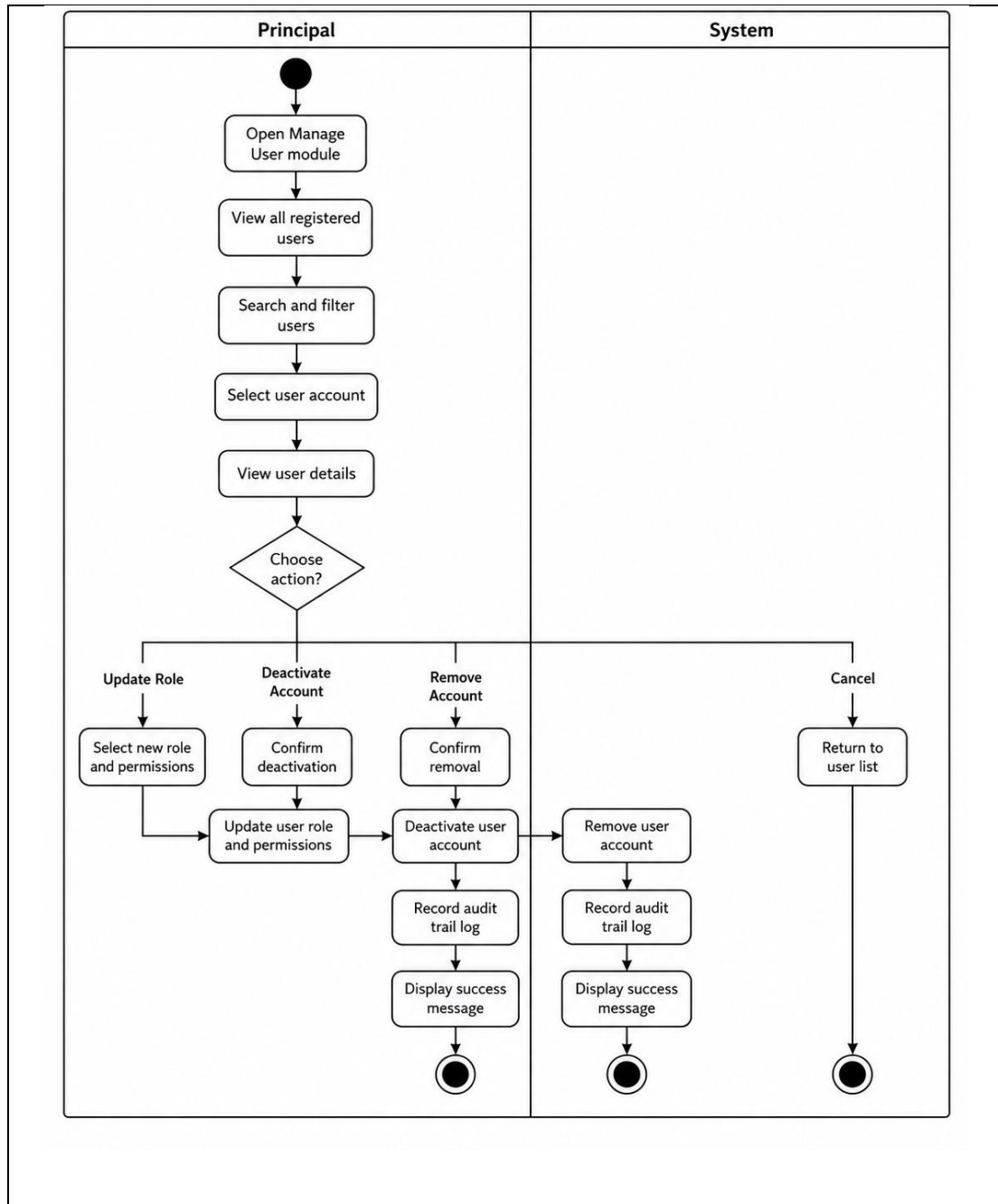


Figure 4.5.3.2.3 Activity Diagram for User Account Management

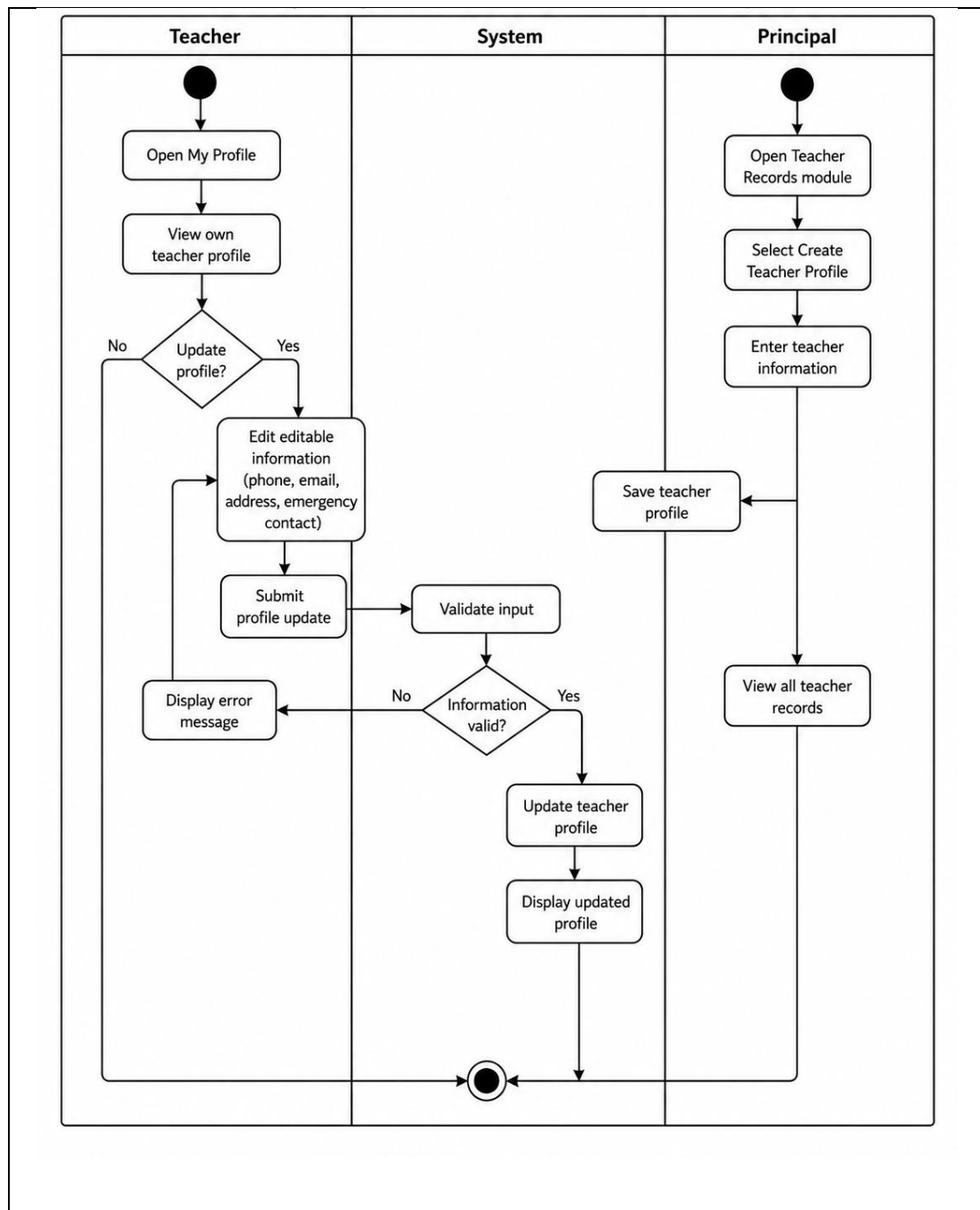


Figure 4.5.3.2.4 Activity Diagram for Teacher Profile Management

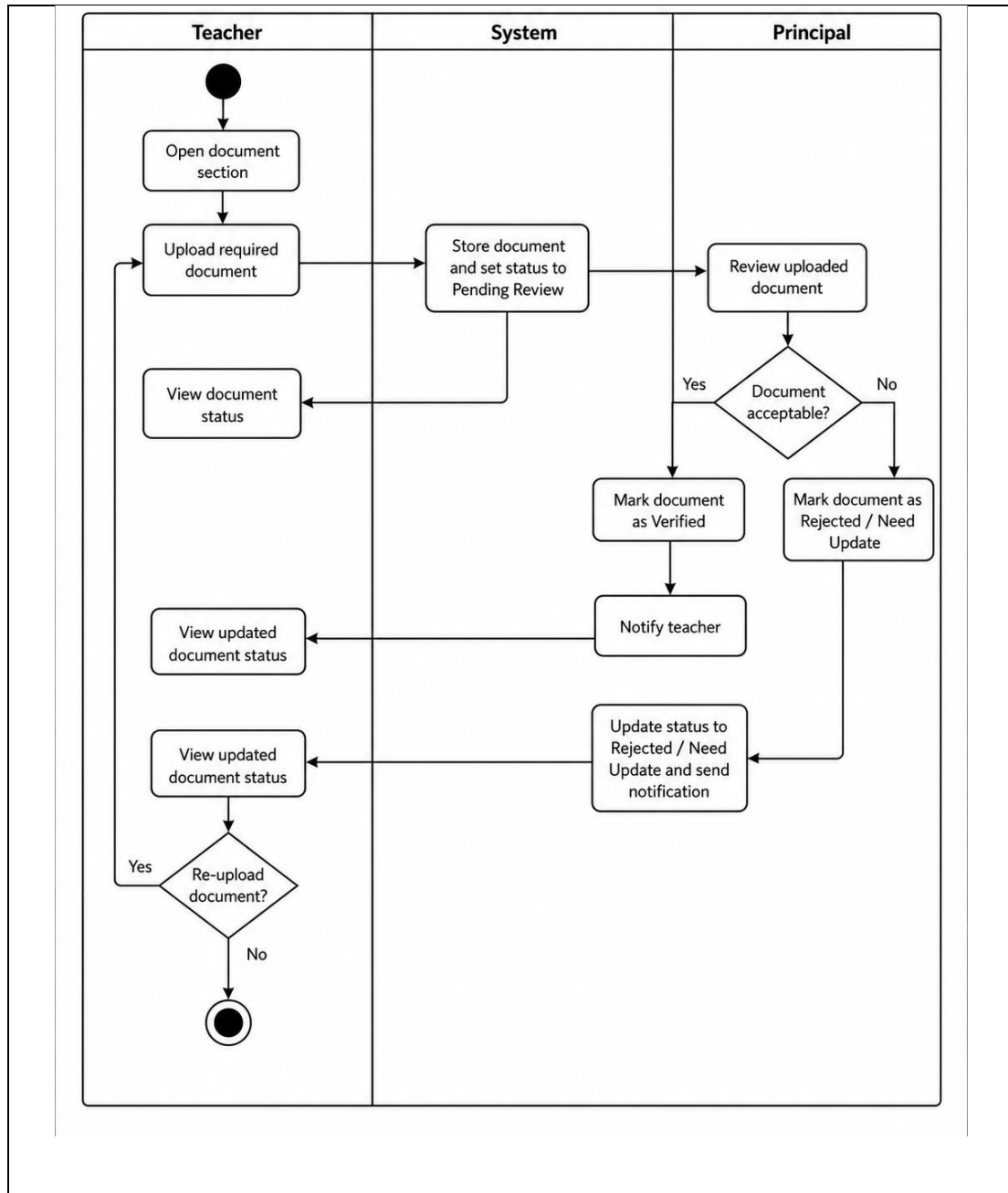


Figure 4.5.3.2.5 Activity Diagram for Teacher Document Management

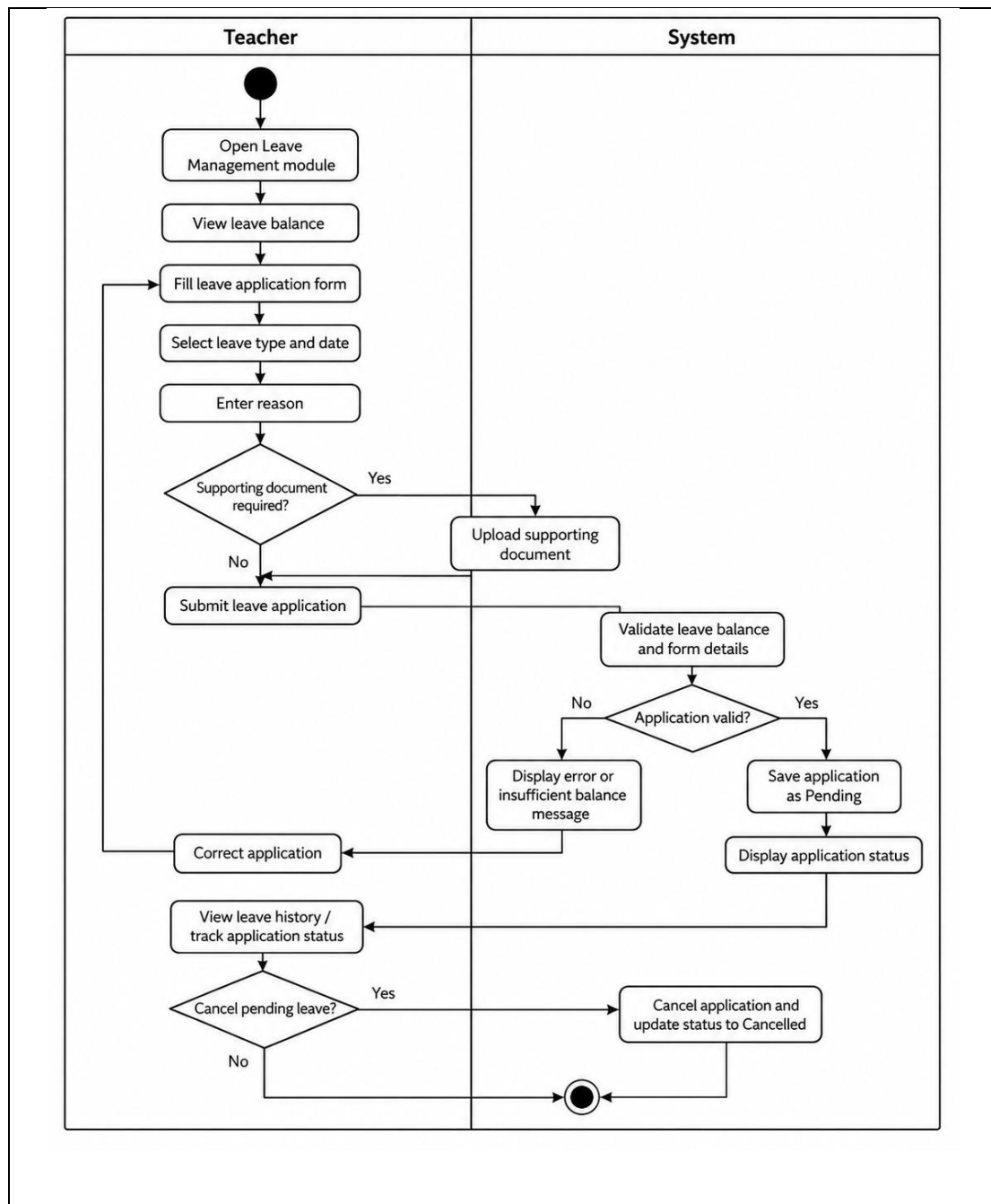


Figure 4.5.3.2.6 Activity Diagram for Leave Application

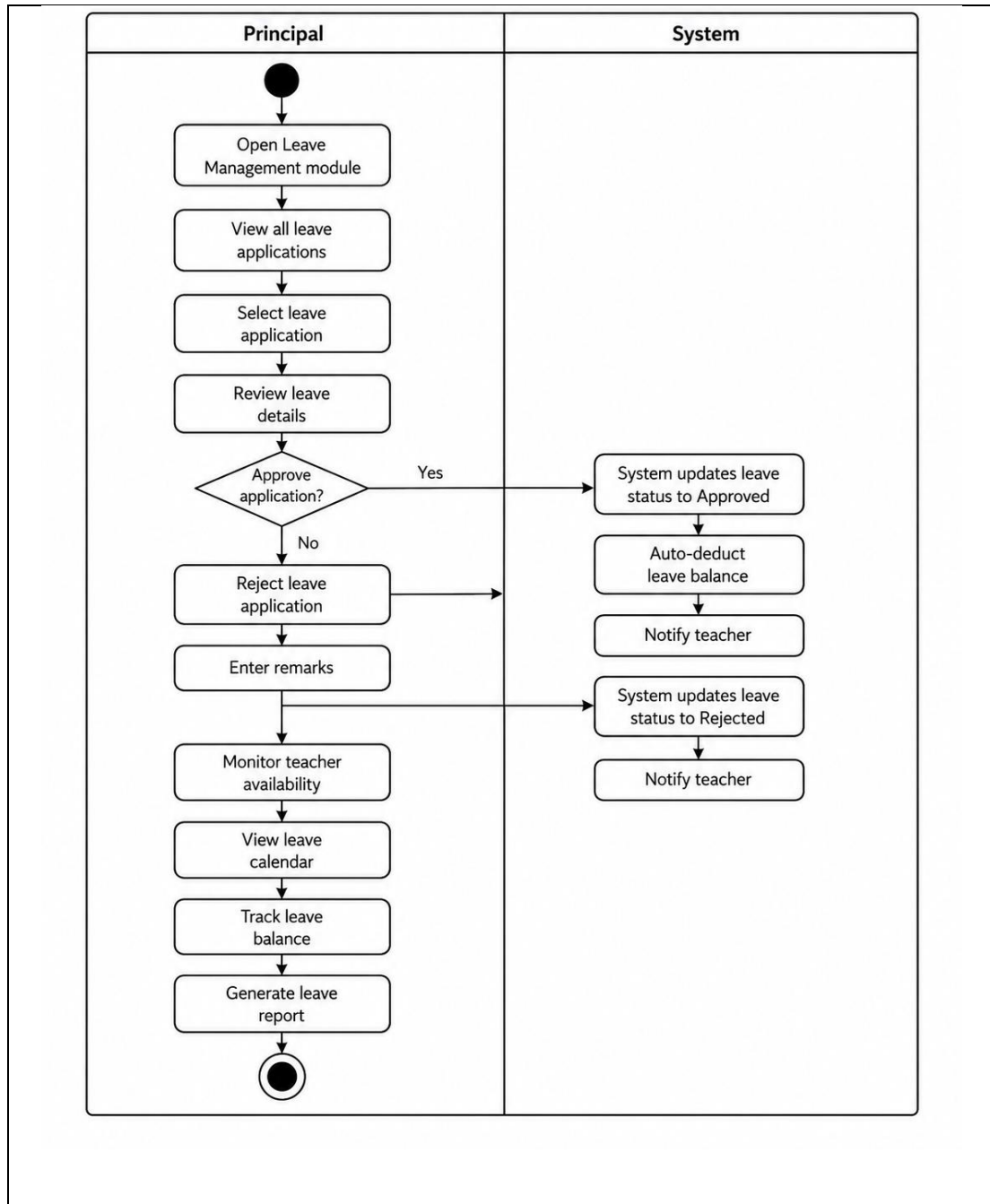


Figure 4.5.3.2.7 Activity Diagram for Leave Approval and Reporting

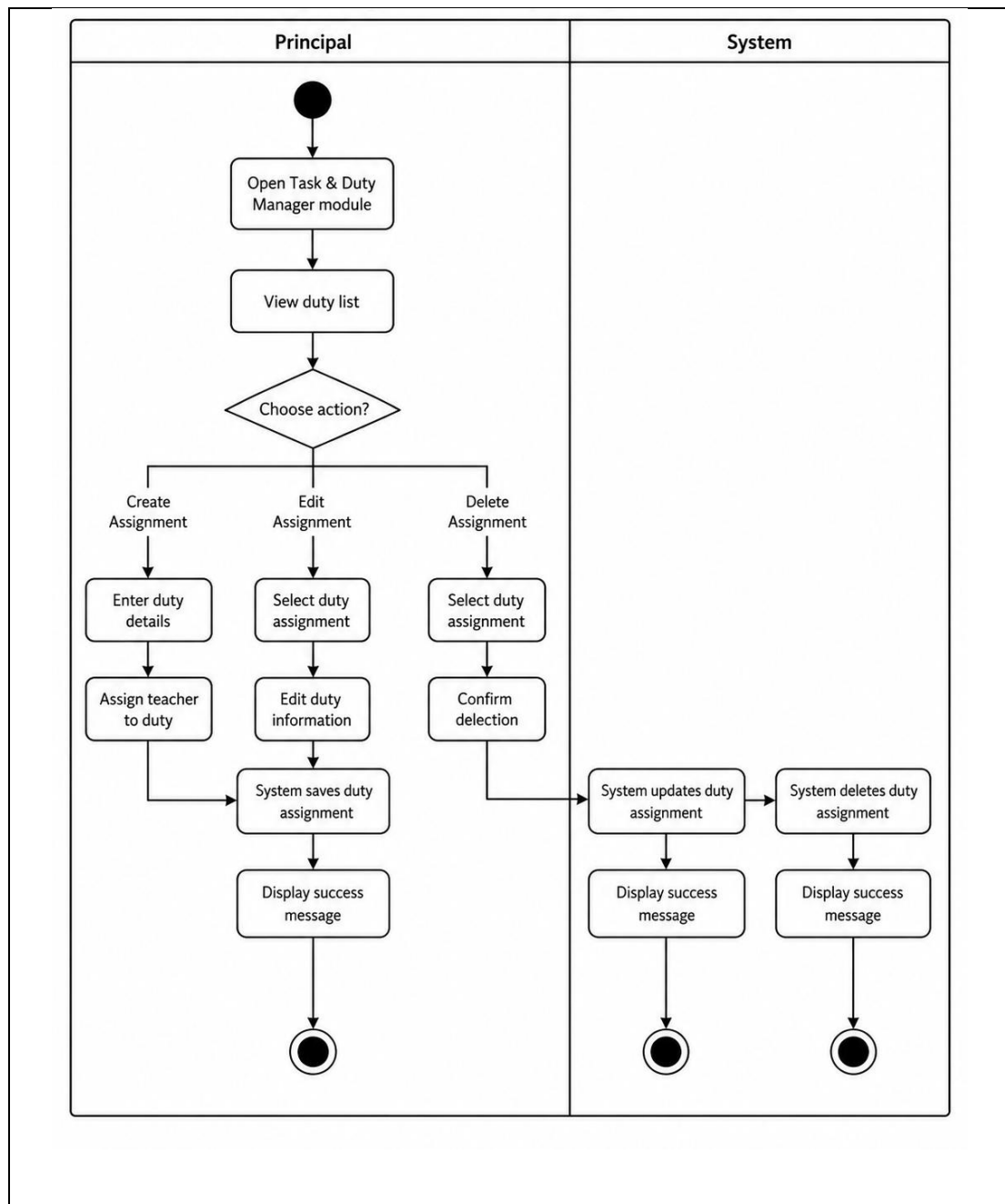


Figure 4.5.3.2.8 Activity Diagram for Duty Assignment Management

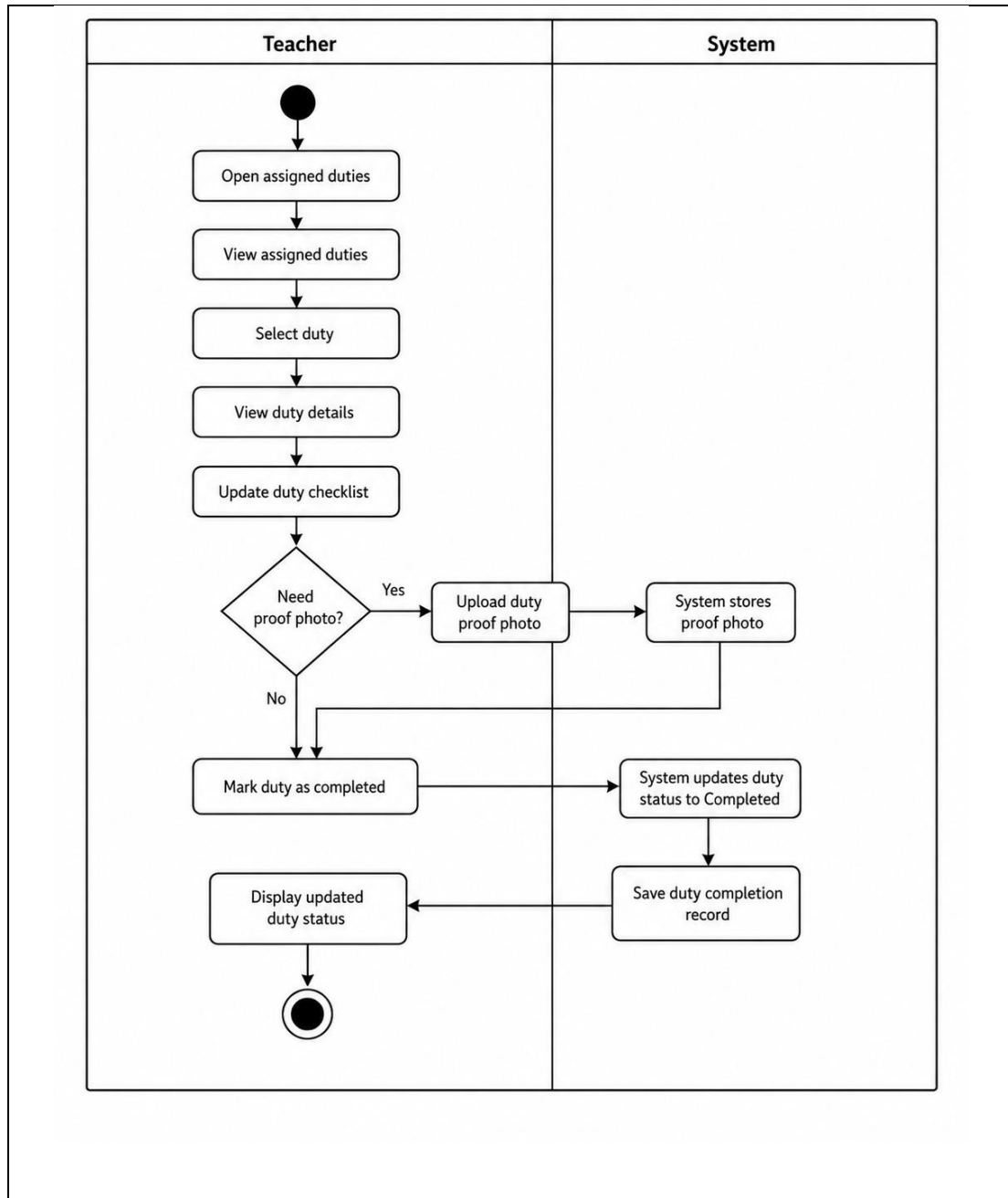


Figure 4.5.3.2.9 Activity Diagram for Duty Completion Tracking

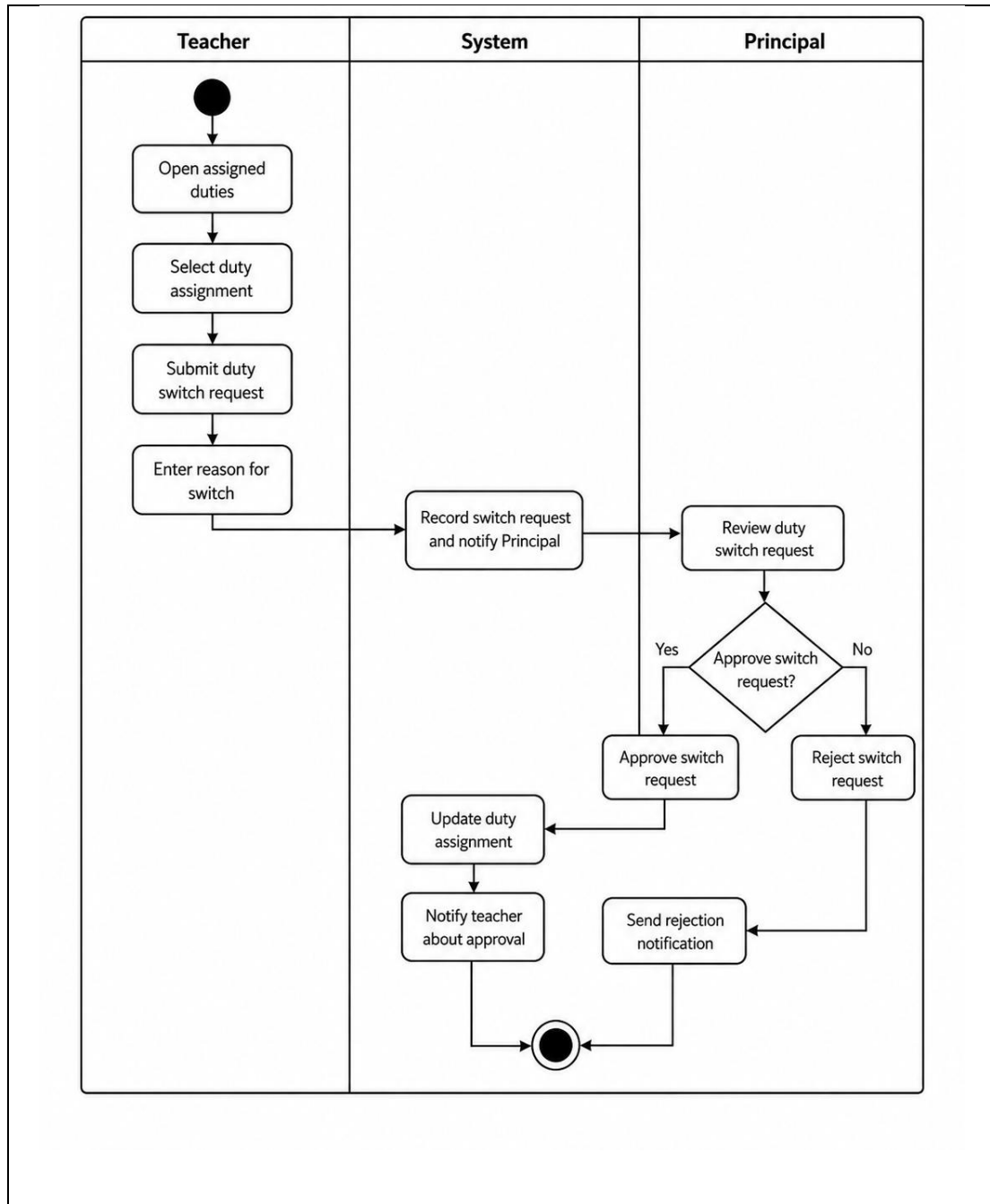


Figure 4.5.3.2.10 Activity Diagram for Duty Switch Request

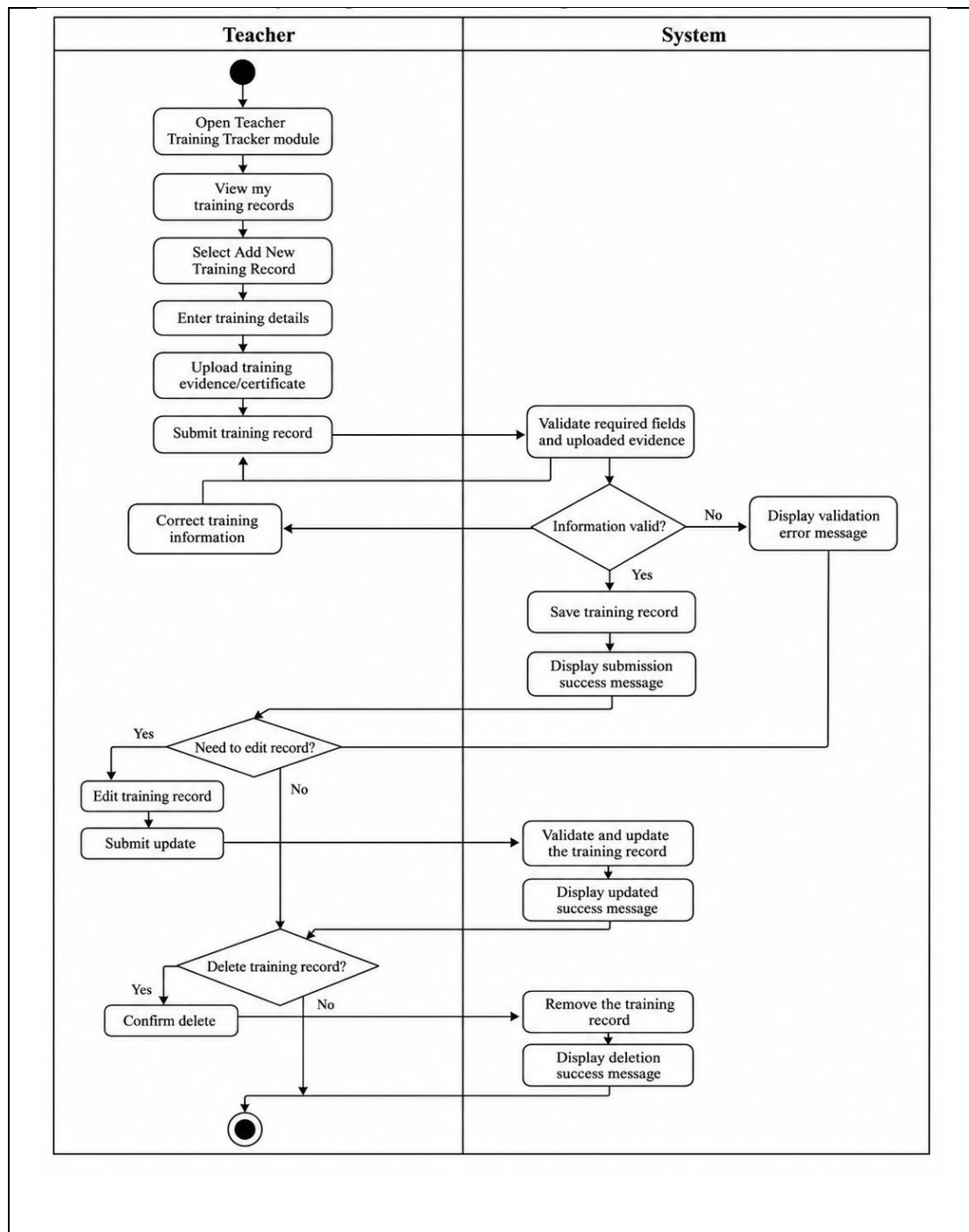


Figure 4.5.3.2.11 Activity Diagram for Training Record Submission

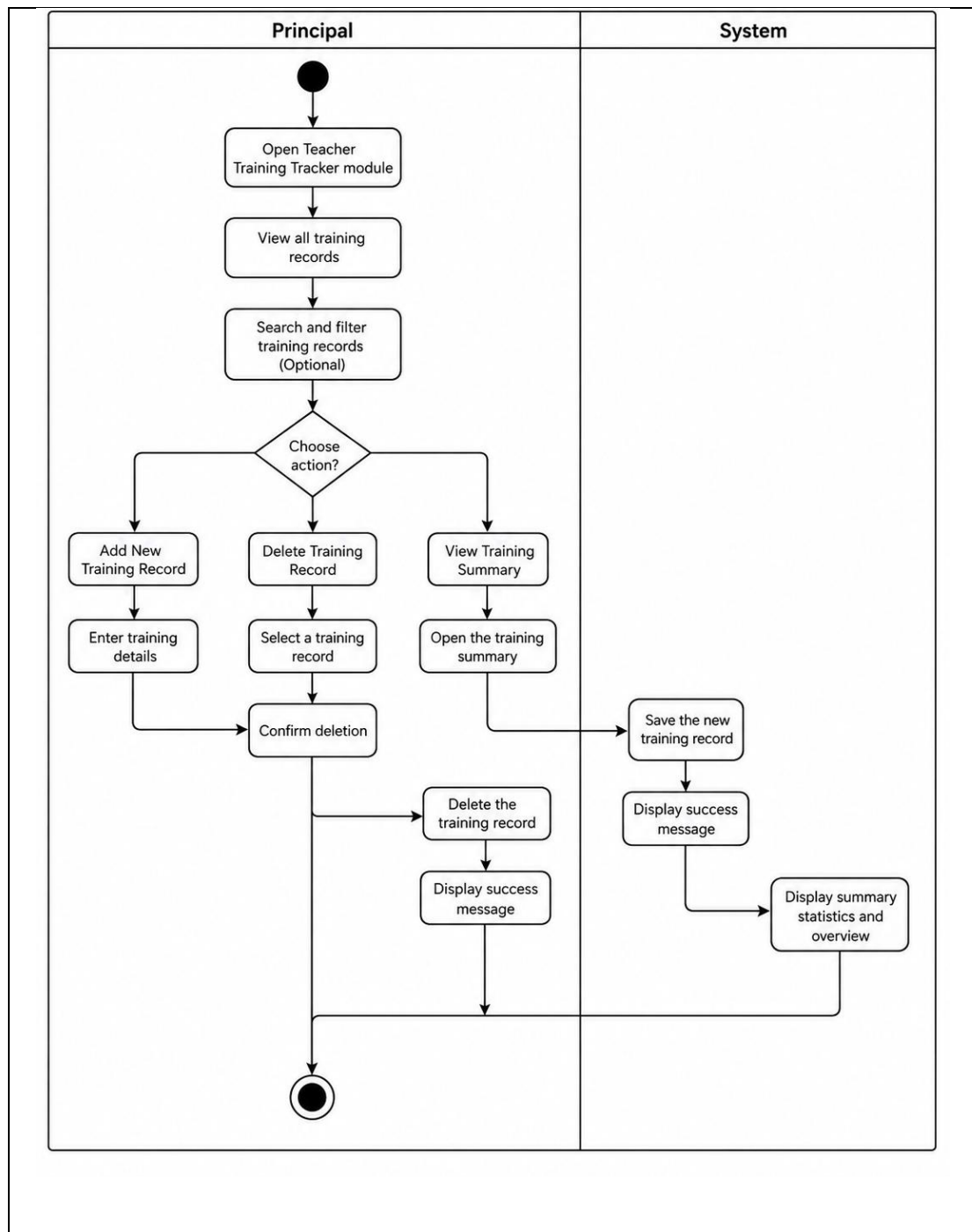


Figure 4.5.3.2.12 Activity Diagram for Training Monitoring

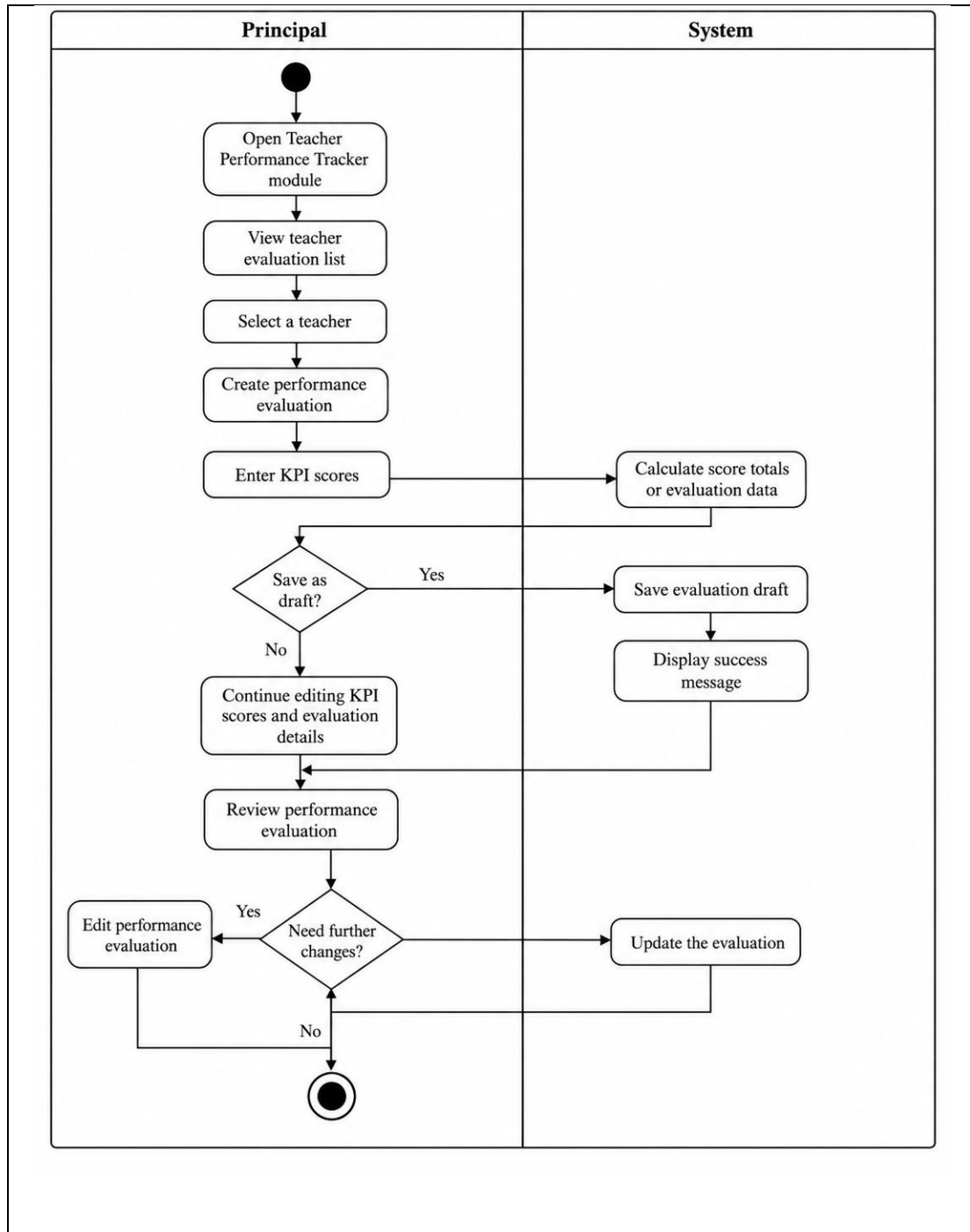


Figure 4.5.3.2.13 Activity Diagram for Performance Evaluation Management

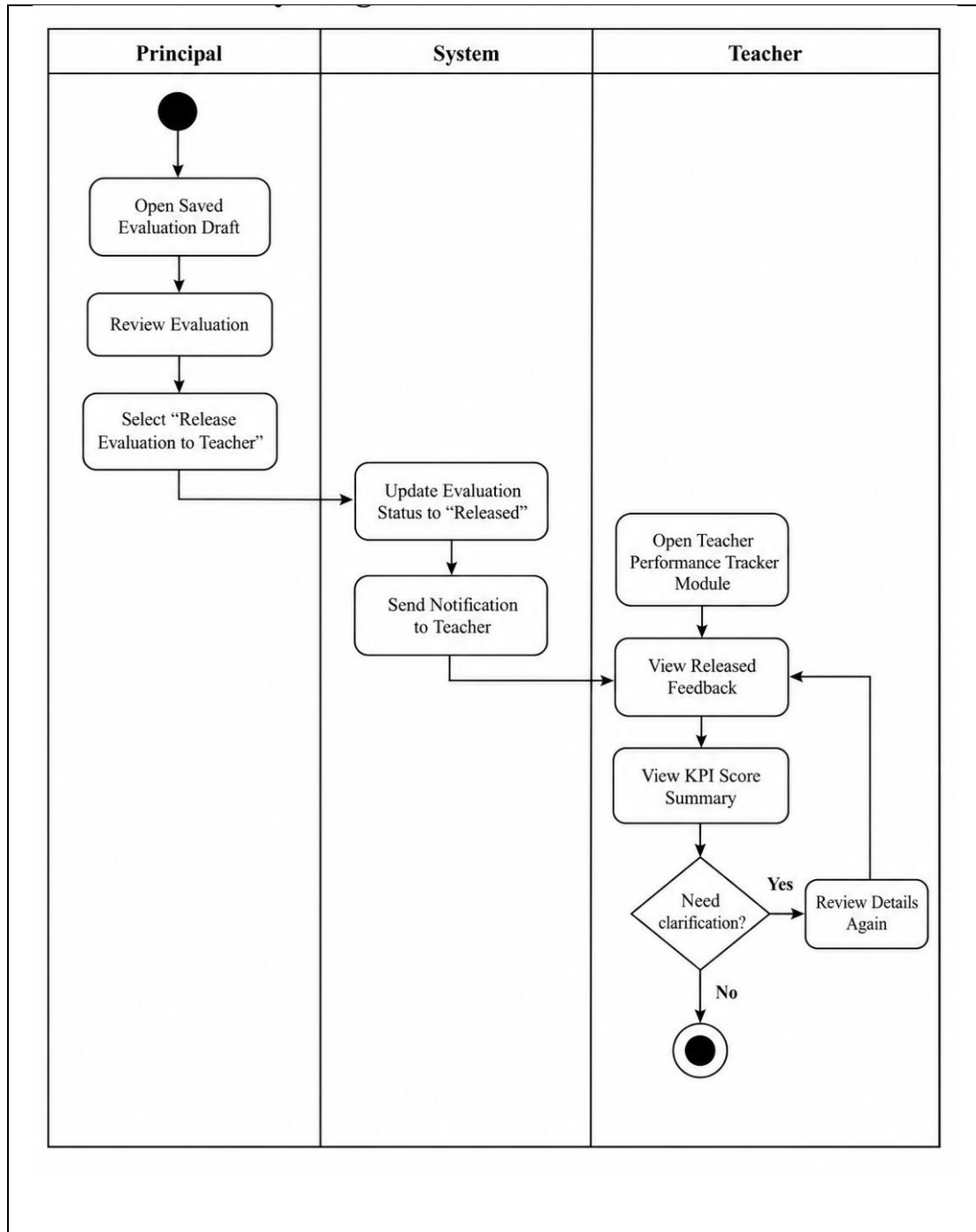


Figure 4.5.3.2.14 Activity Diagram for Performance Feedback

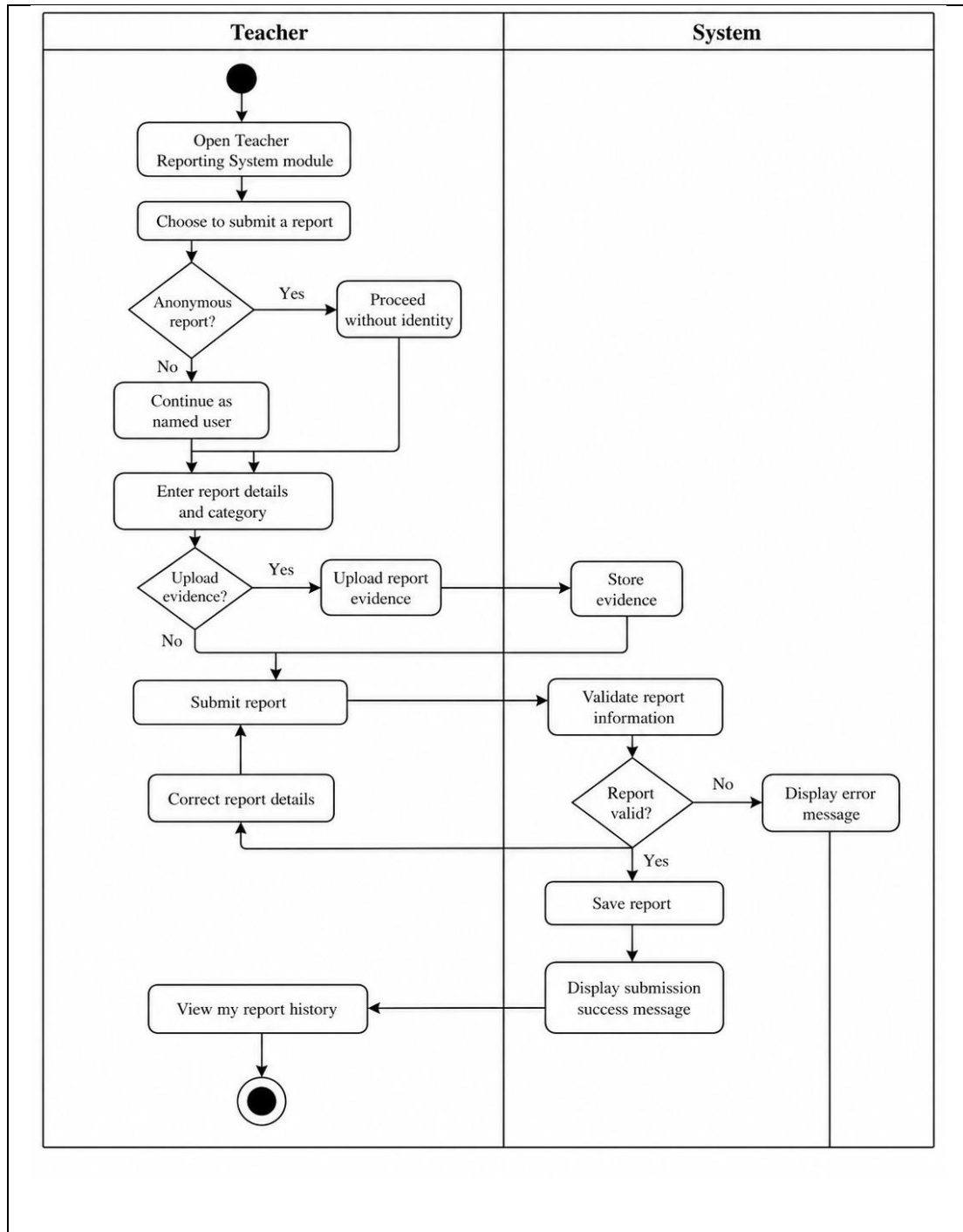


Figure 4.5.3.2.15 Activity Diagram for Report Submission

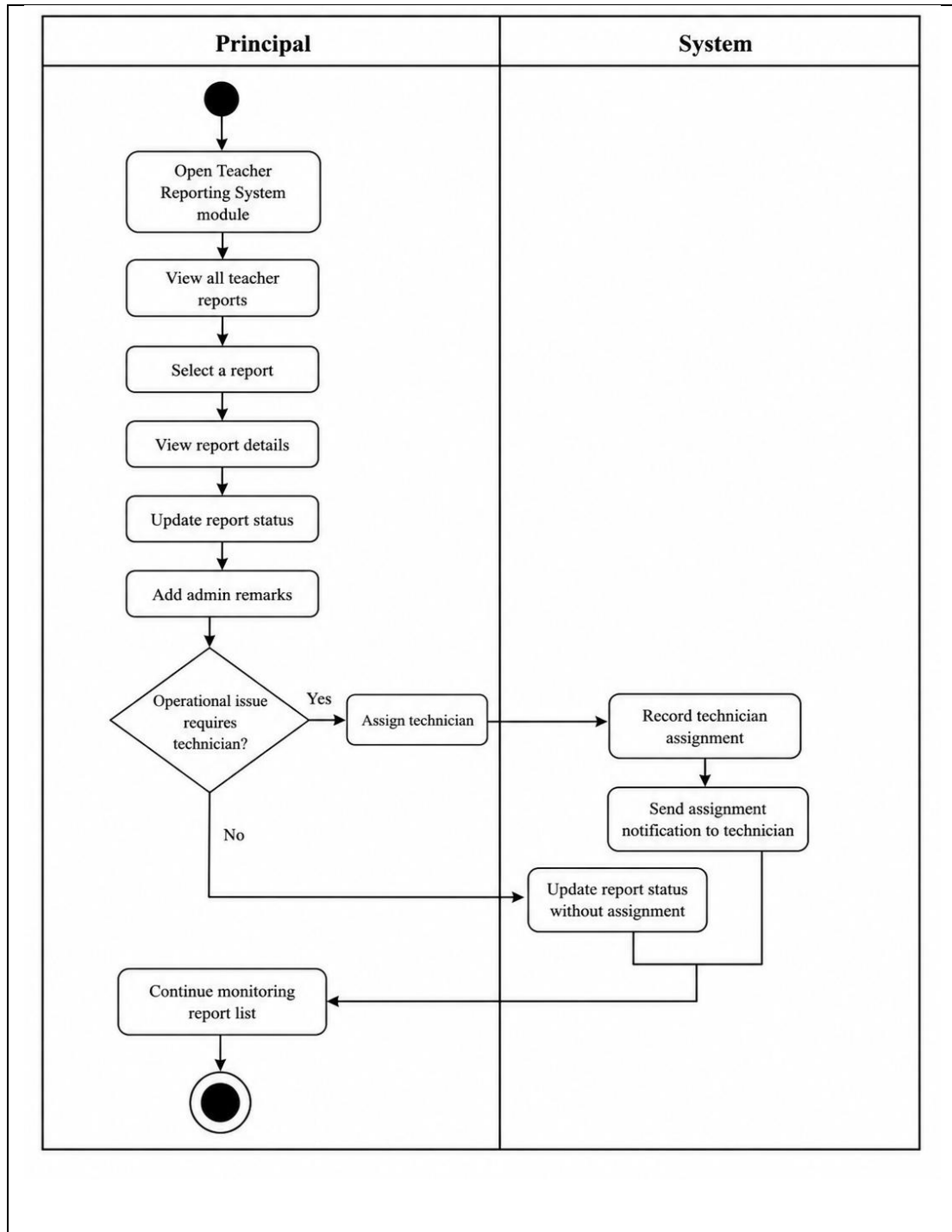


Figure 4.5.3.2.16 Activity Diagram for Report Review and Assignment

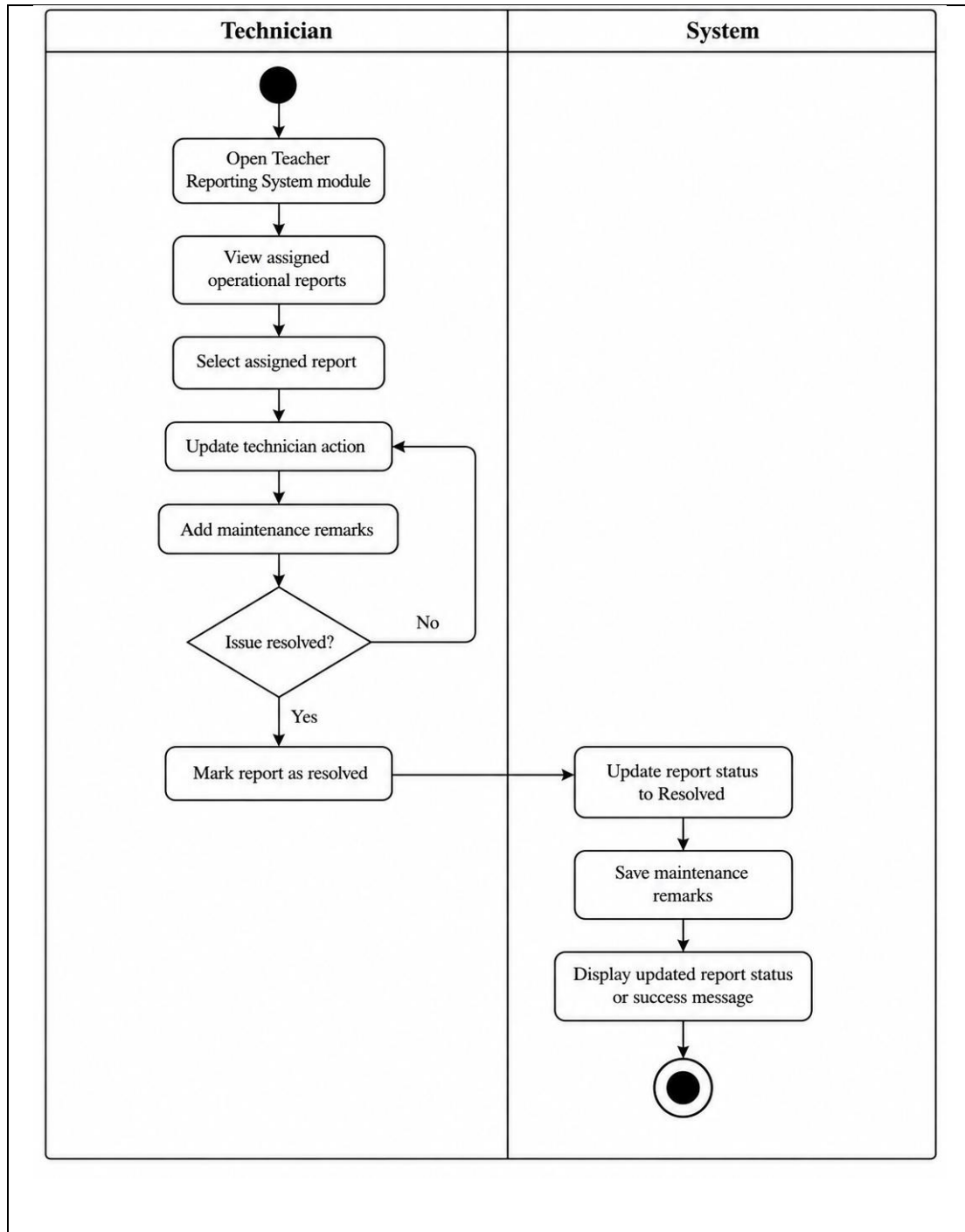


Figure 4.5.3.2.17 Activity Diagram for Technician Report Handling

4.5.3.3 Sequence Diagram

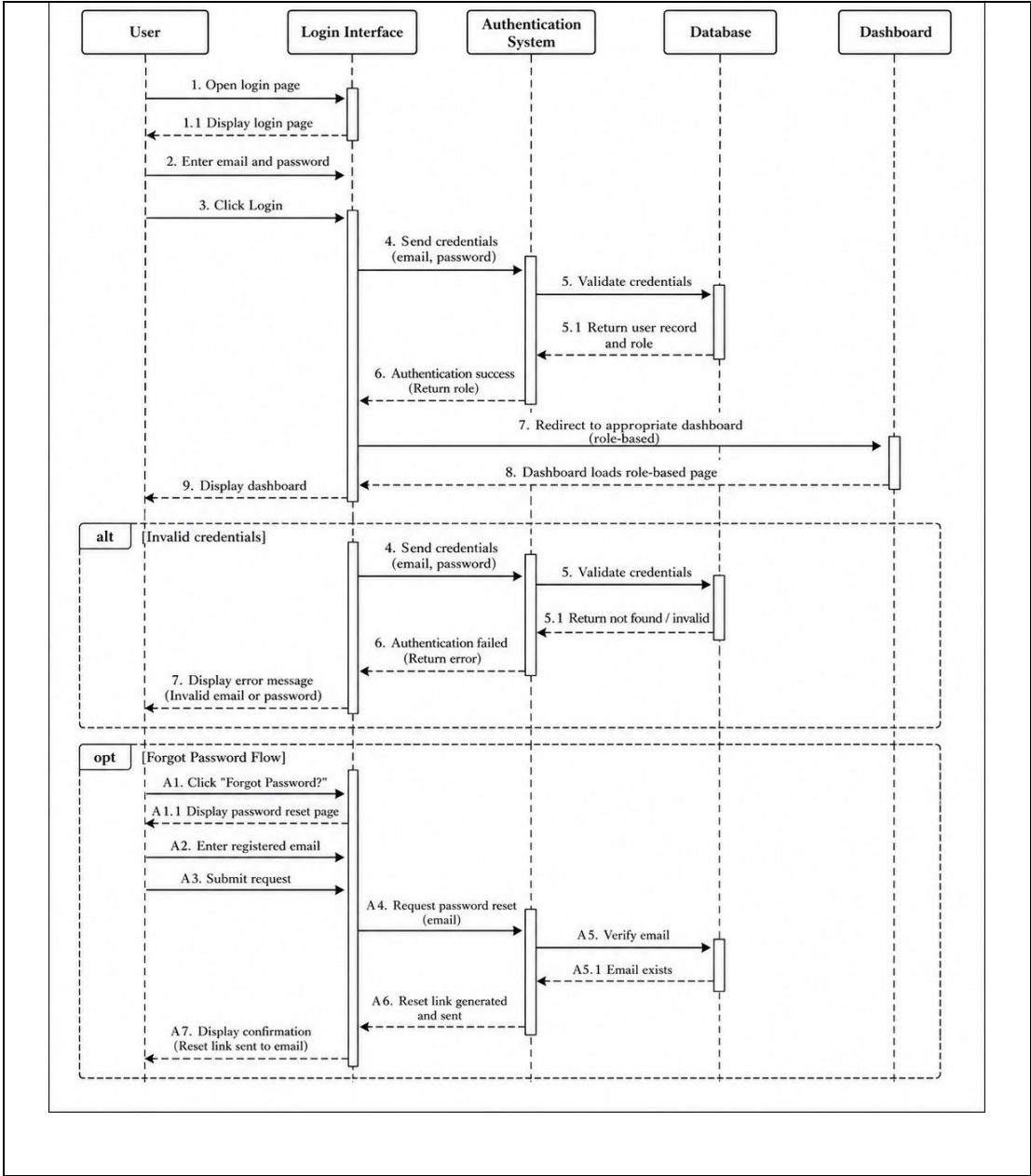


Figure 4.5.3.3.1 Sequence Diagram for Login and Authentication

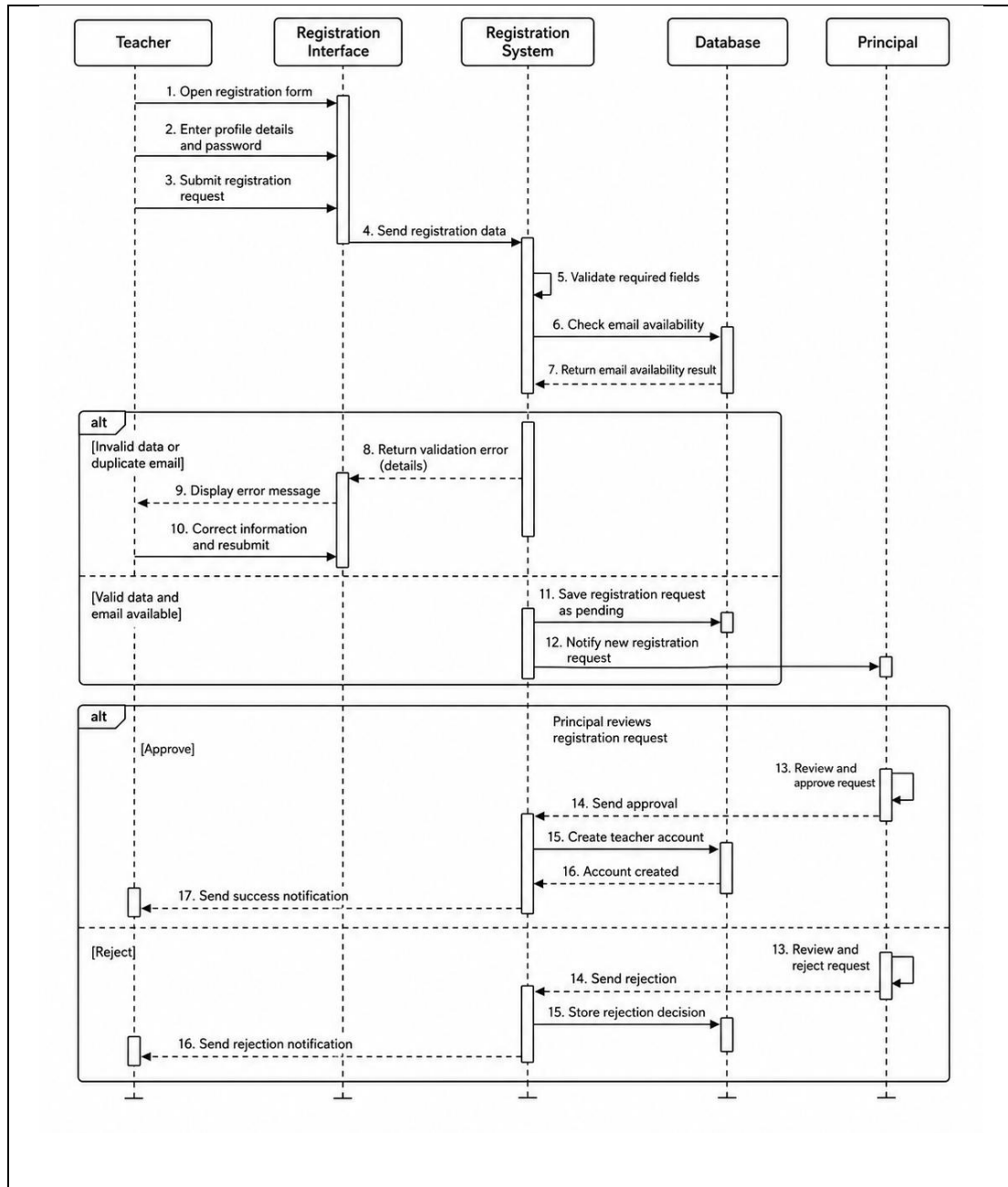


Figure 4.5.3.3.2 Sequence Diagram for User Registration

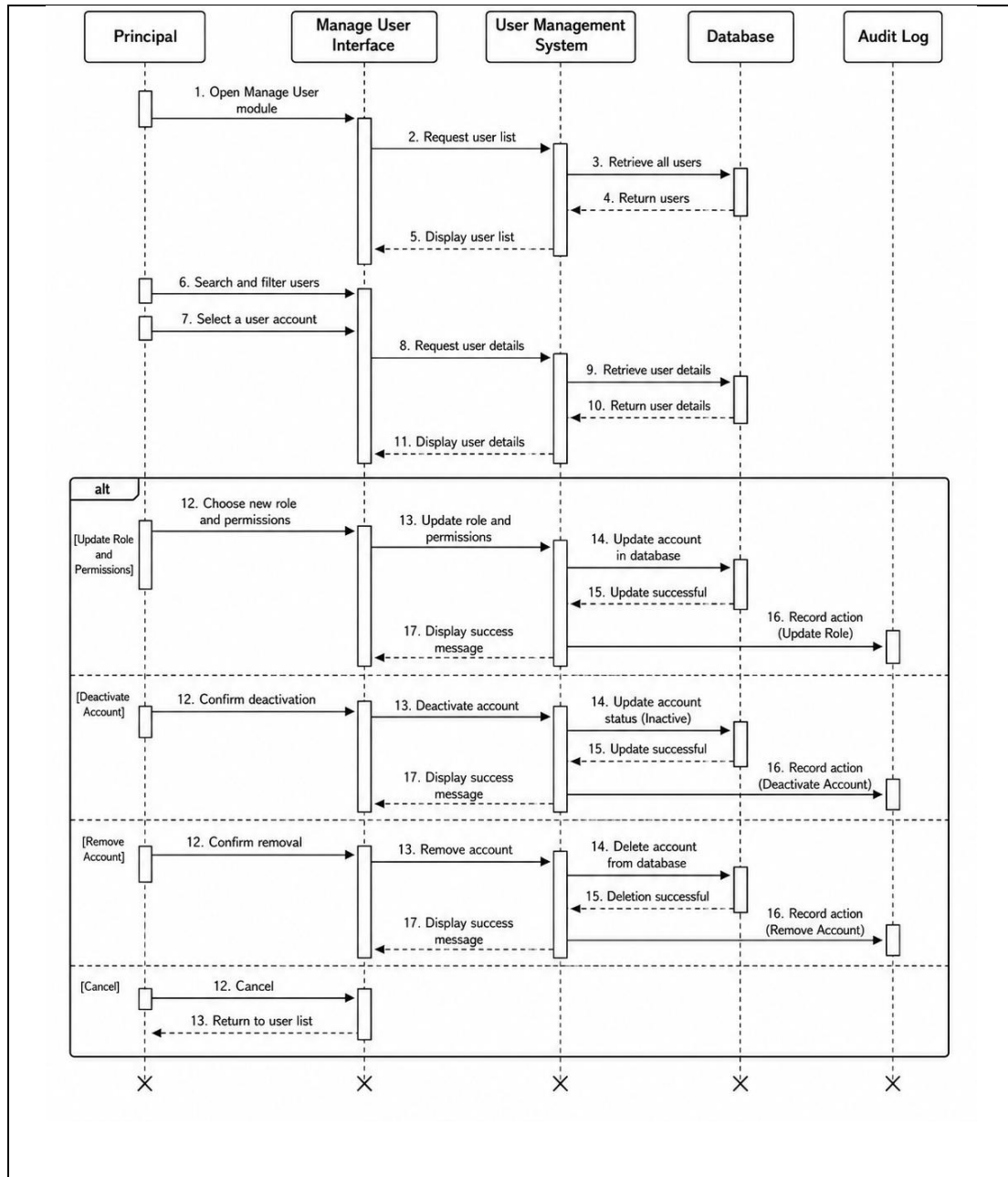


Figure 4.5.3.3 Sequence Diagram for User Account Management

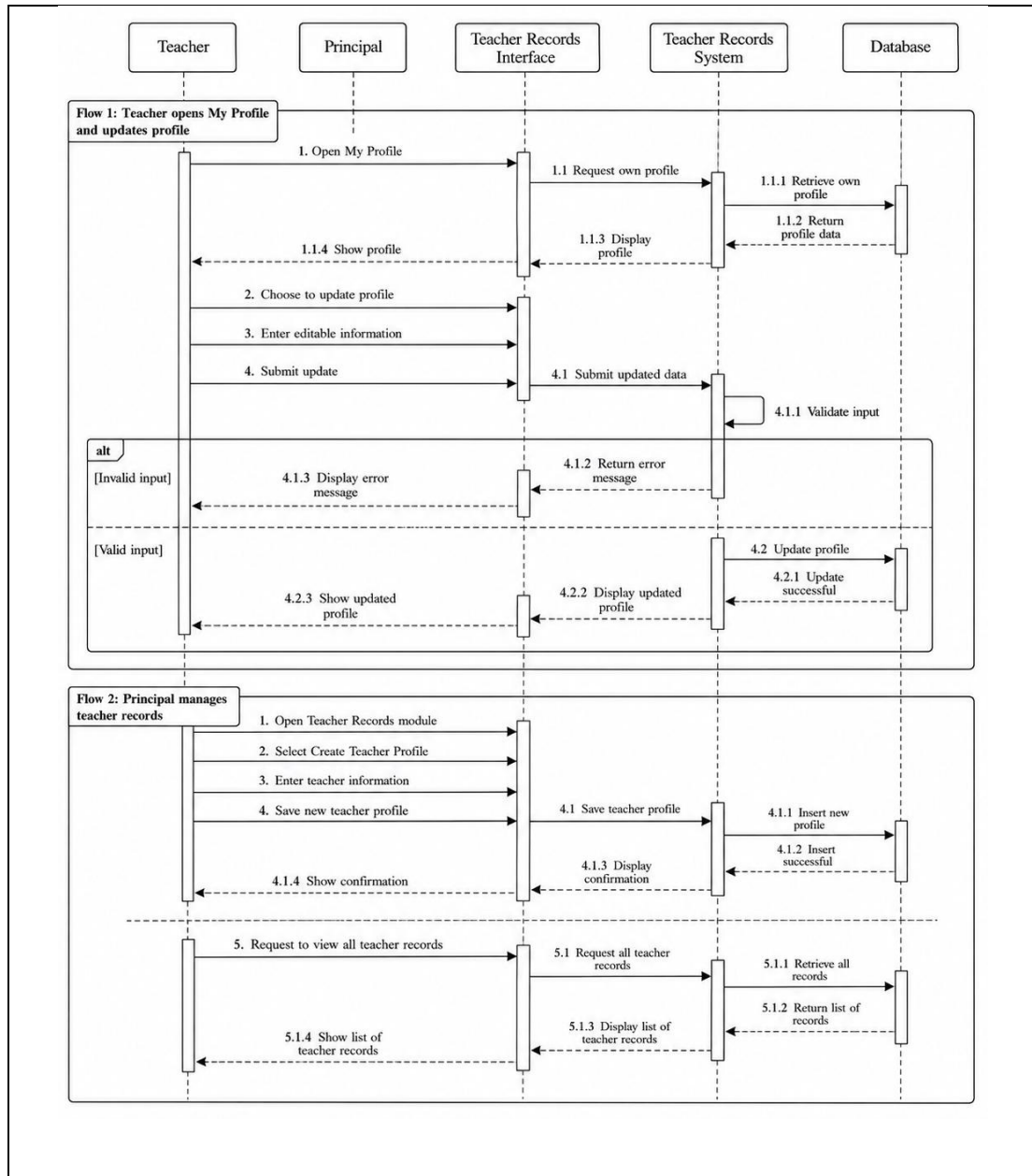


Figure 4.5.3.3.4 Sequence Diagram for Teacher Profile Management

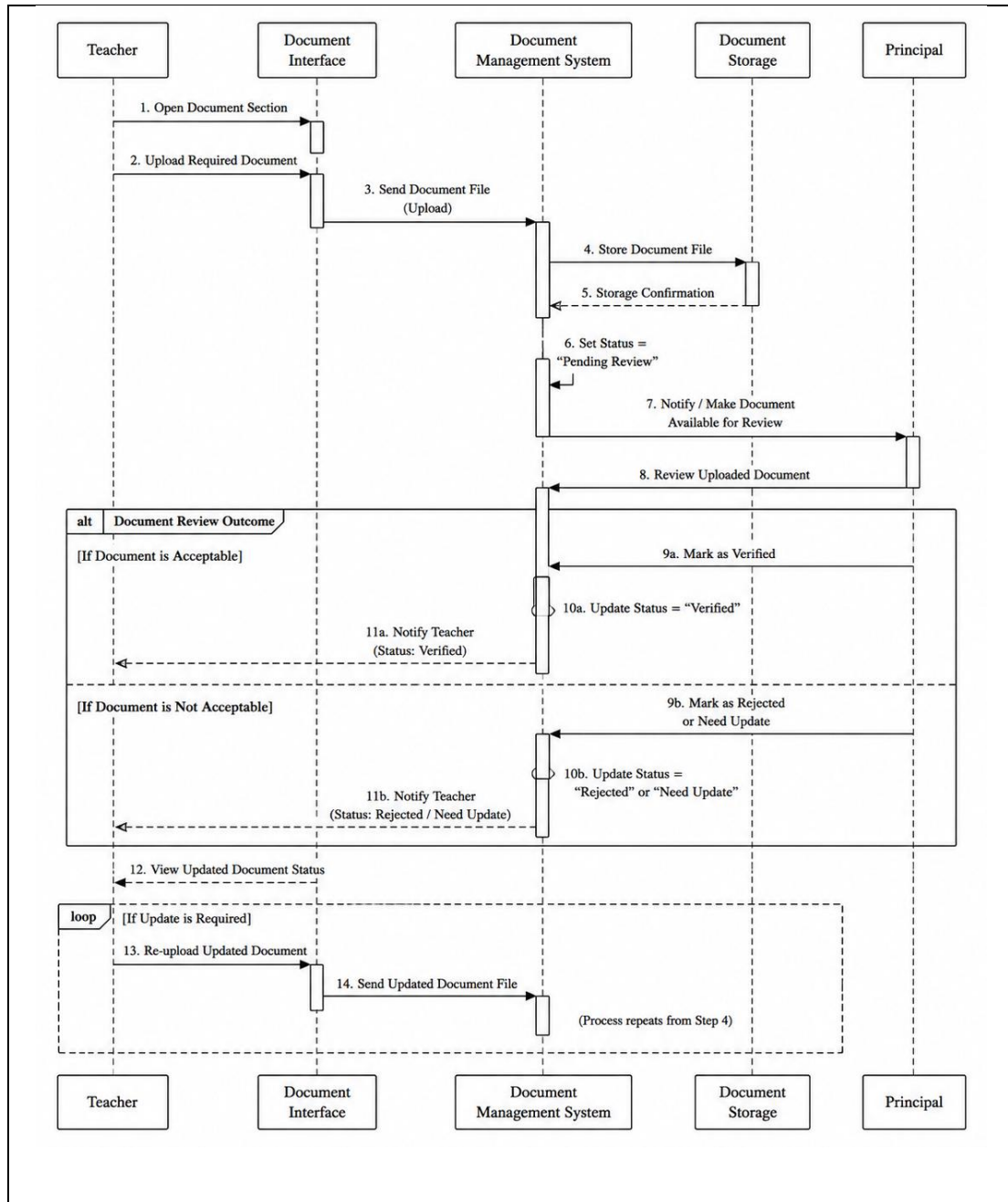


Figure 4.5.3.3.5 Sequence Diagram for Teacher Document Management

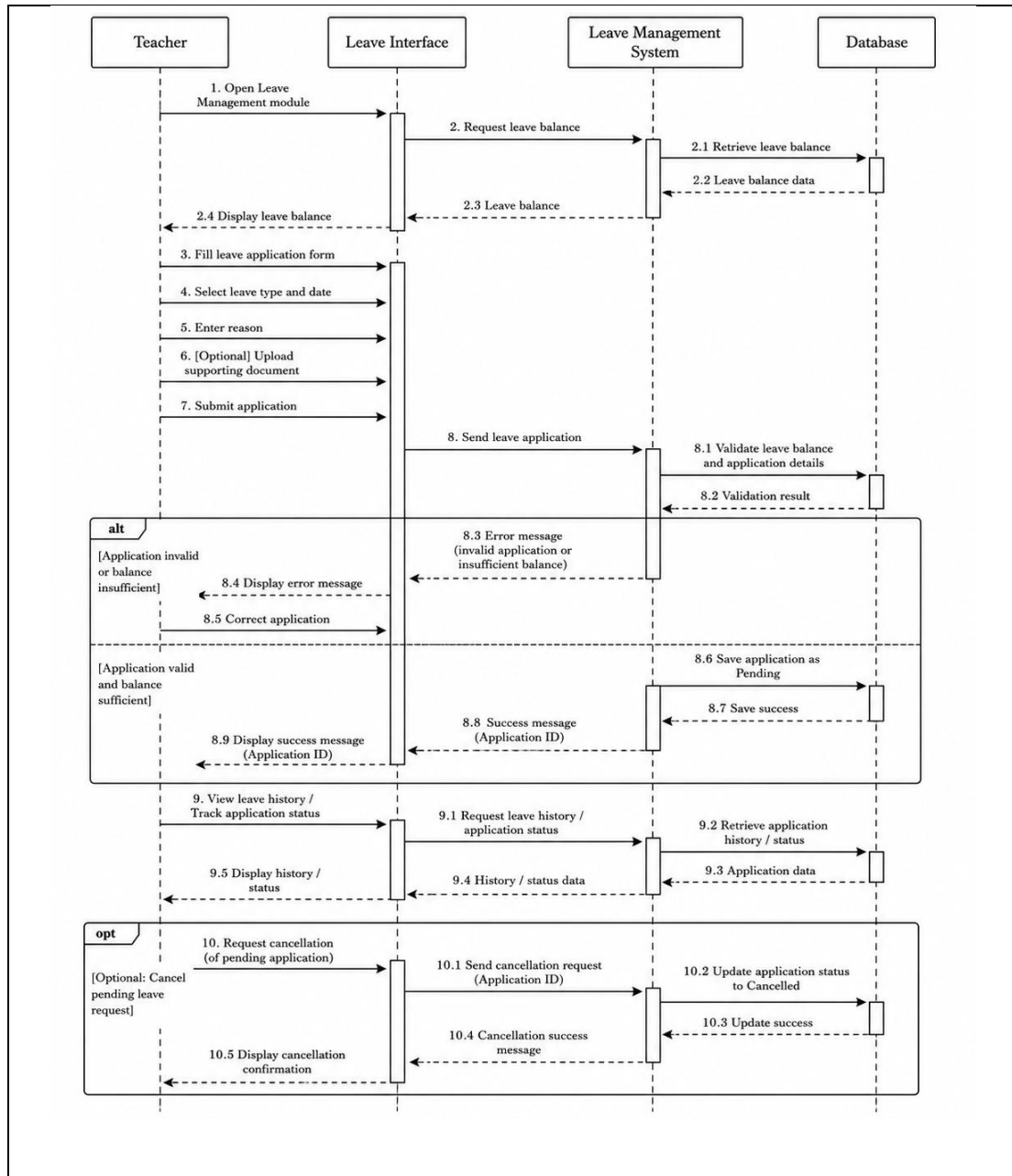


Figure 4.5.3.3.6 Sequence Diagram for Leave Application

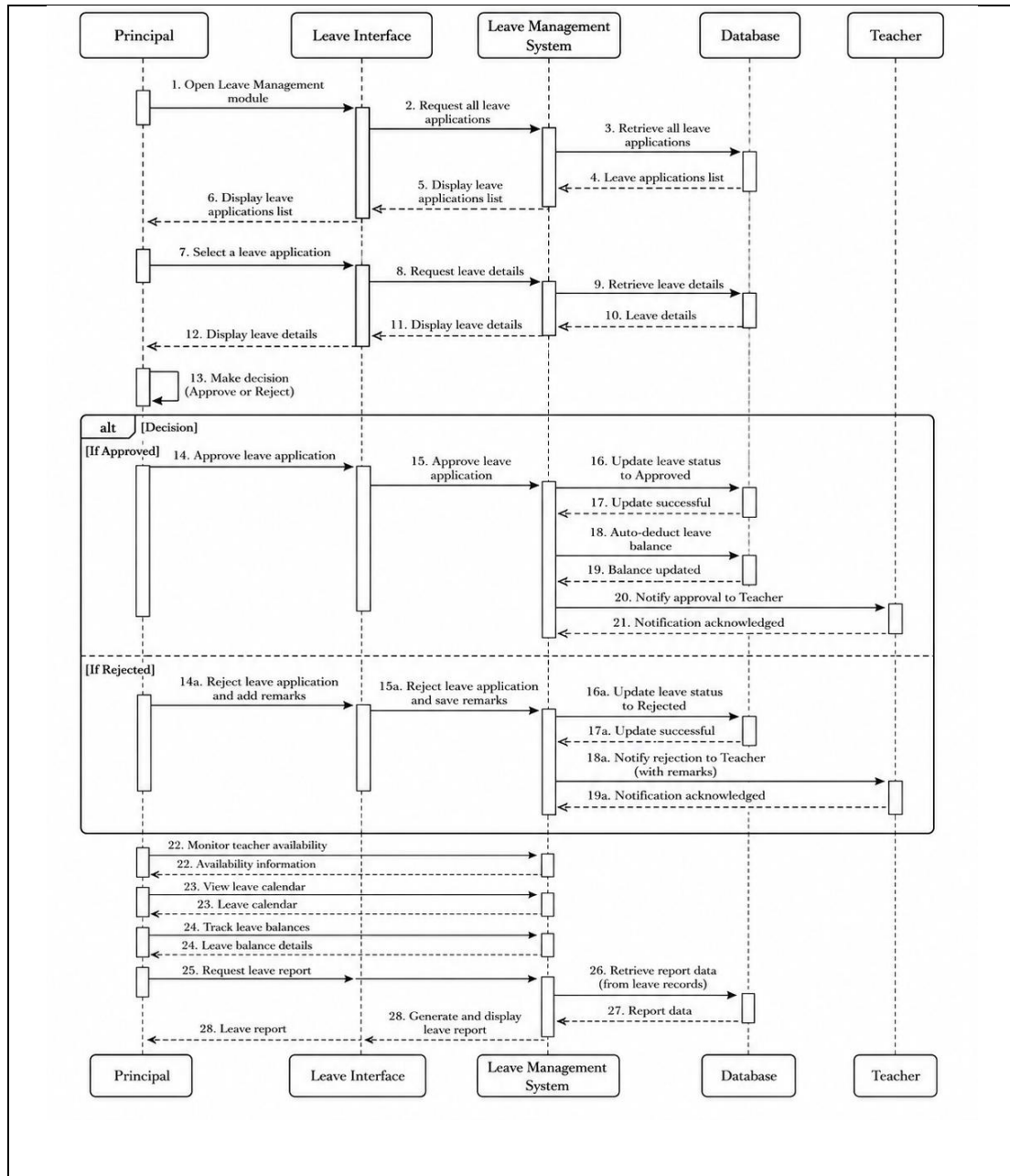


Figure 4.5.3.3.7 Sequence Diagram for Leave Approval and Reporting

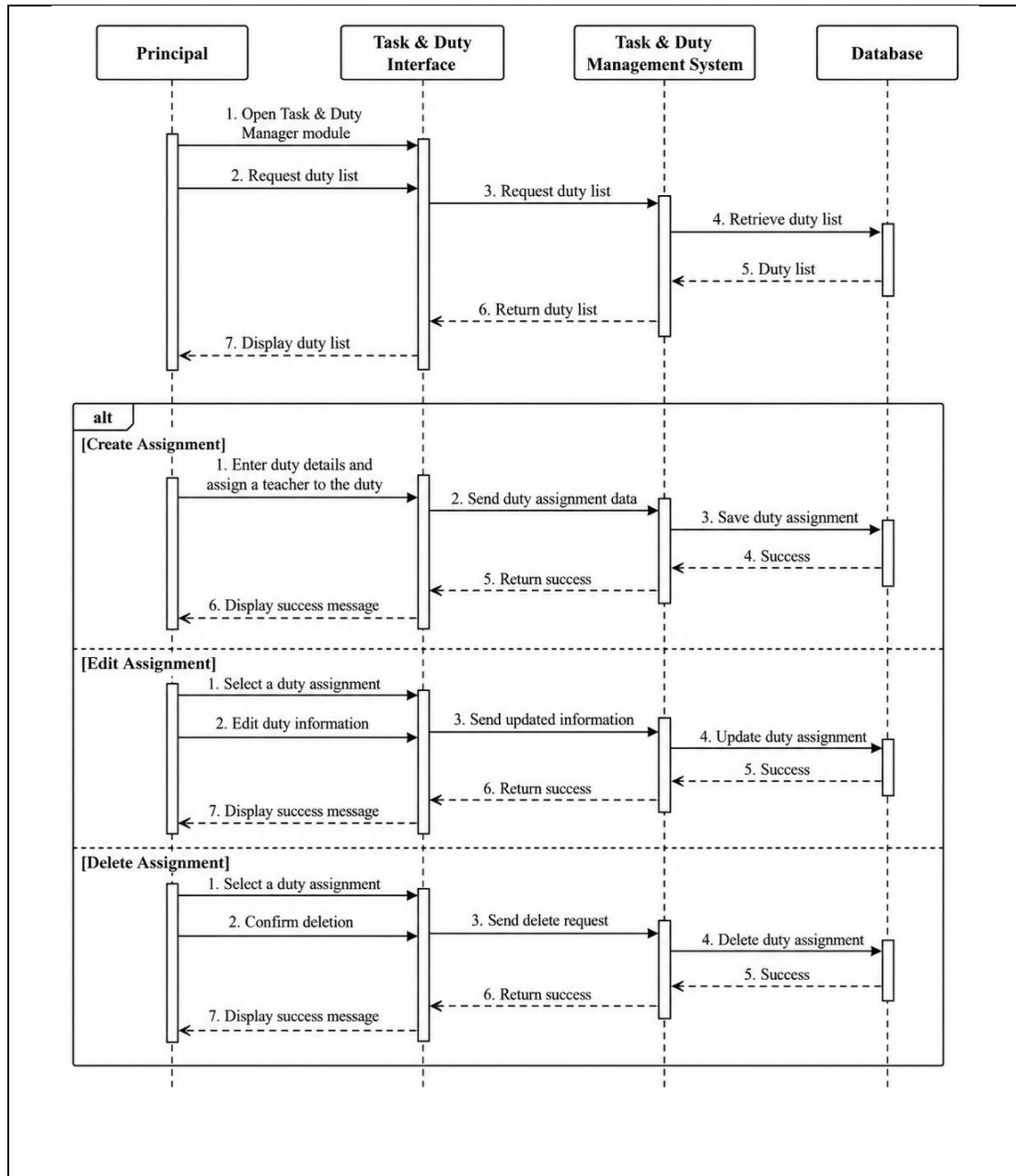


Figure 4.5.3.3.8 Sequence Diagram for Duty Assignment Management

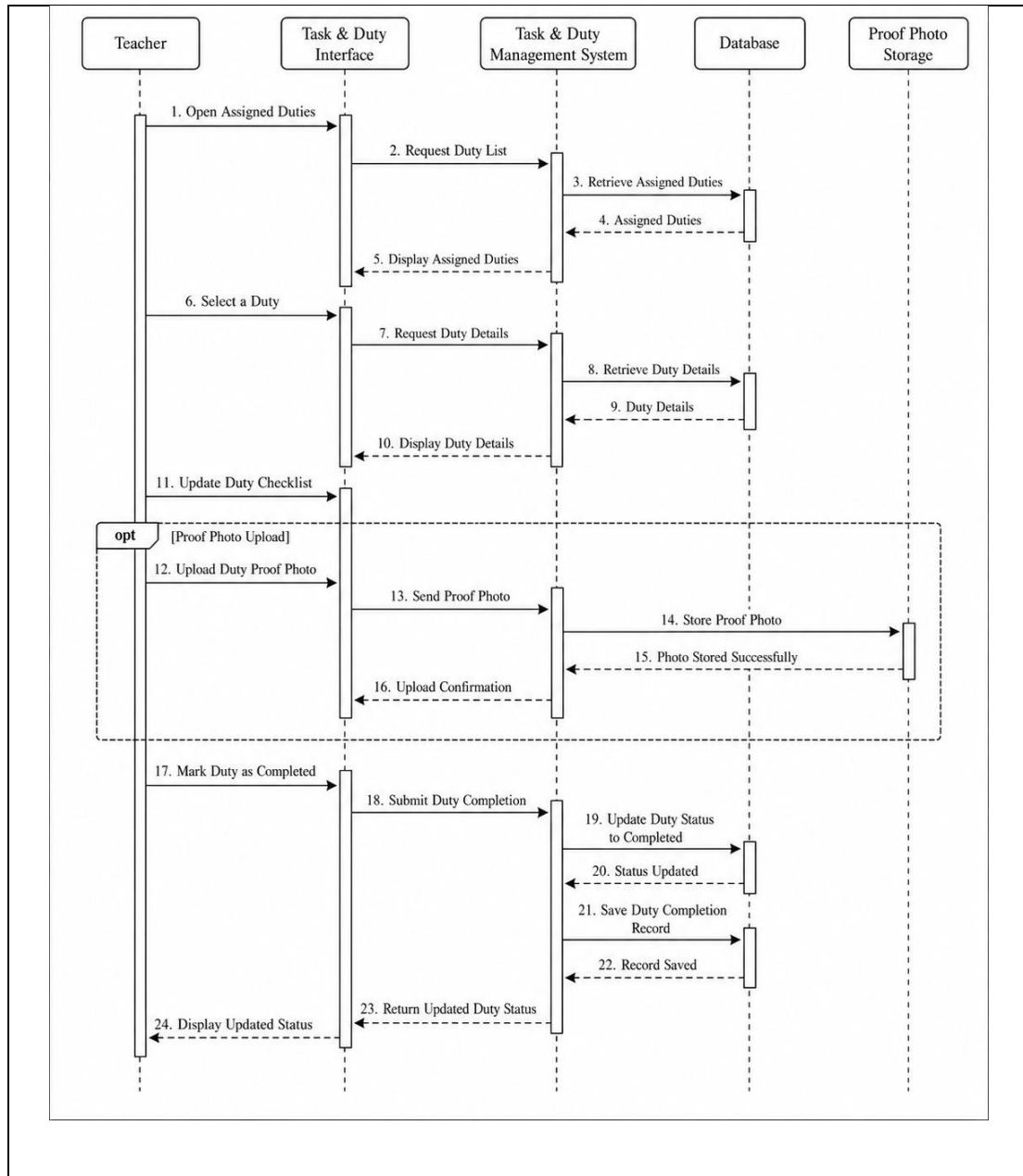


Figure 4.5.3.3.9 Sequence Diagram for Duty Completion Tracking

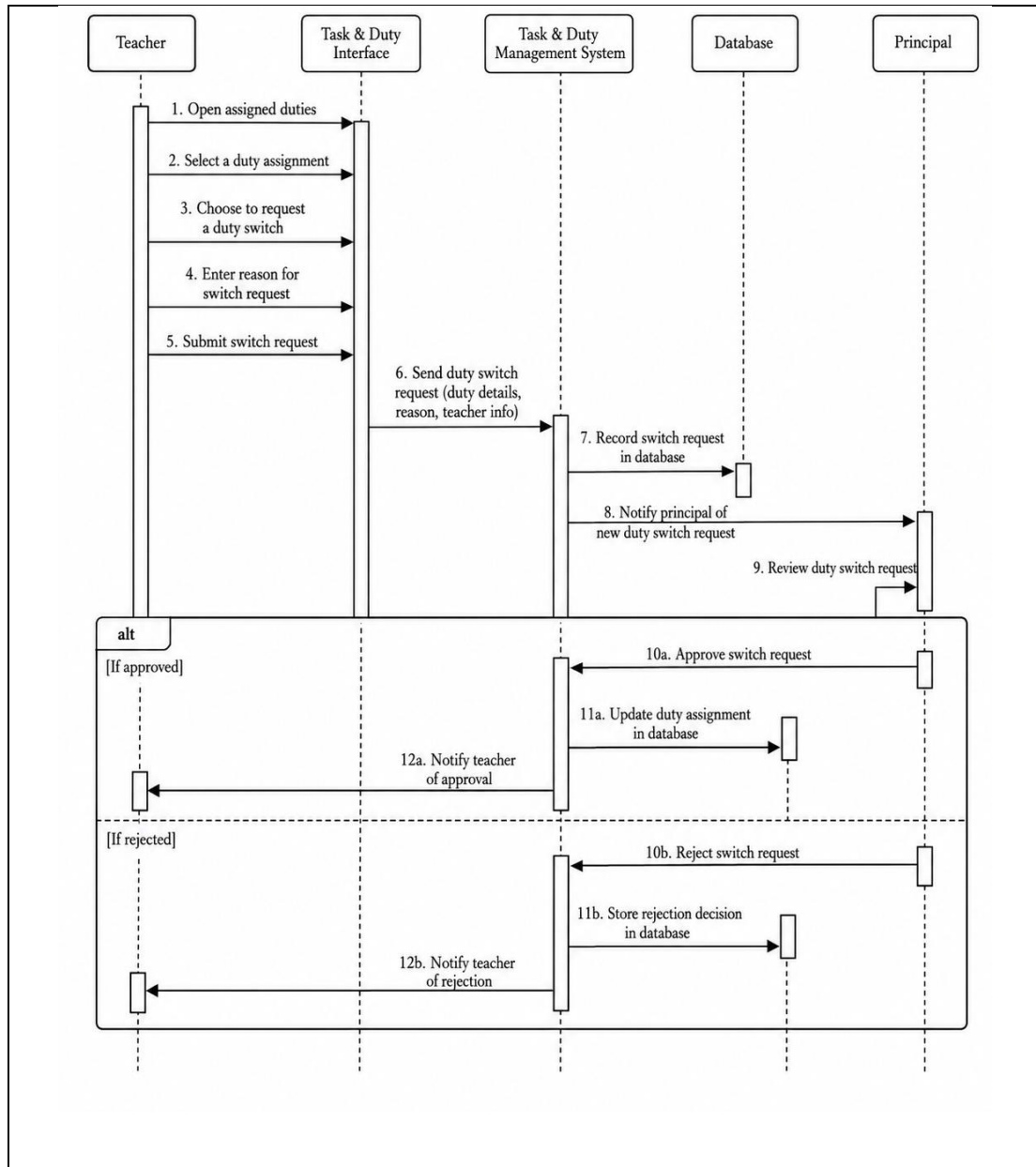


Figure 4.5.3.3.10 Sequence Diagram for Duty Switch Request

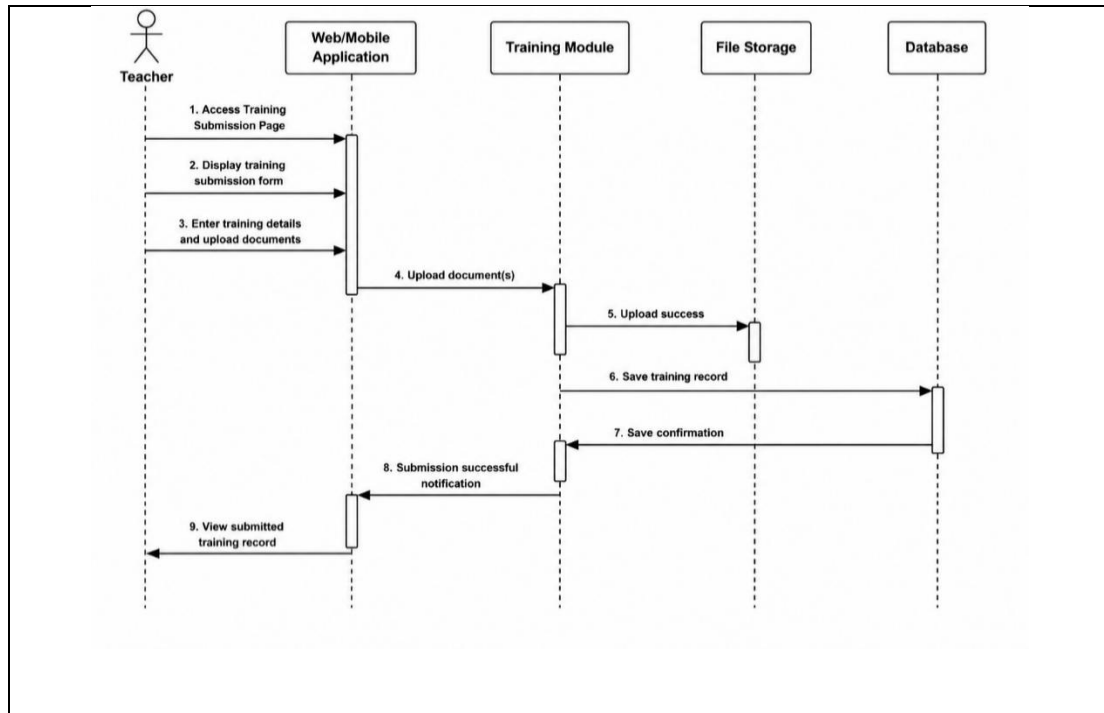


Figure 4.5.3.3.11 Sequence Diagram for Training Record Submission

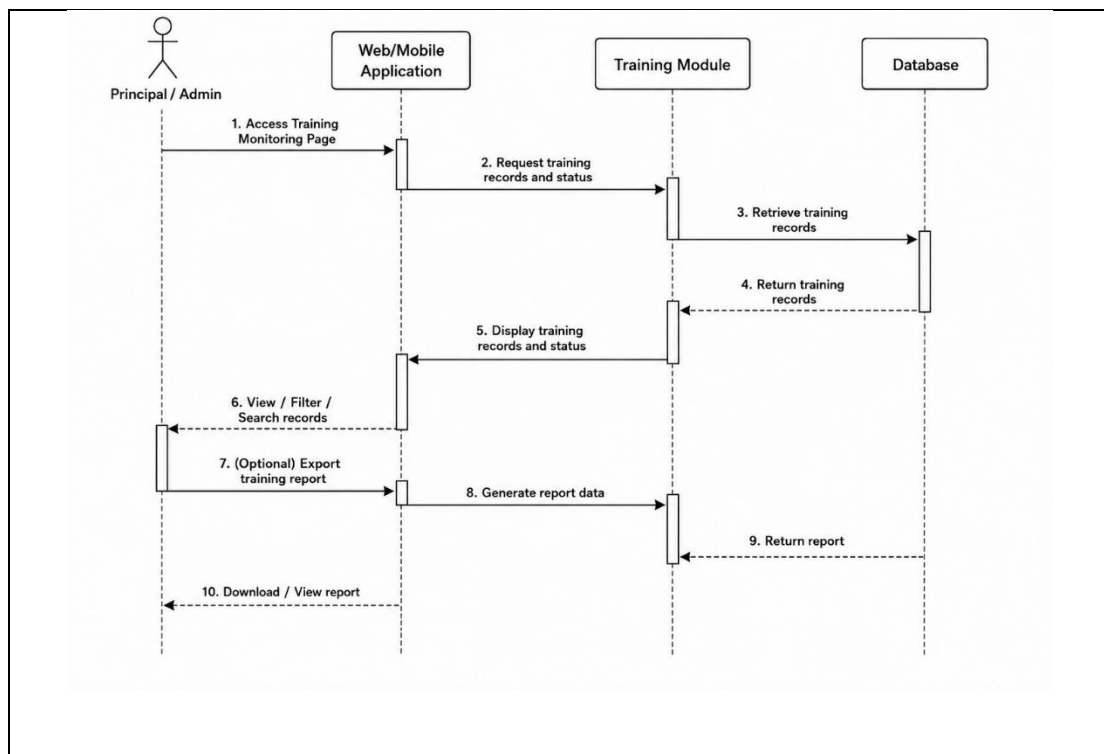


Figure 4.5.3.3.12 Sequence Diagram for Training Monitoring

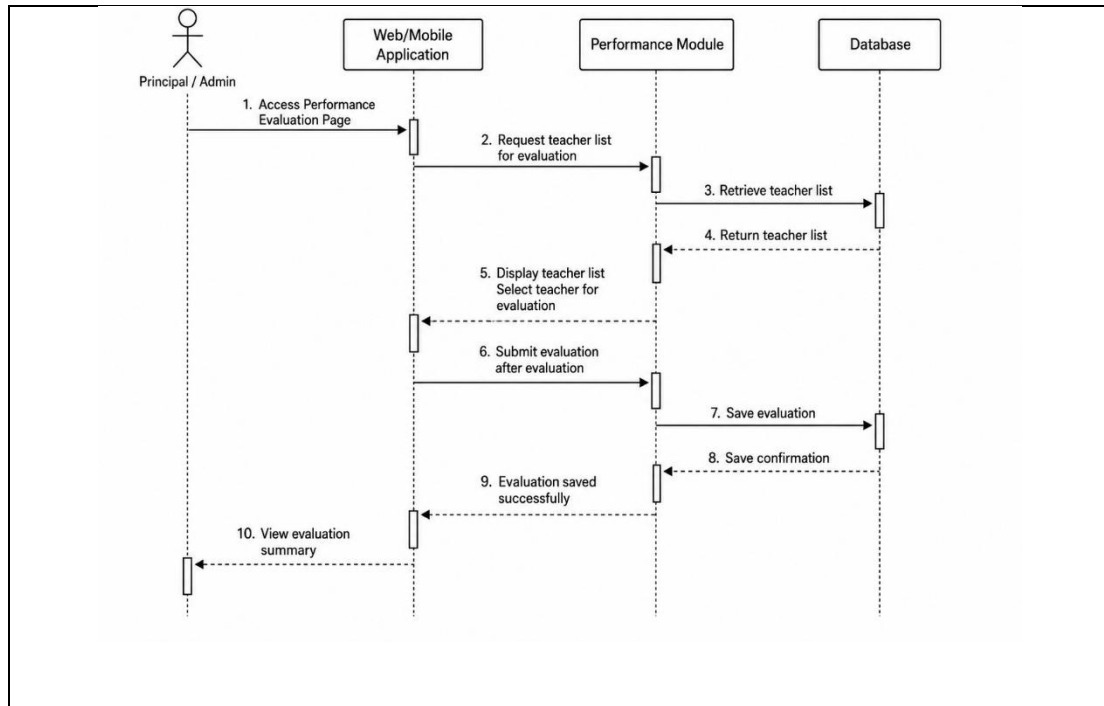


Figure 4.5.3.13 Sequence Diagram for Performance Evaluation Management

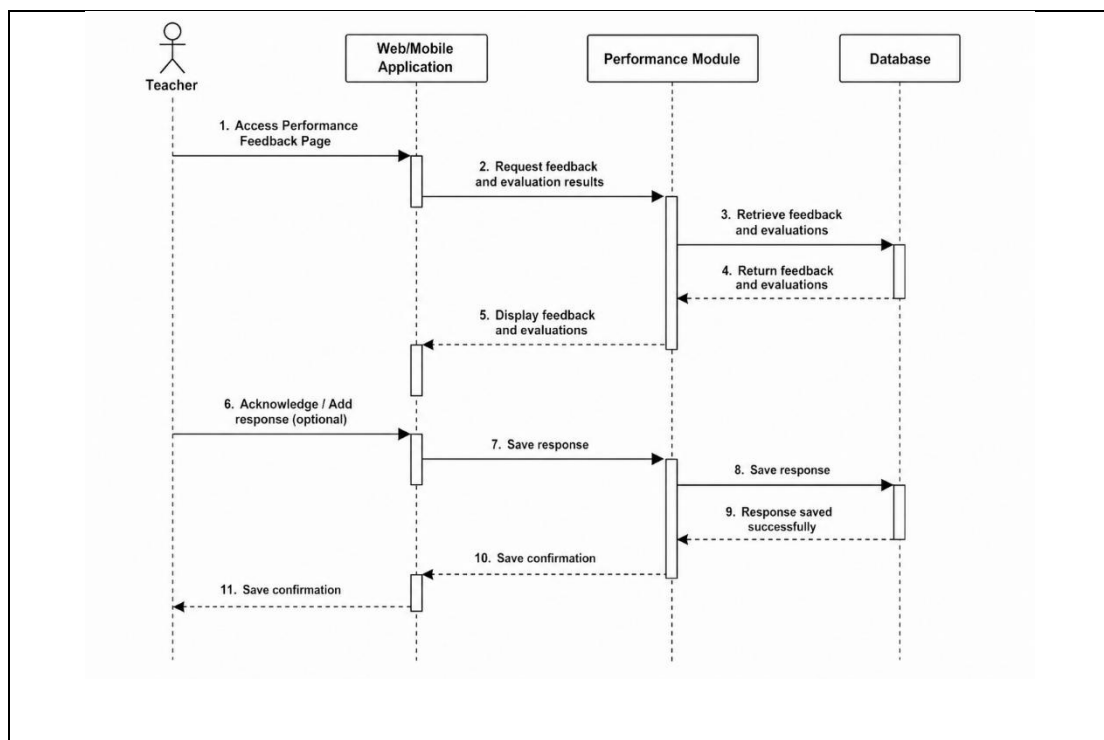


Figure 4.5.3.14 Sequence Diagram for Performance Feedback

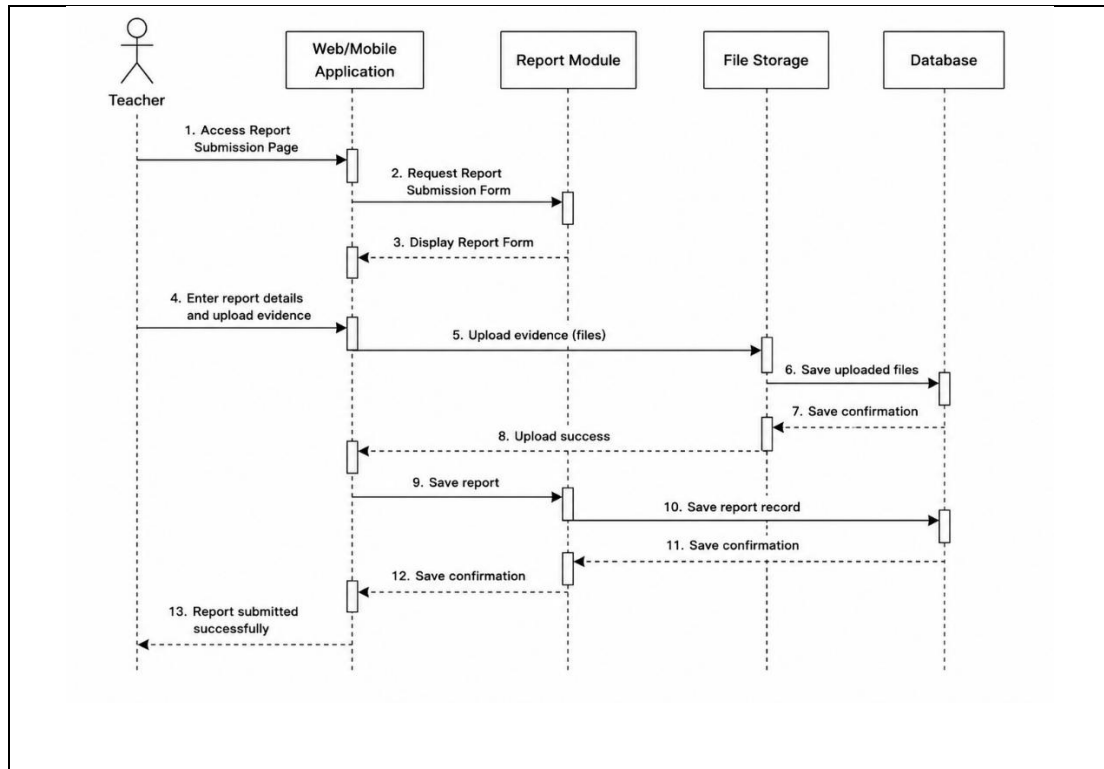


Figure 4.5.3.3.15 Sequence Diagram for Report Submission

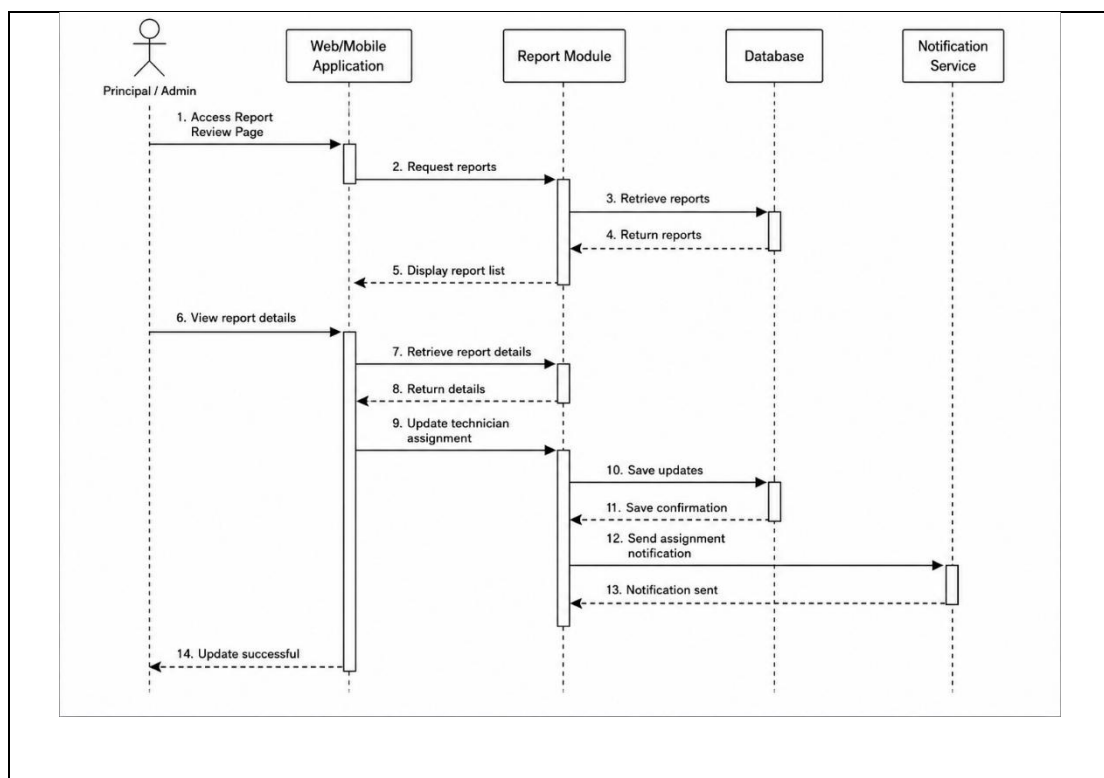


Figure 4.5.3.3.16 Sequence Diagram for Report Review and Assignment

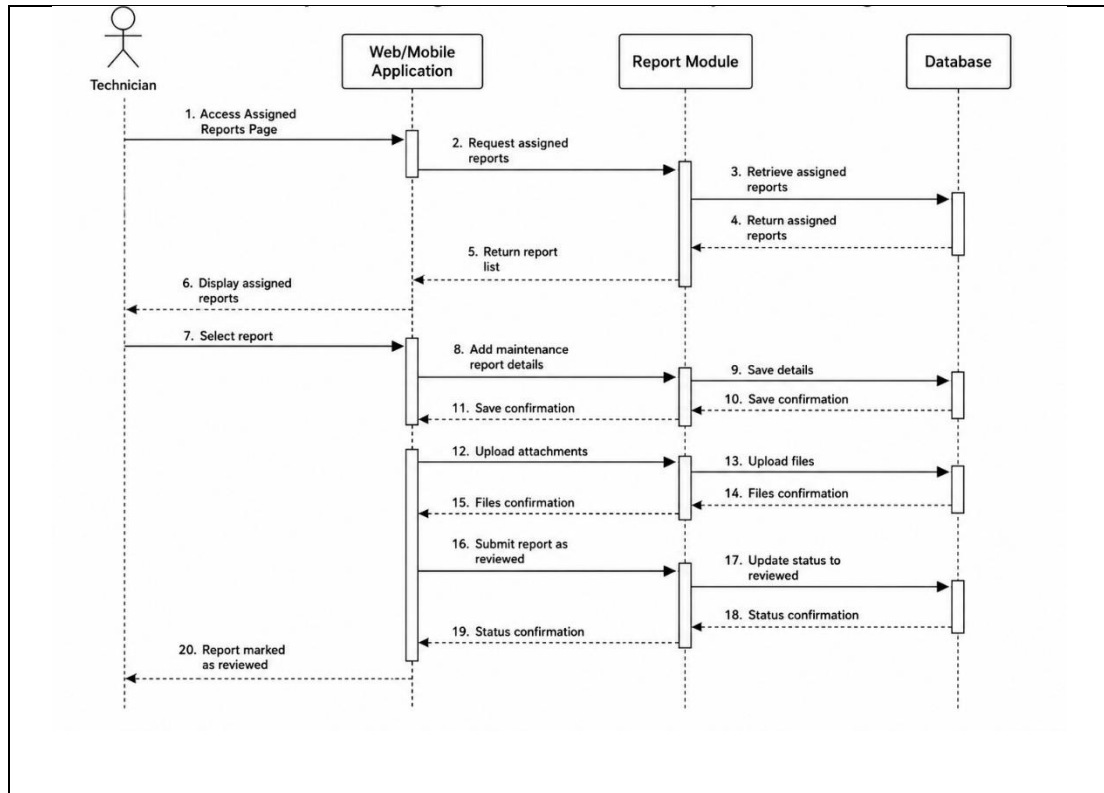


Figure 4.5.3.3.17 Sequence Diagram for Technician Report Handling

4.5.4 Database Design

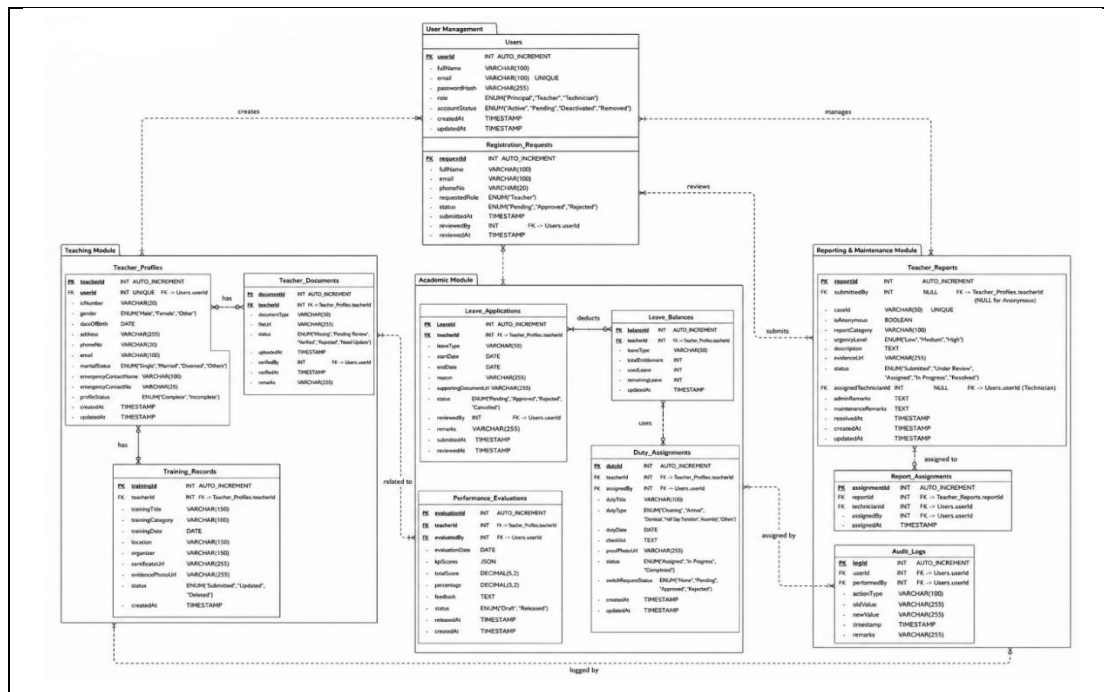


Figure 4.5.4.1 Overall Database Design for Centralized Teacher Management System

4.5.5 Interface Design

4.5.5.1 Overall Interface Design List

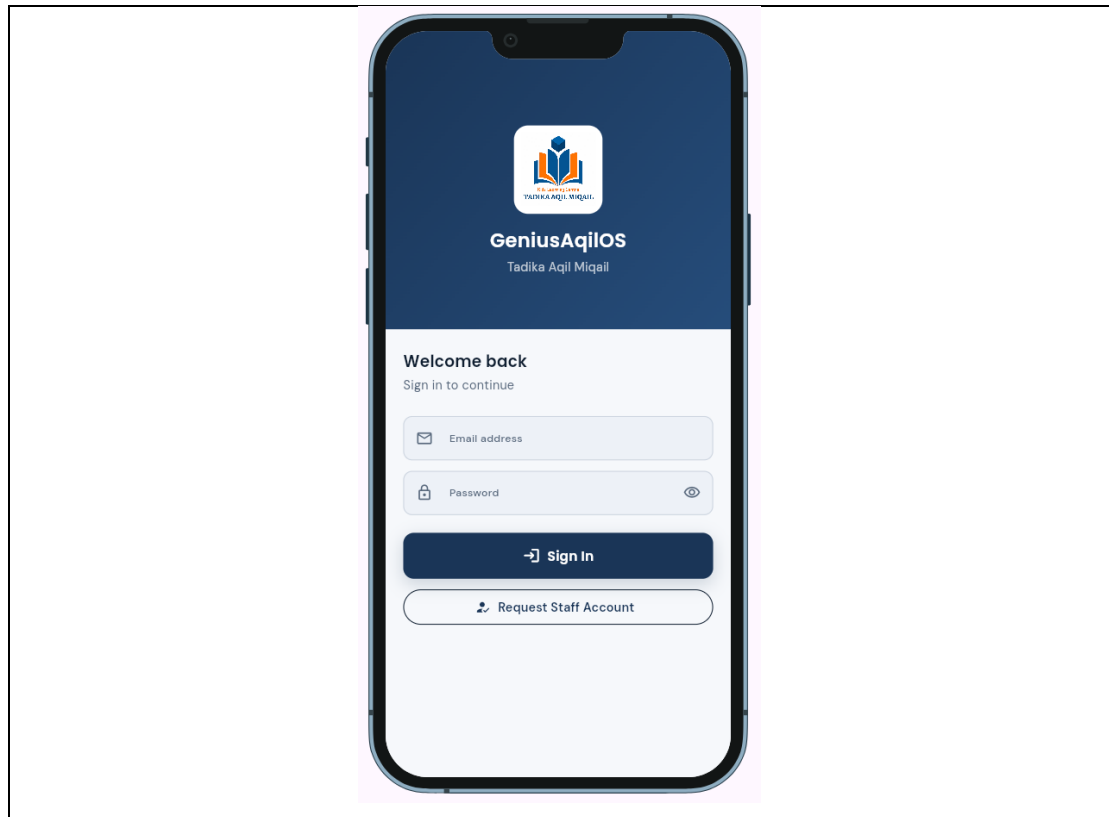


Figure 4.5.5.1.1 Login Interface

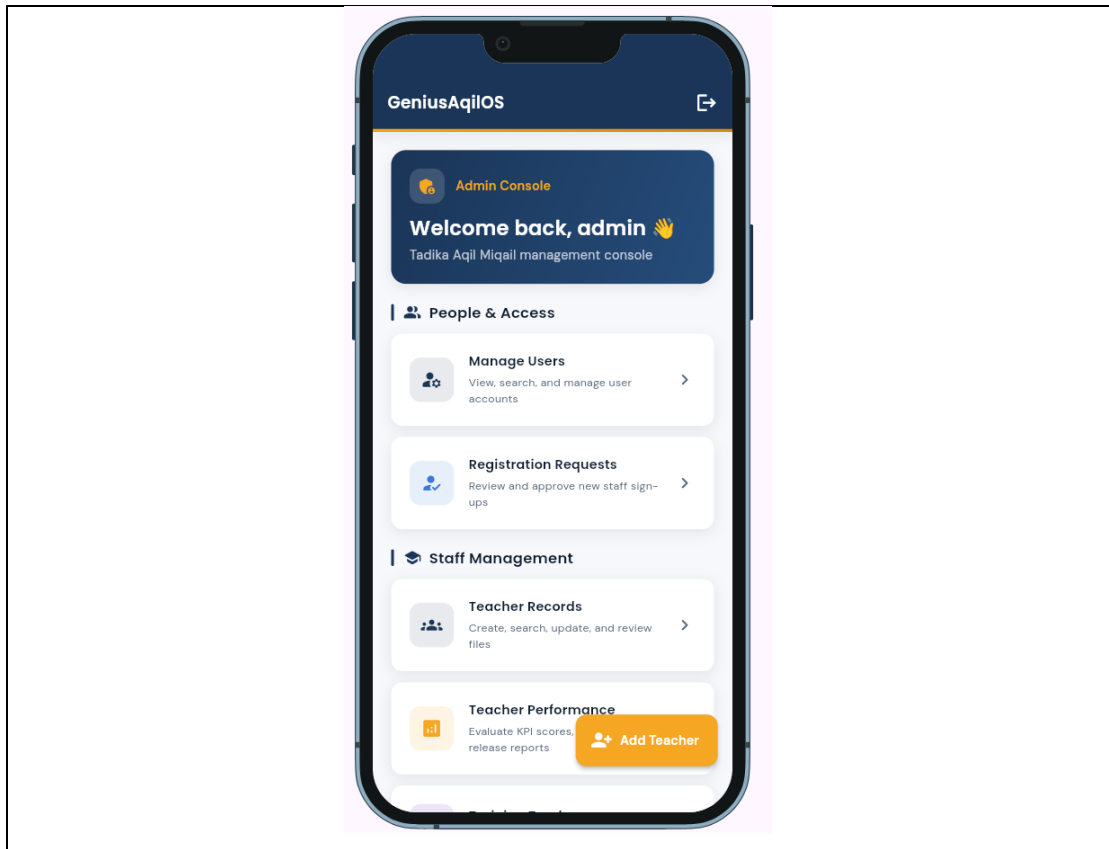


Figure 4.5.5.1.2 Admin (Principal) Dashboard Interface

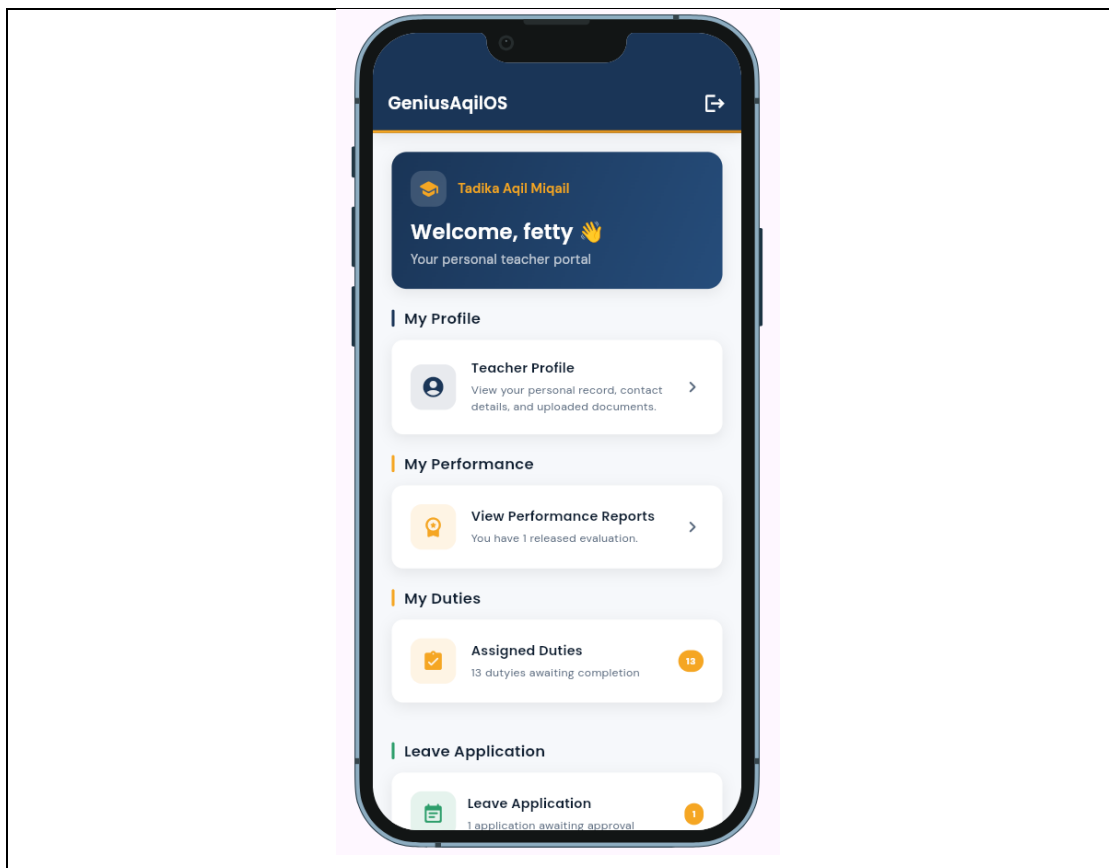


Figure 4.5.5.1.3 Teacher Dashboard Interface

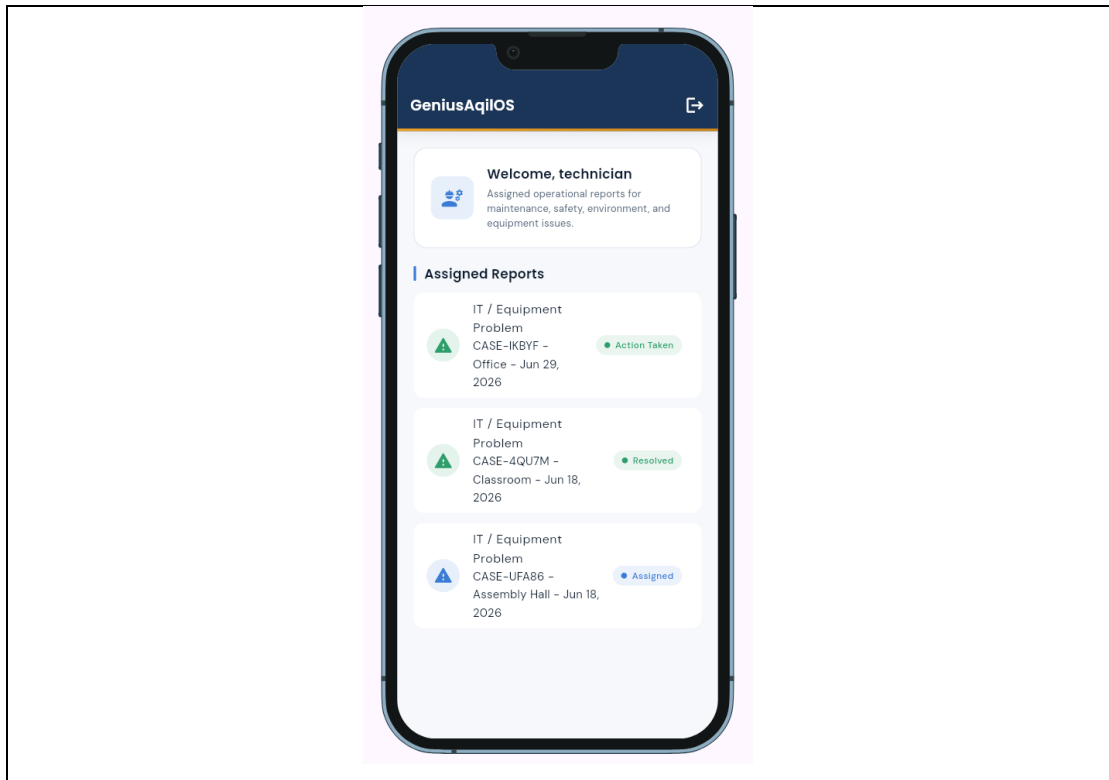


Figure 4.5.5.1.4 Technician Dashboard Interface

4.5.5.2 Registration Module

The screenshot shows the Staff Registration form. It starts with a back arrow and the title "Staff Registration". Below that, it says "Tadika Aqil Miqail" and "Create Staff Account Request". A note states: "New teachers can submit their basic profile credentials. The principal must approve the request before login access is created." The form contains the following fields:

- Full Name
- Email Address
- Phone Number
- Password (with an eye icon to toggle visibility)
- Confirm Password (with an eye icon to toggle visibility)

At the bottom, there is a dark blue button labeled "Submit Request" with a person icon.

Figure 4.5.5.2.1 Teacher Registration Form Interface

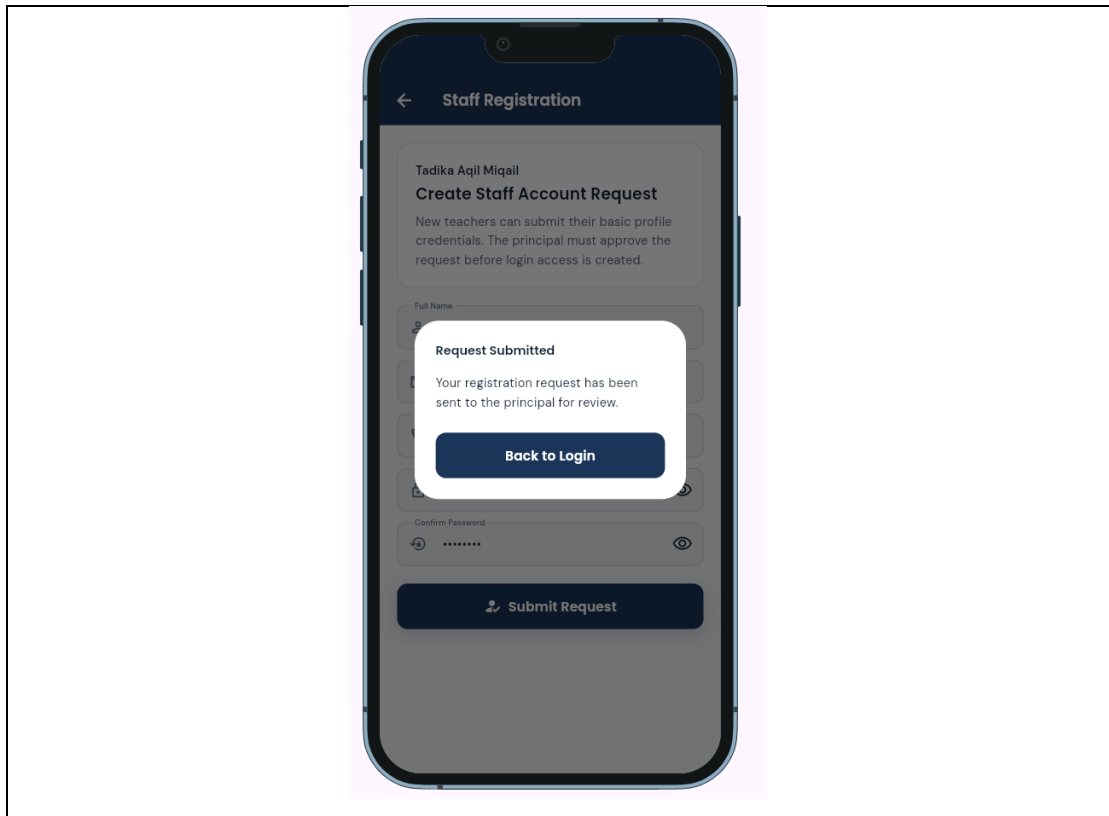


Figure 4.5.5.2.2 Registration Submitted / Pending Approval Interface

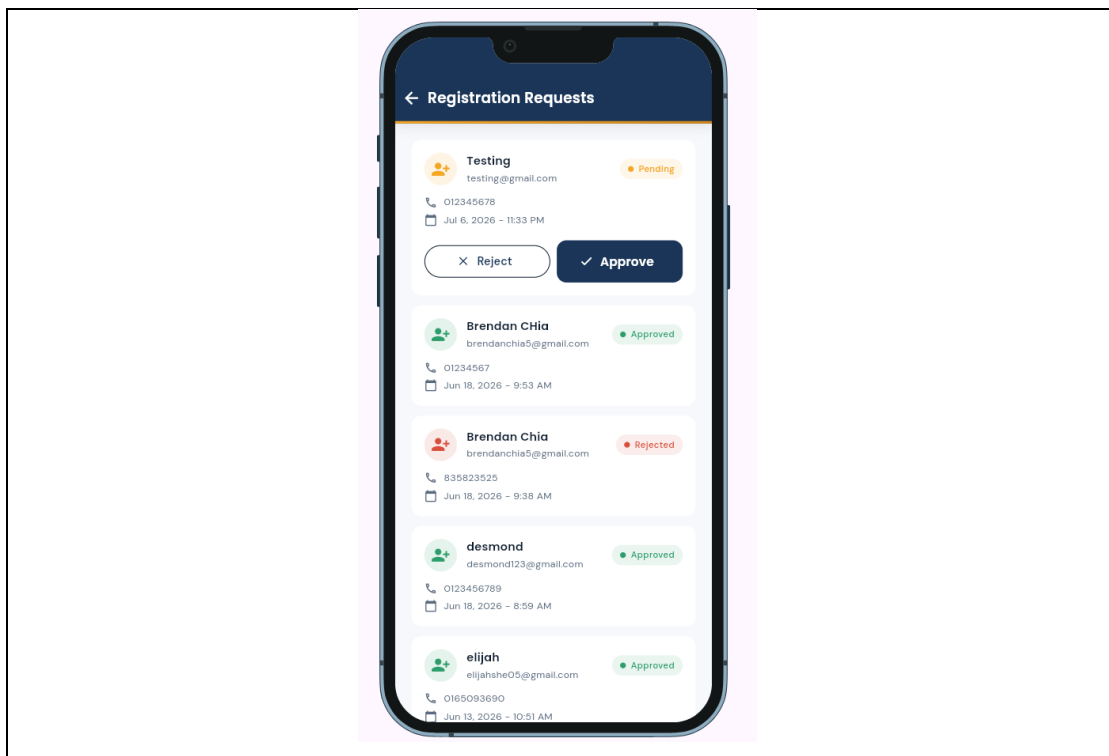


Figure 4.5.5.2.3 Registration Request List Interface

4.5.5.3 Manage User Module

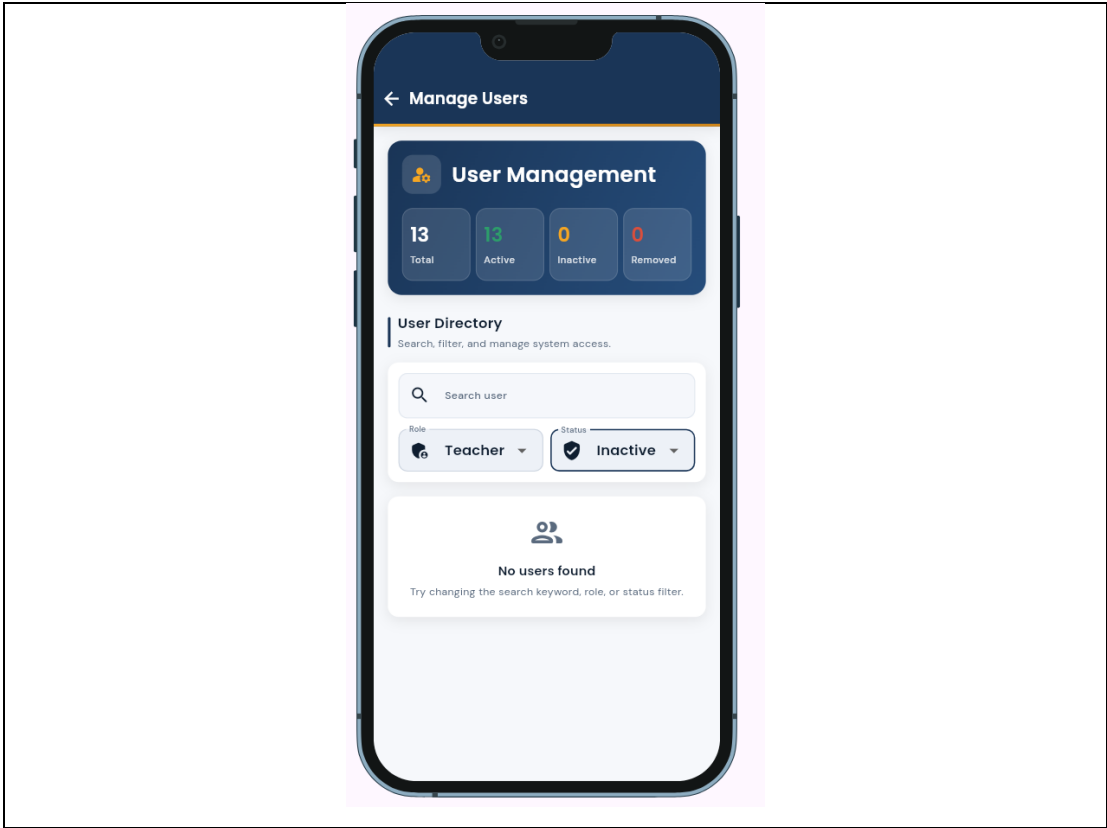


Figure 4.5.5.3.1 Manage User List Interface

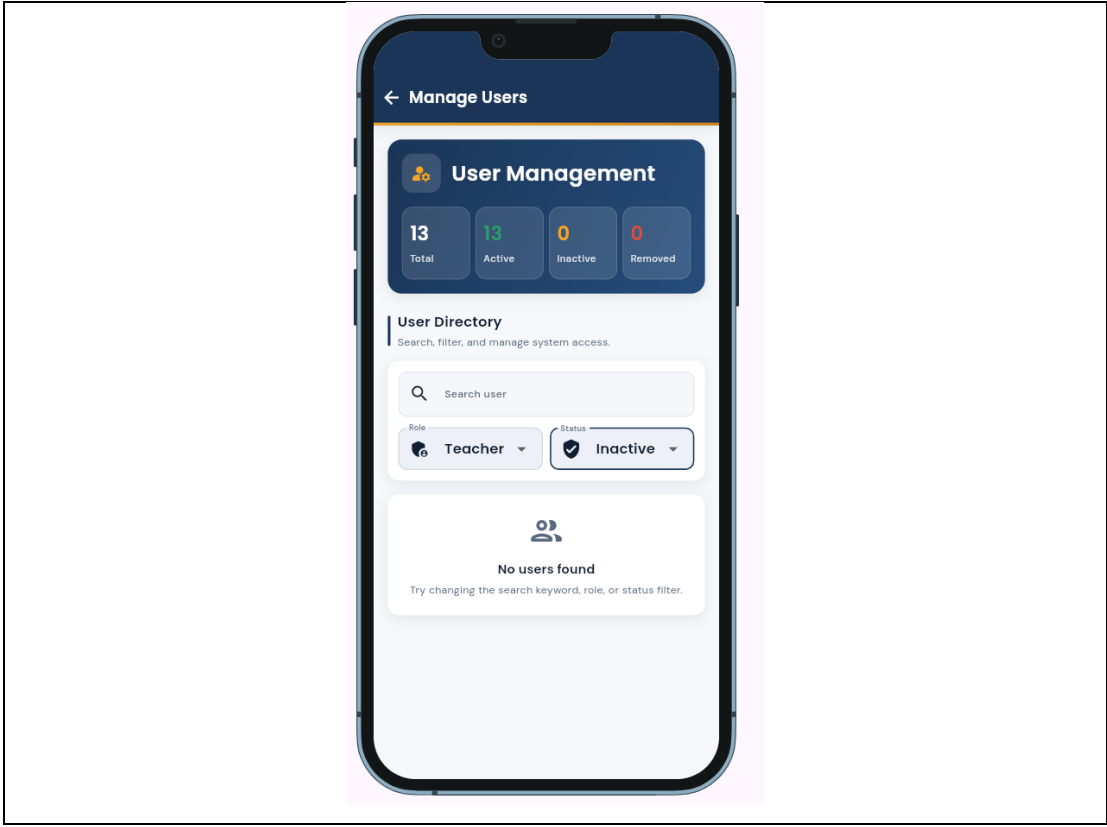


Figure 4.5.5.3.2 Search and Filter User Interface

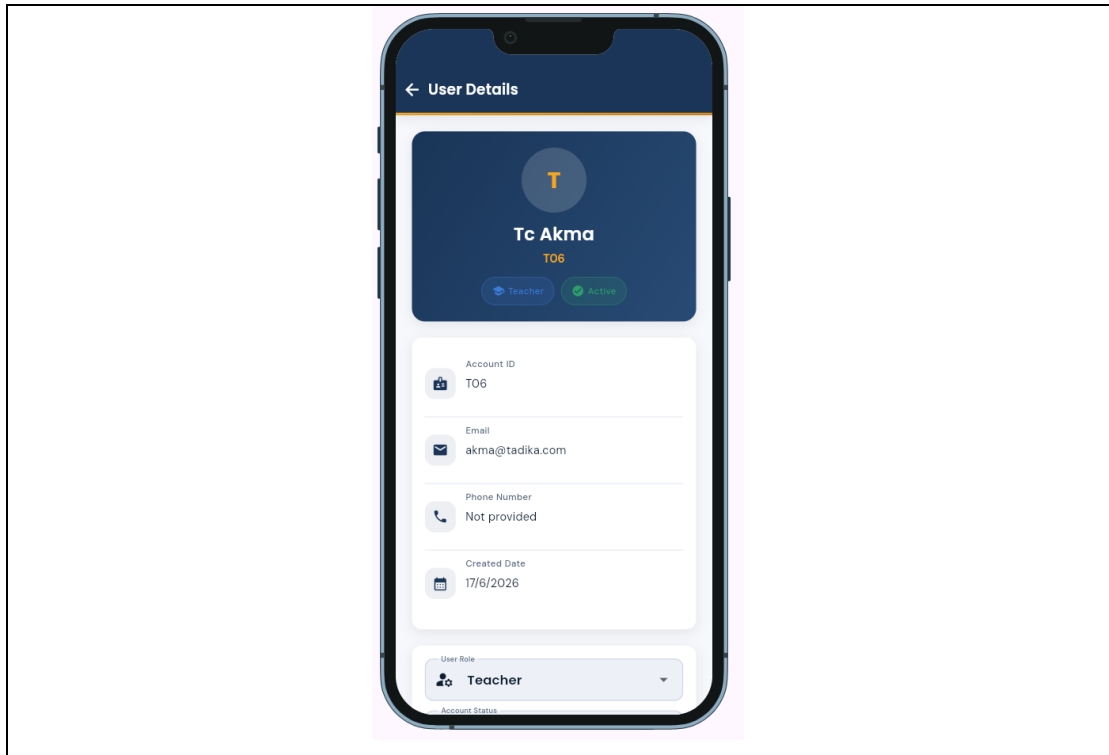


Figure 4.5.5.3.3 User Account Details Interface

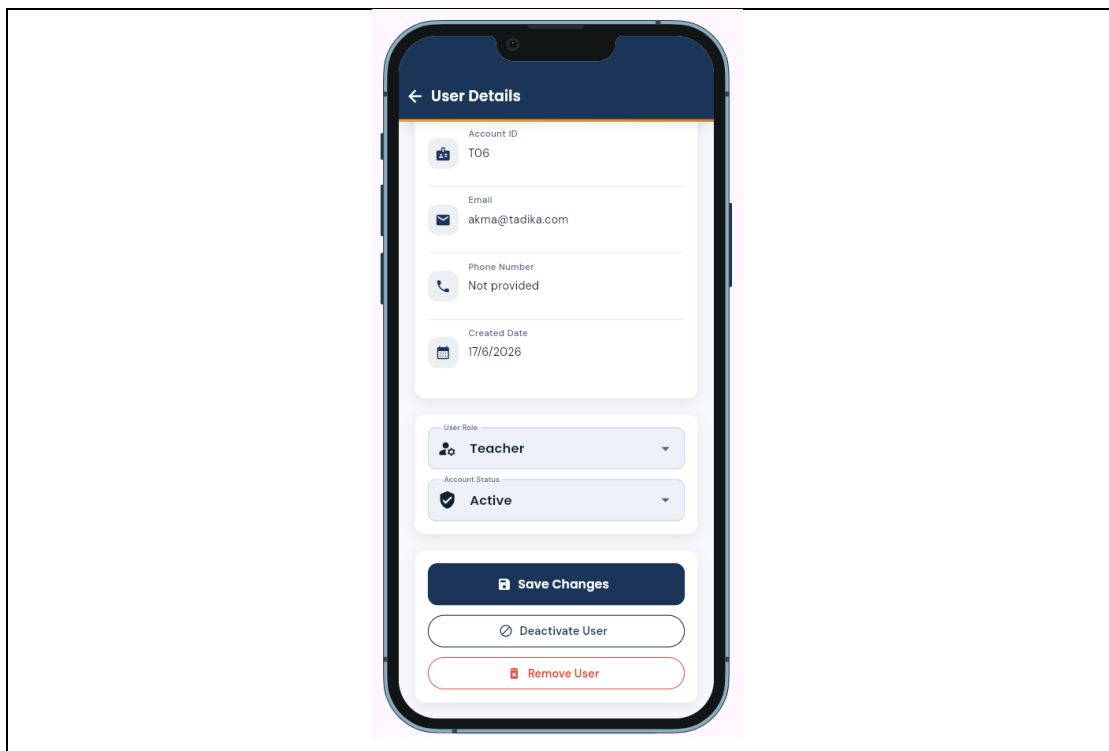


Figure 4.5.5.3.4 Deactivate / Remove User Account Interface

4.5.5.4 Teacher Records Module

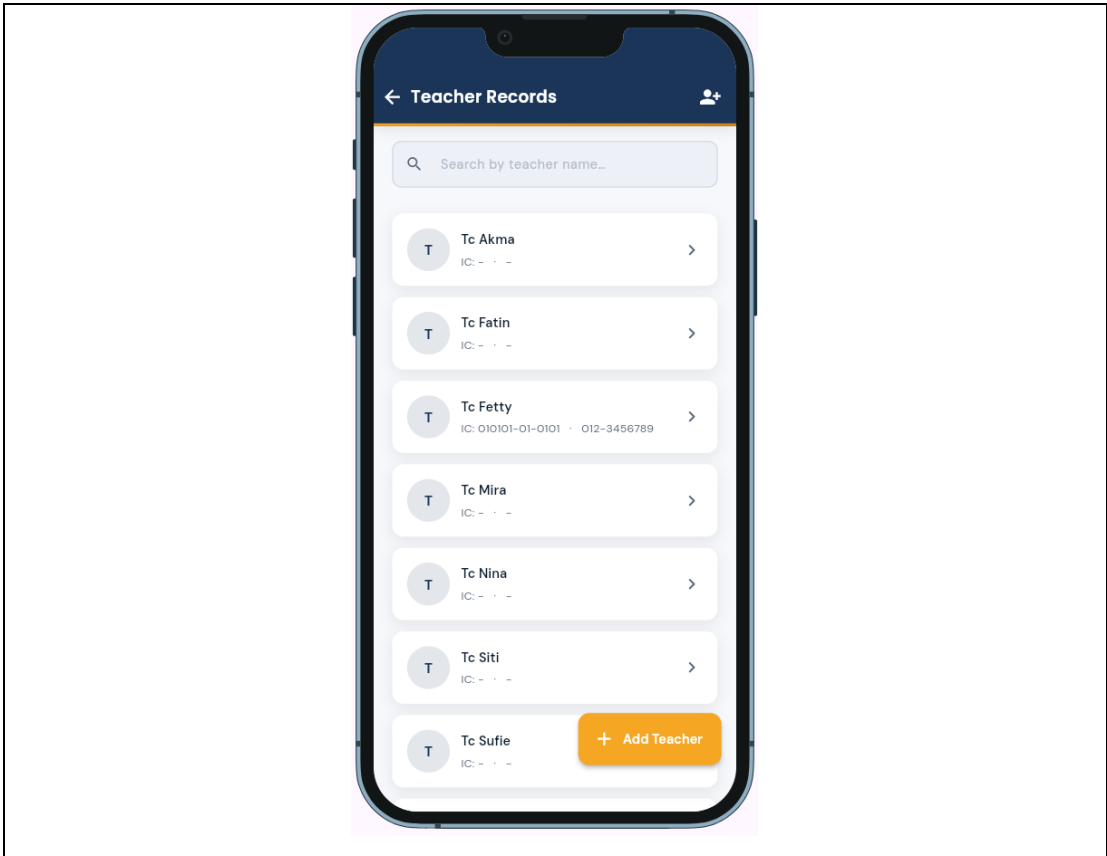


Figure 4.5.5.4.1 Teacher Records List Interface

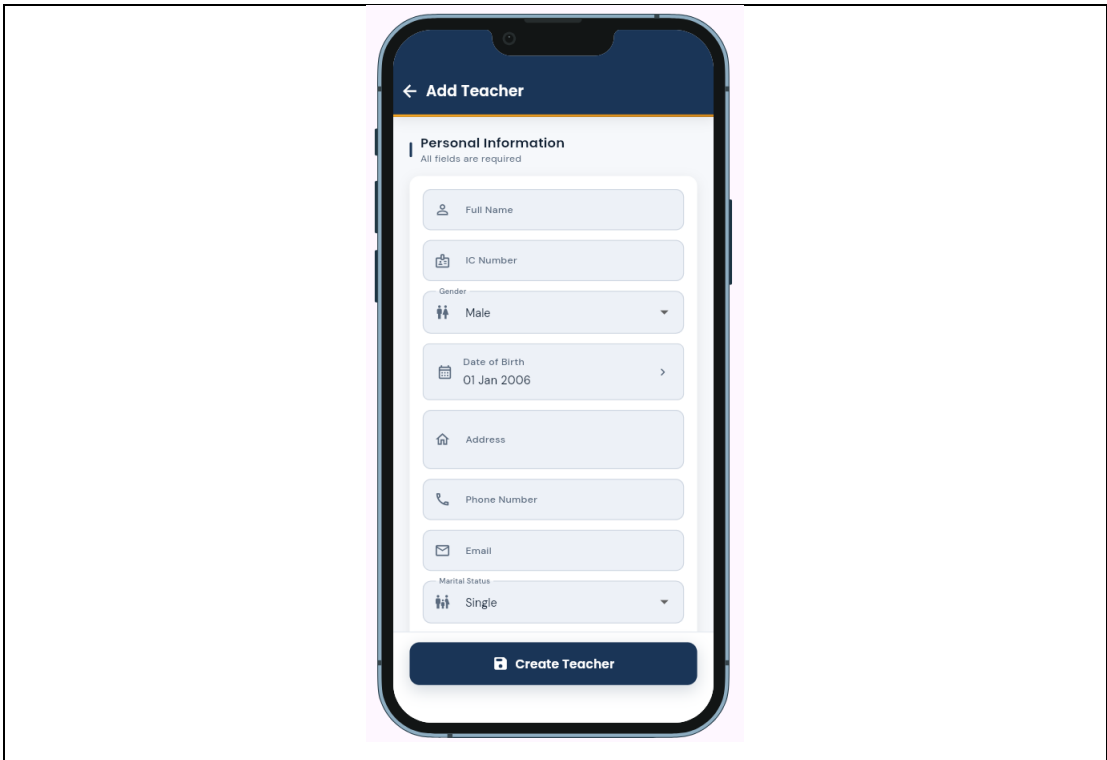


Figure 4.5.5.4.2 Create Teacher Profile Interface

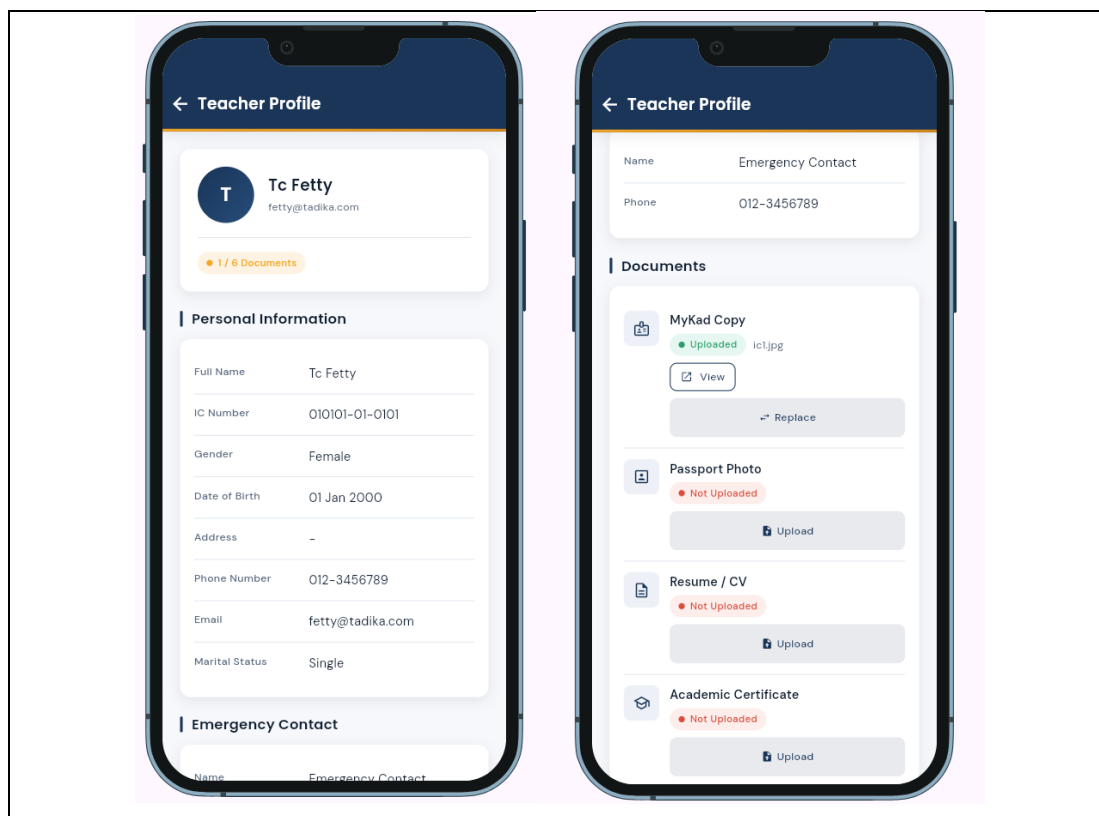


Figure 4.5.5.4.3 Teacher Profile Details Interface

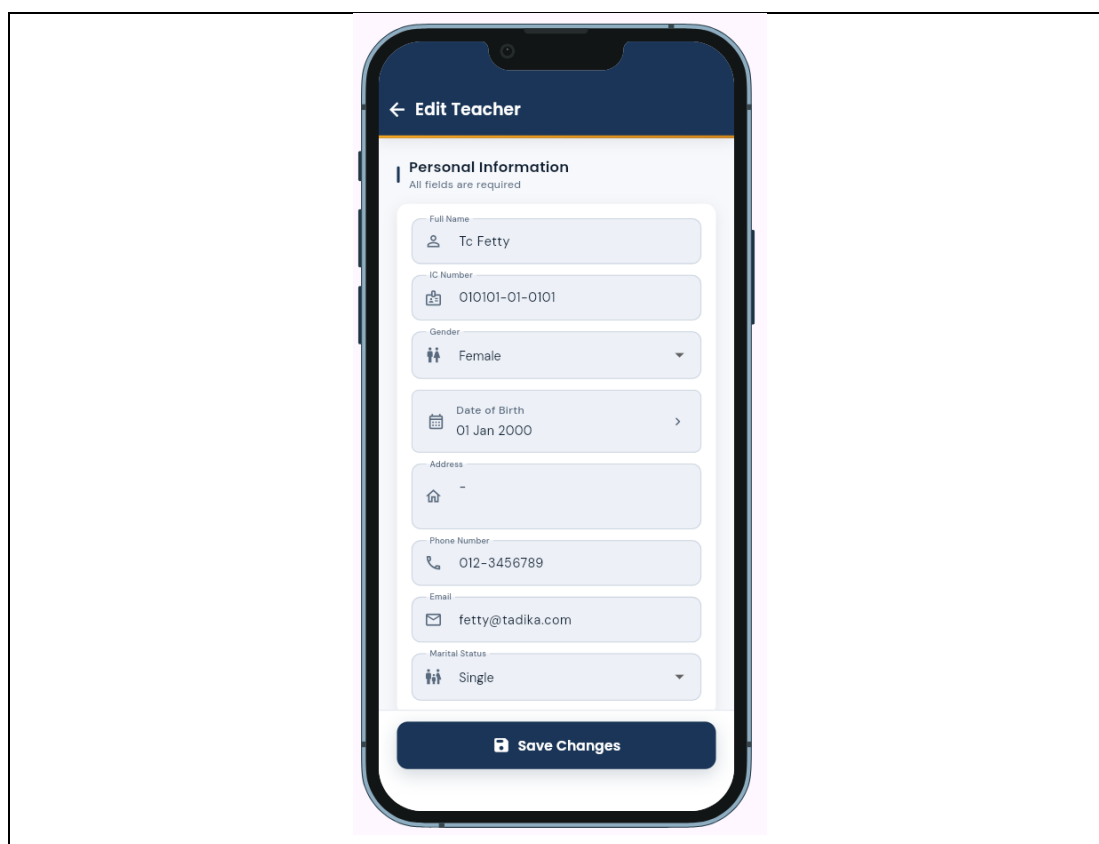


Figure 4.5.5.4.4 Edit Teacher Profile Interface

4.5.5.5 Leave Management Module

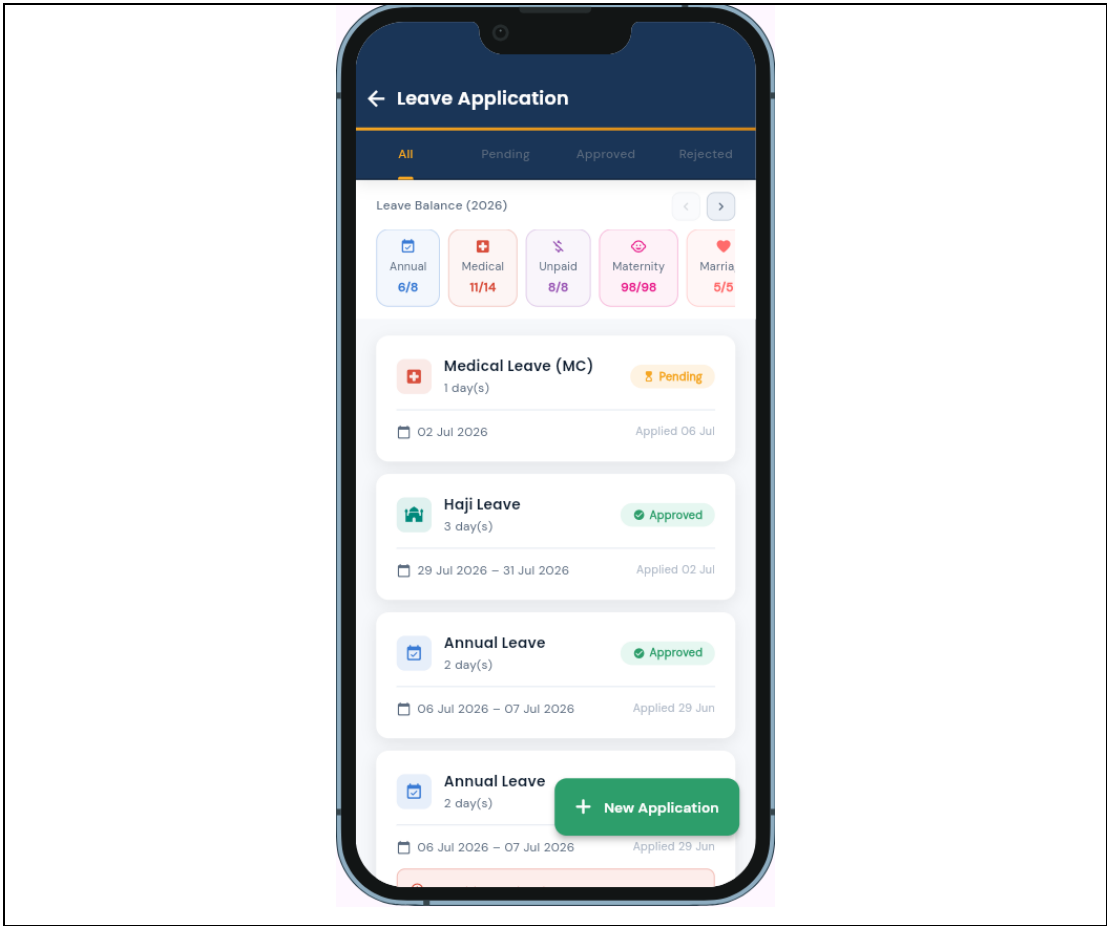


Figure 4.5.5.5.1 Leave Application Interface

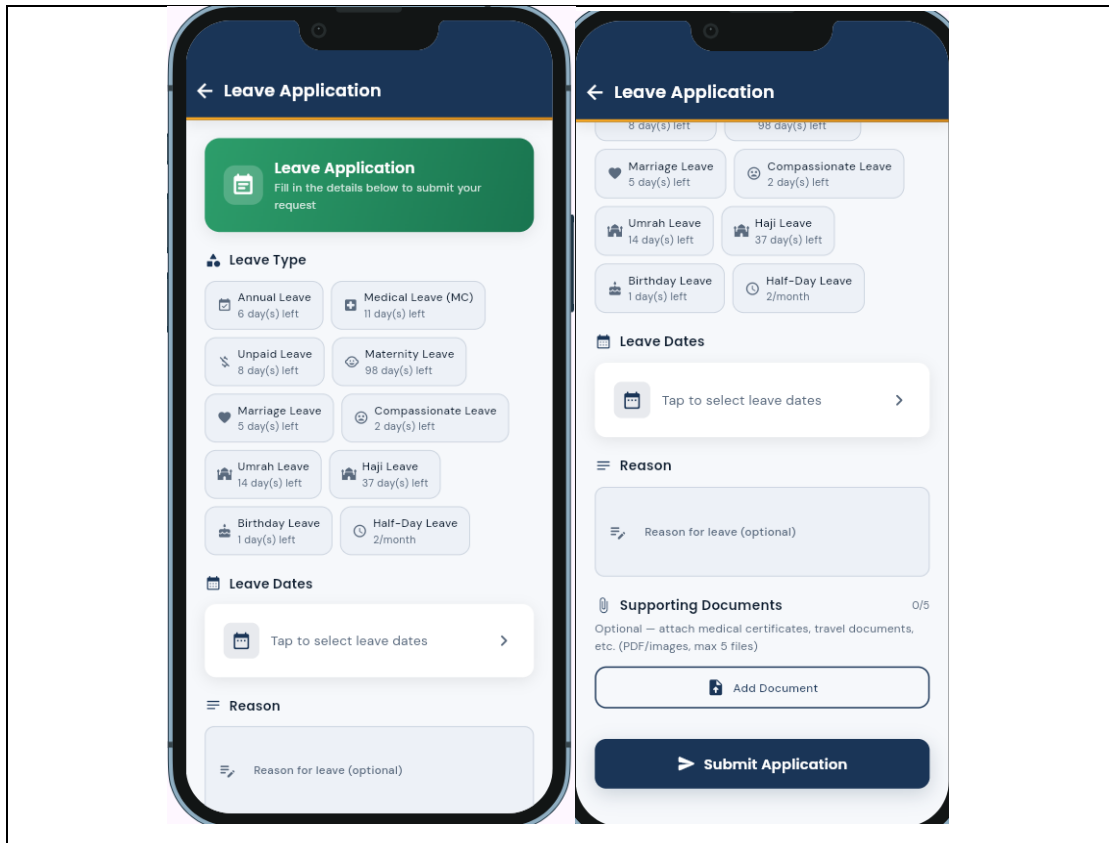


Figure 4.5.5.5.2 Leave Application Form Interface

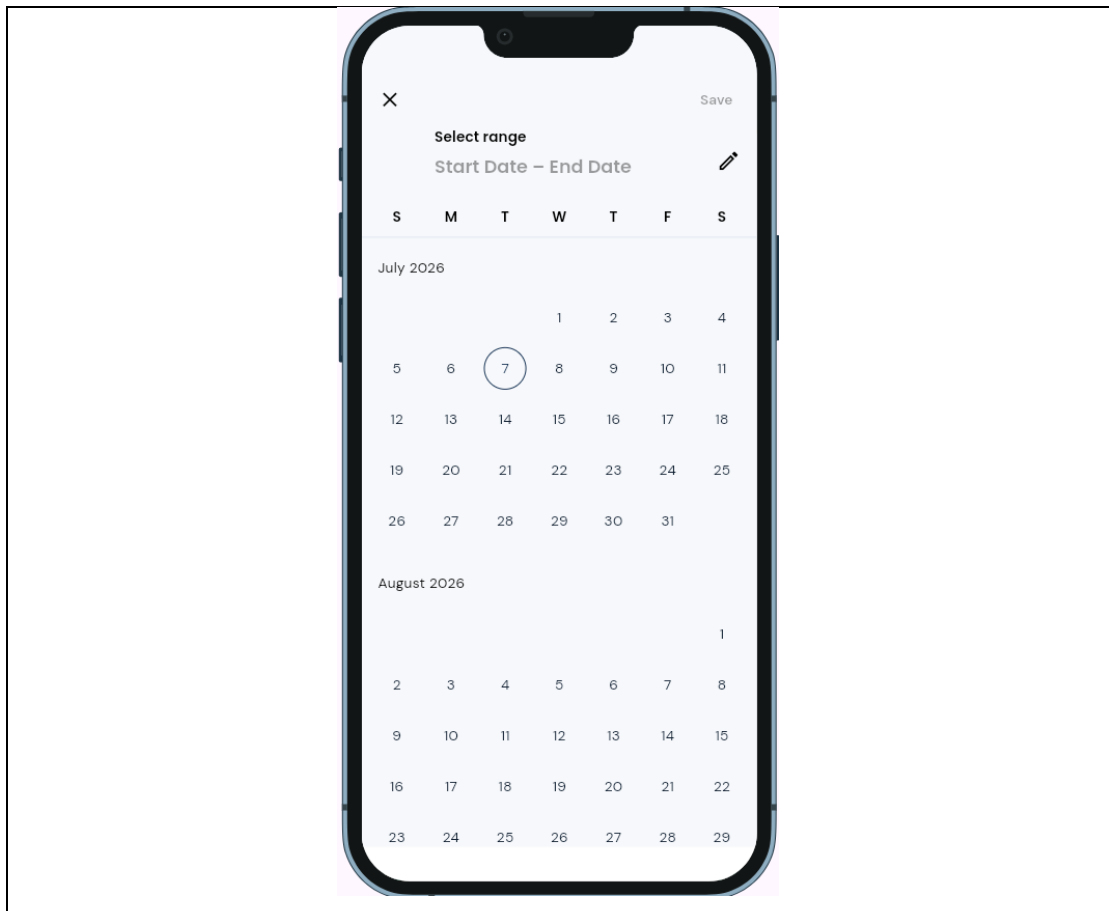


Figure 4.5.5.5.3 Leave Calendar Interface

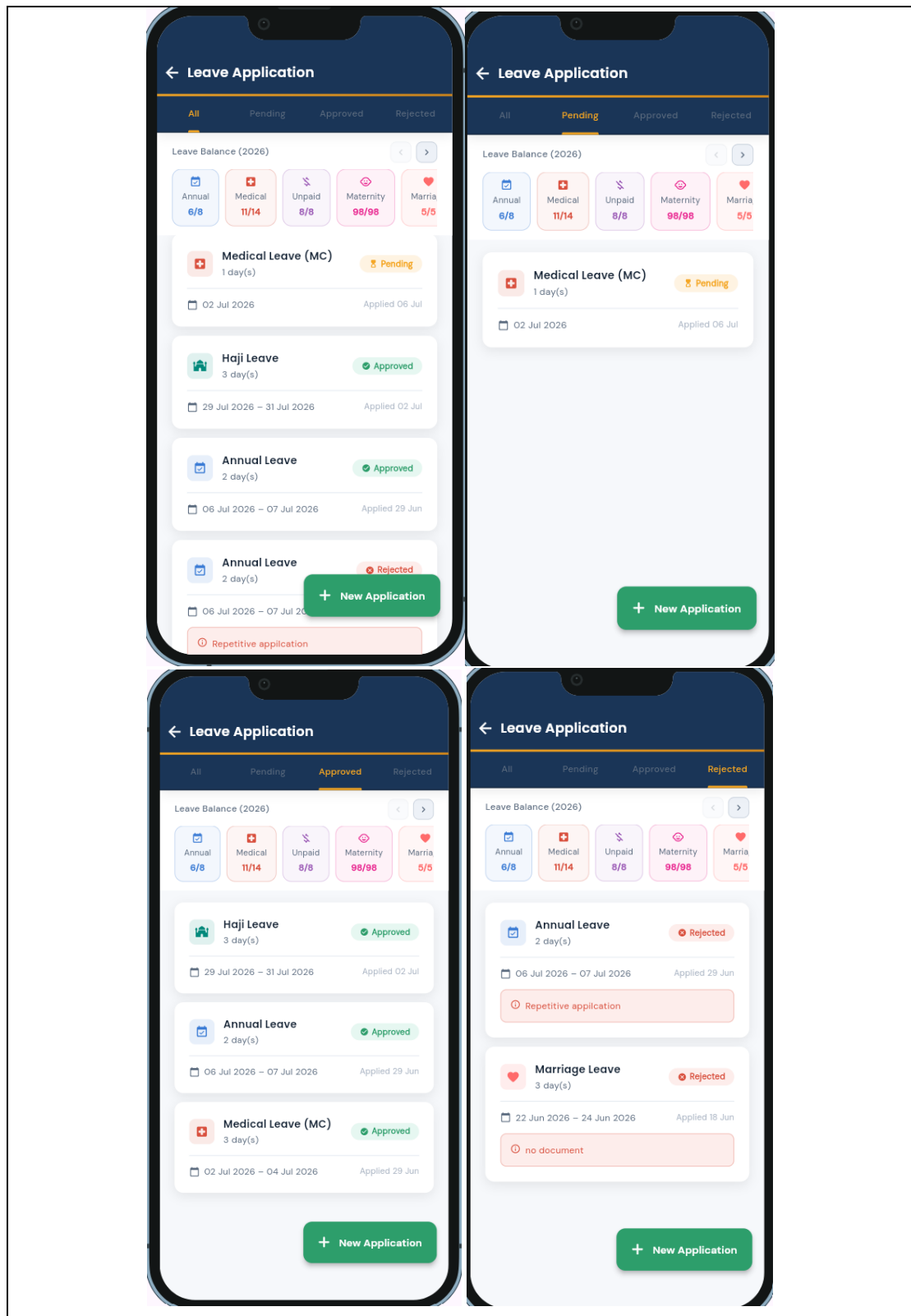


Figure 4.5.5.4 Leave Application Status Interface

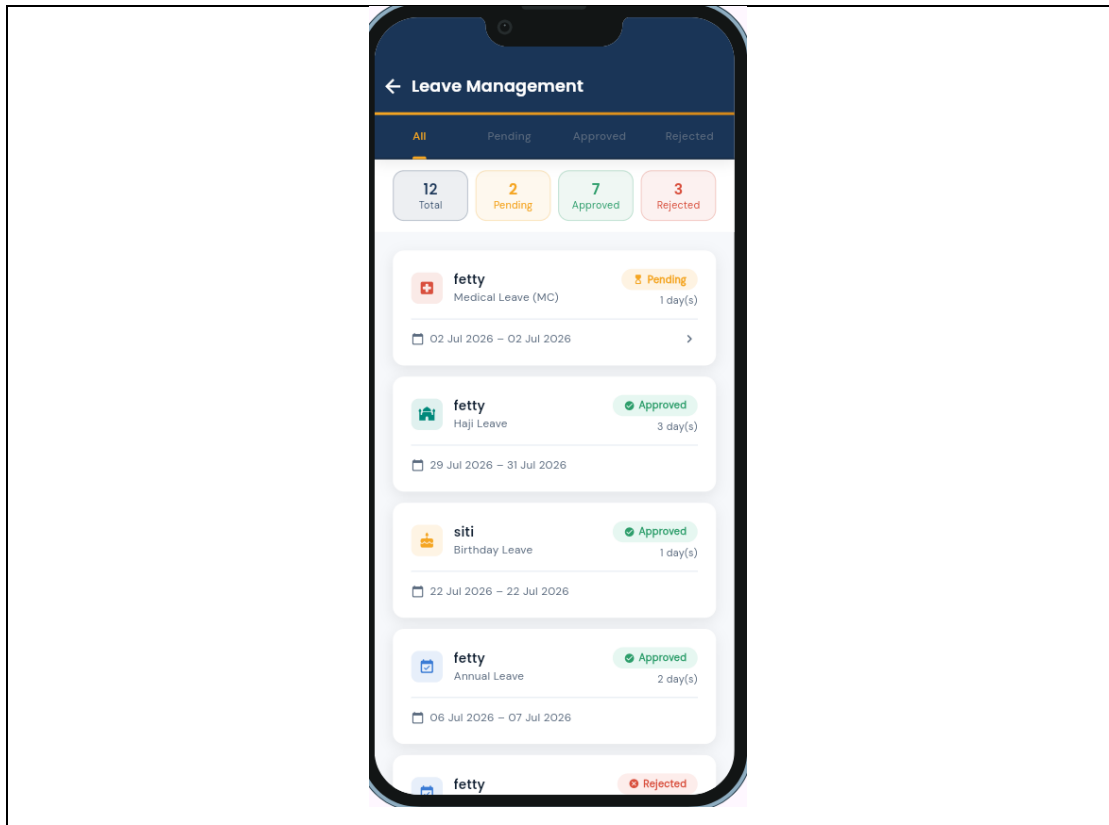


Figure 4.5.5.5 Leave Applications Management Interface

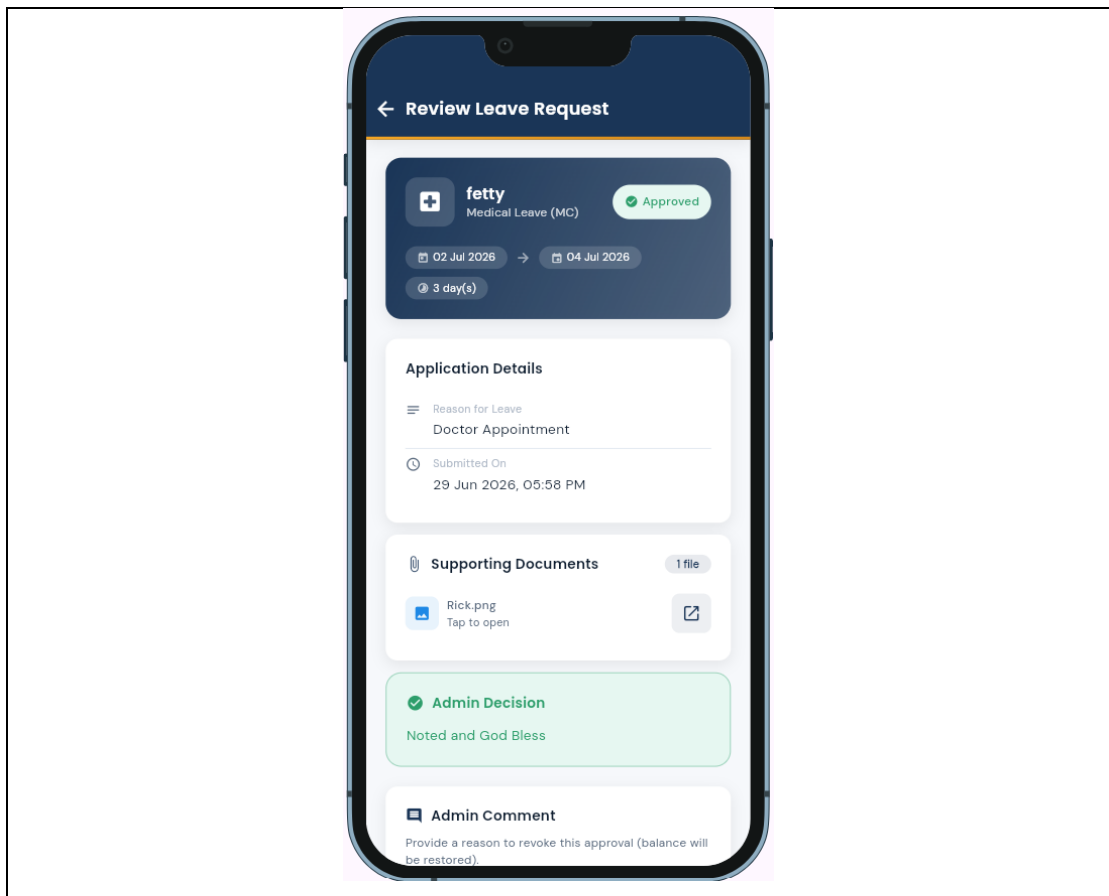


Figure 4.5.5.7 Leave Application Details Interface

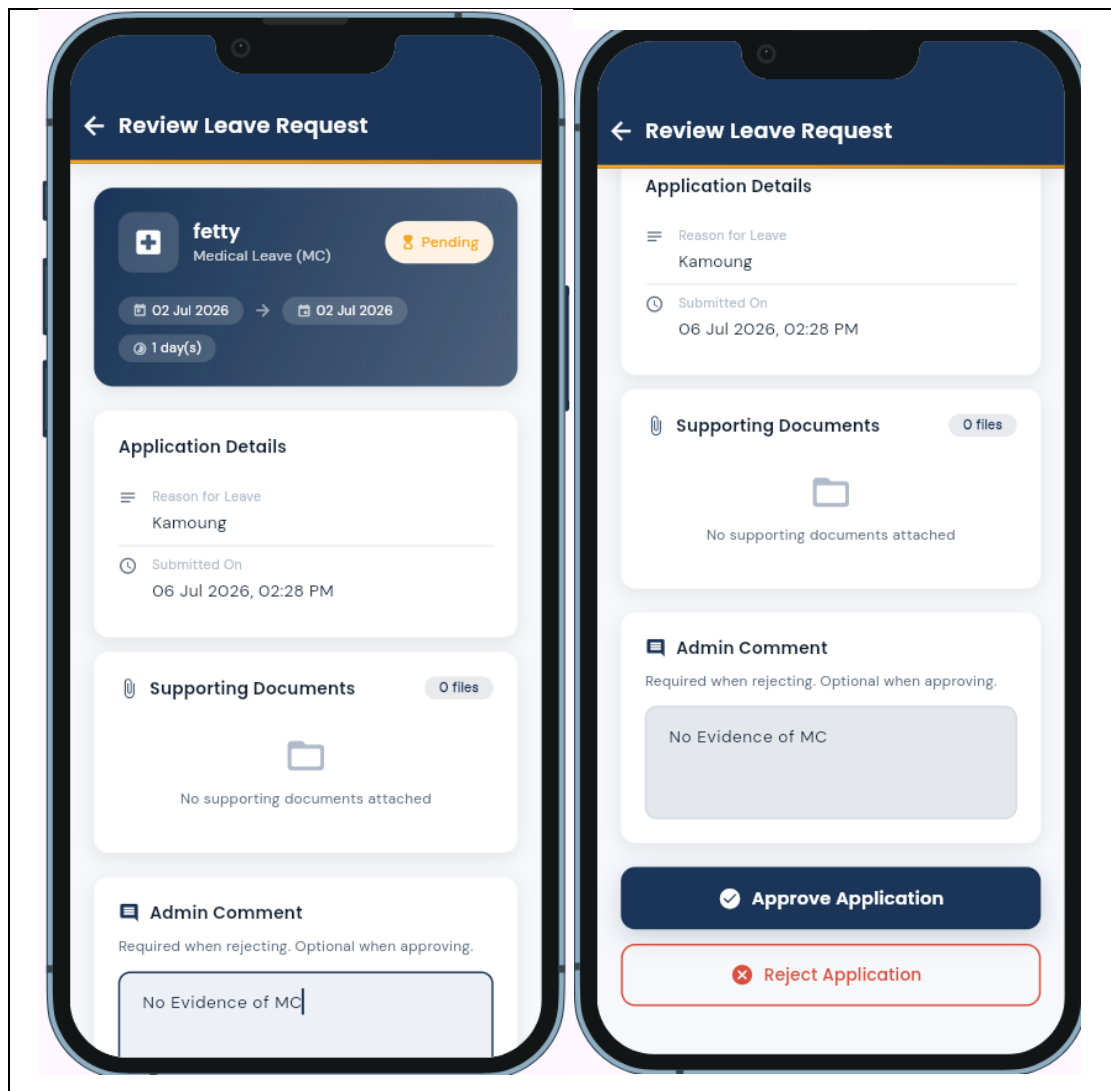


Figure 4.5.5.5.8 Approve / Reject Leave Interface

4.5.5.6 Task & Duty Manager Module

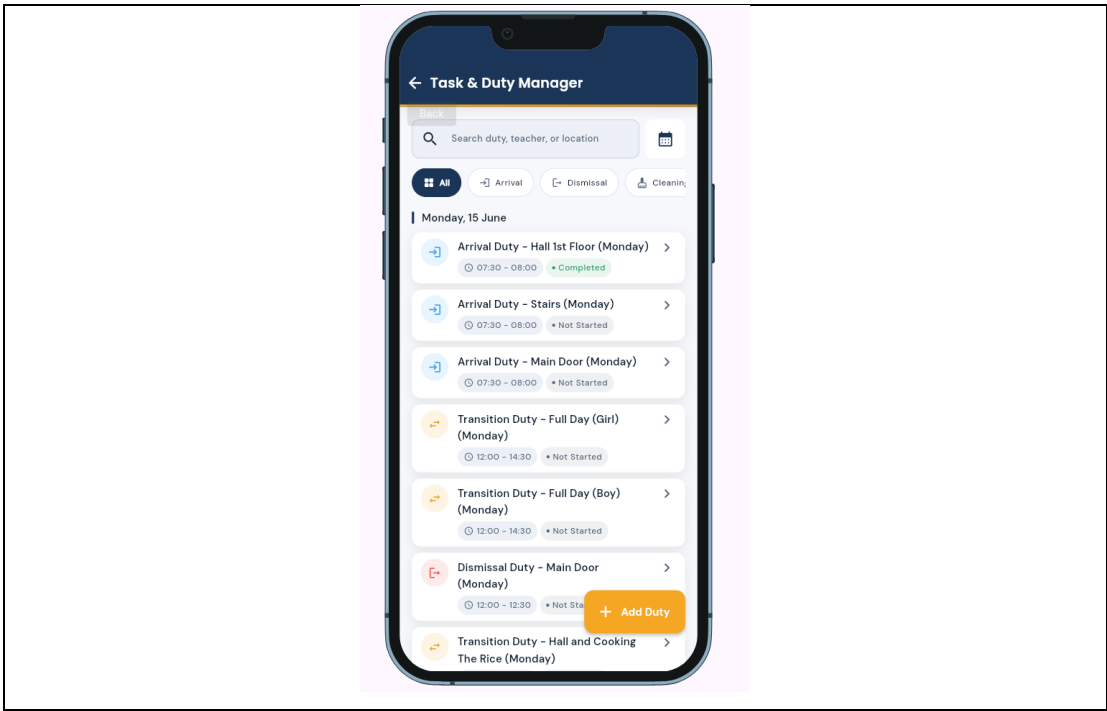


Figure 4.5.5.6.1 Duty List Interface (Admin)

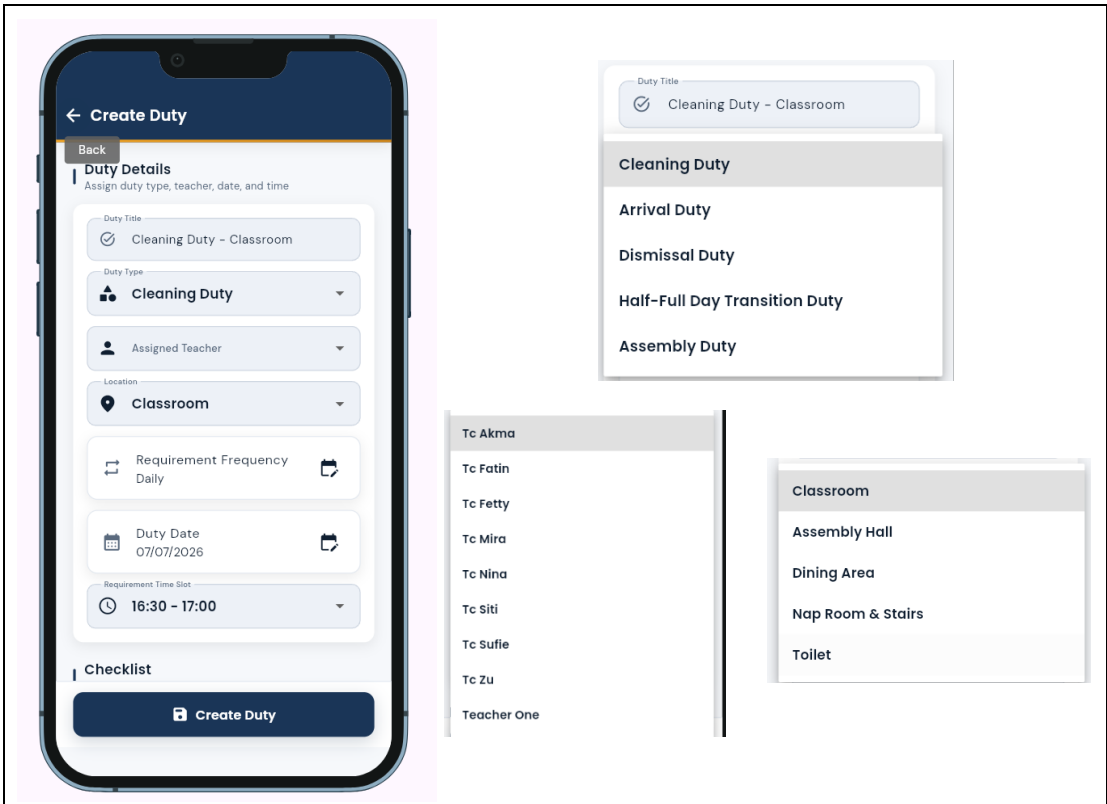


Figure 4.5.5.6.2 Create Duty Assignment Interface (Admin)

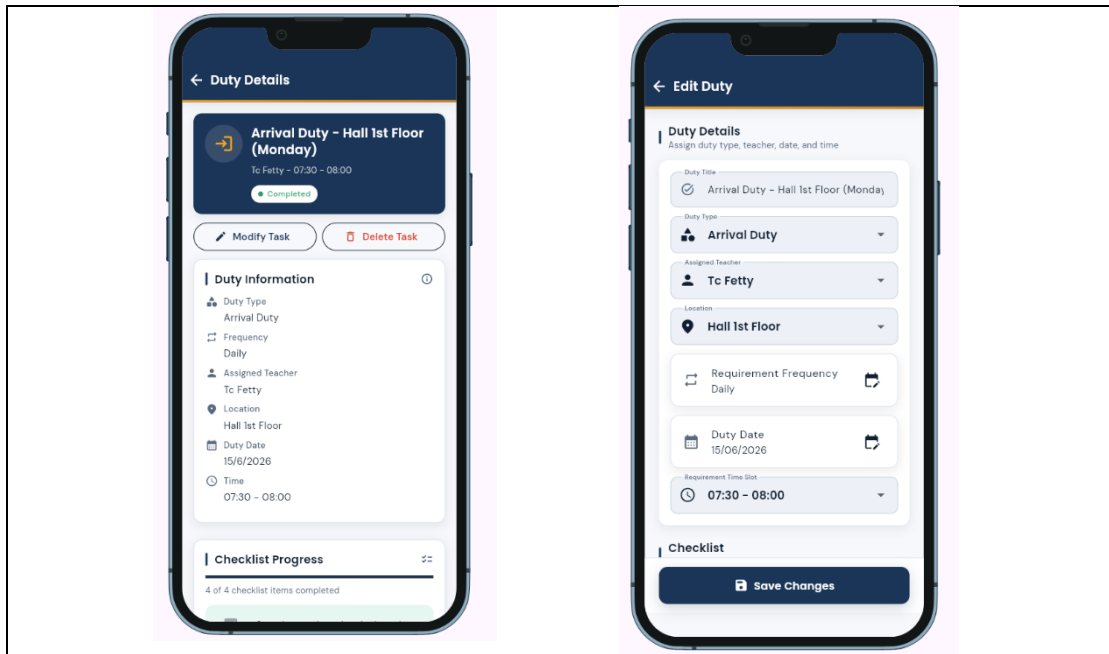


Figure 4.5.5.6.3 Edit Duty Assignment Interface (Admin)

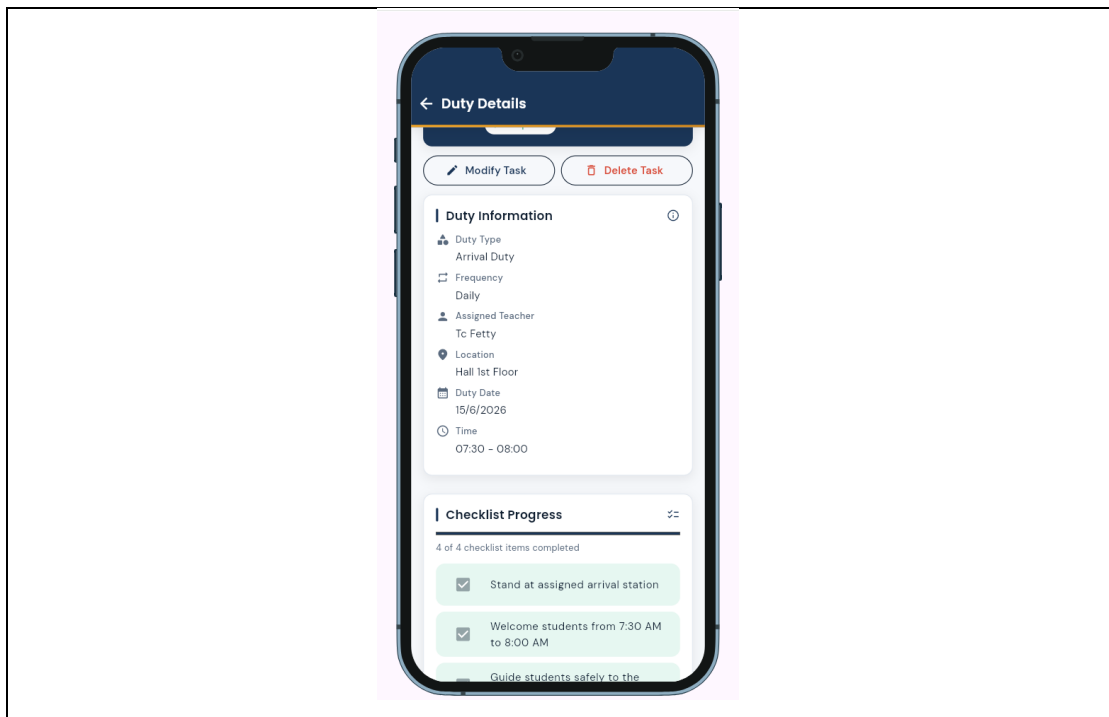


Figure 4.5.5.6.4 Duty Assignment Details Check Interface (Admin)

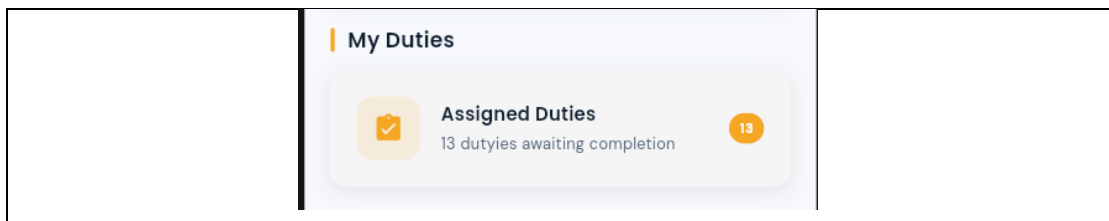


Figure 4.5.5.6.5 Assigned Duties Interface (Teacher Fetty)

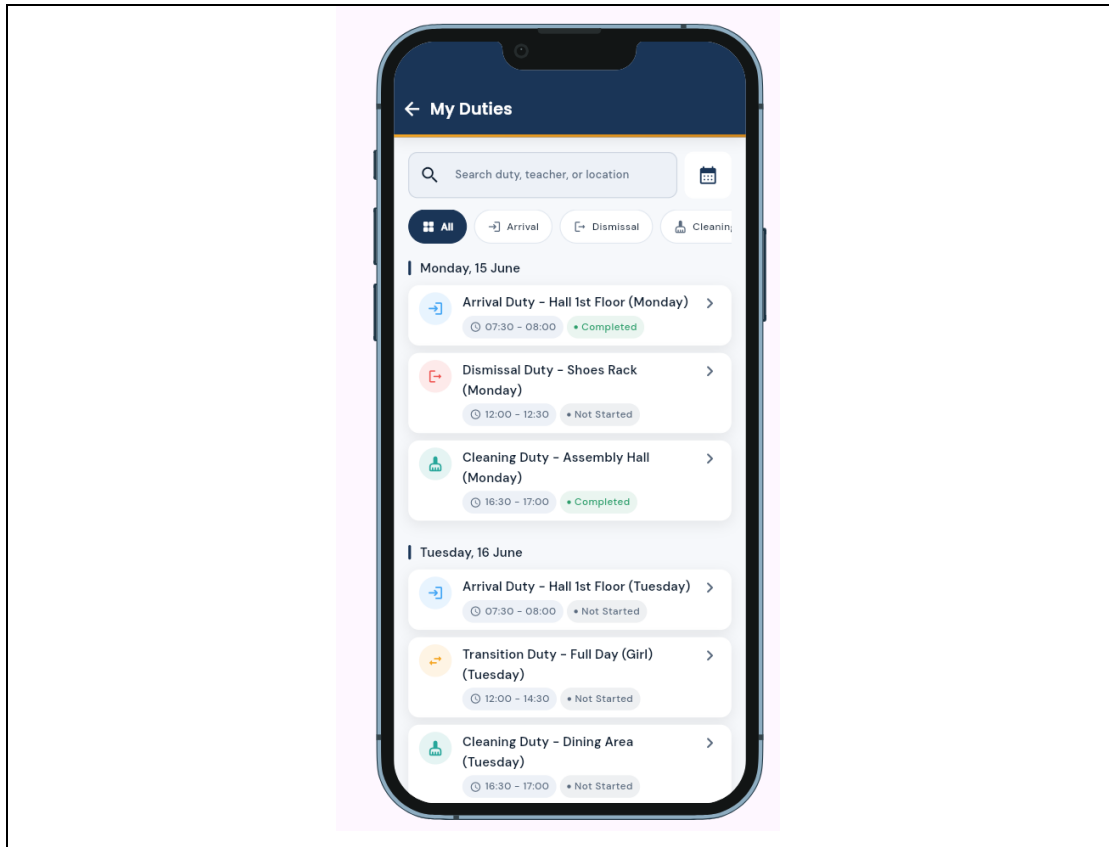


Figure 4.5.5.6.6 Duty Checklist Interface (Teacher Fetty)

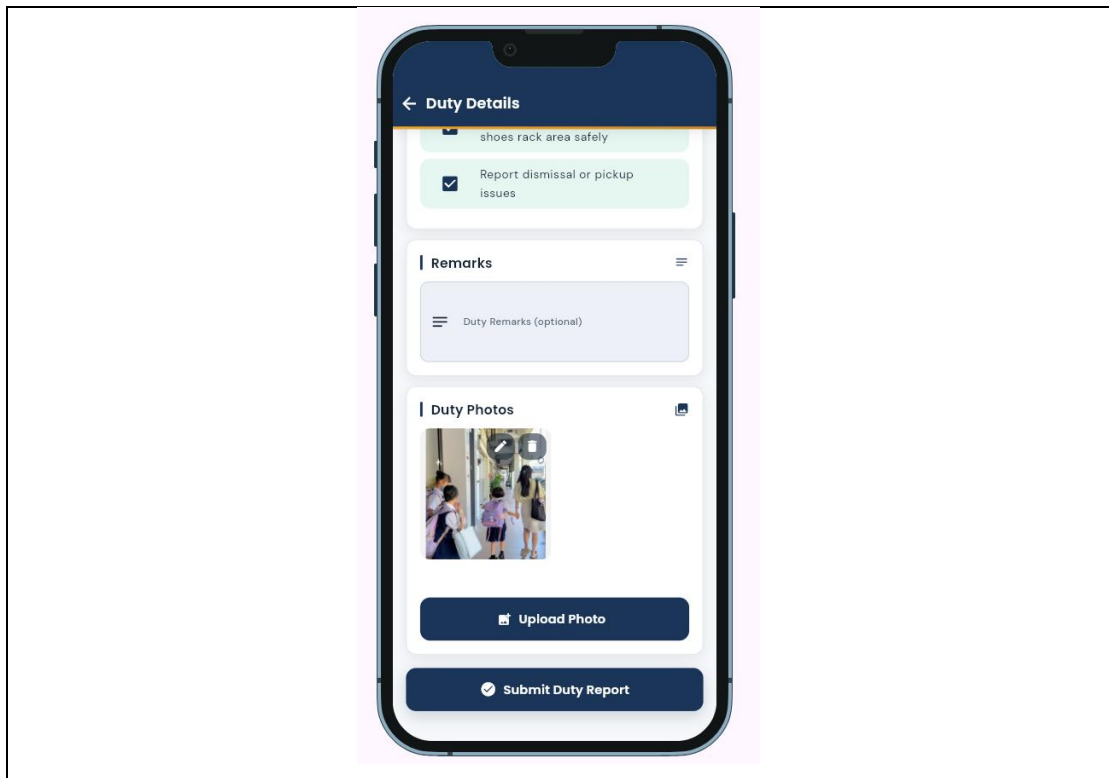


Figure 4.5.5.6.7 Upload Duty Proof Photo Interface (Teacher Fetty)

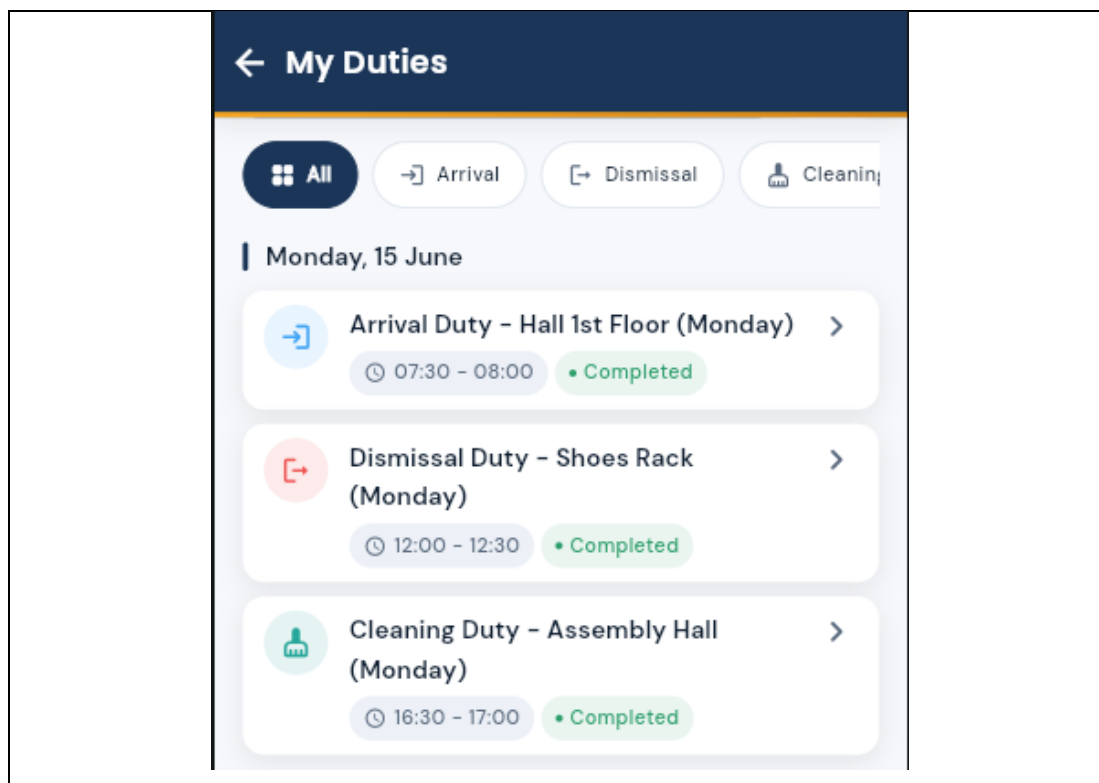


Figure 4.5.5.6.8 Duty Completion Status Interface (Teacher Fetty)

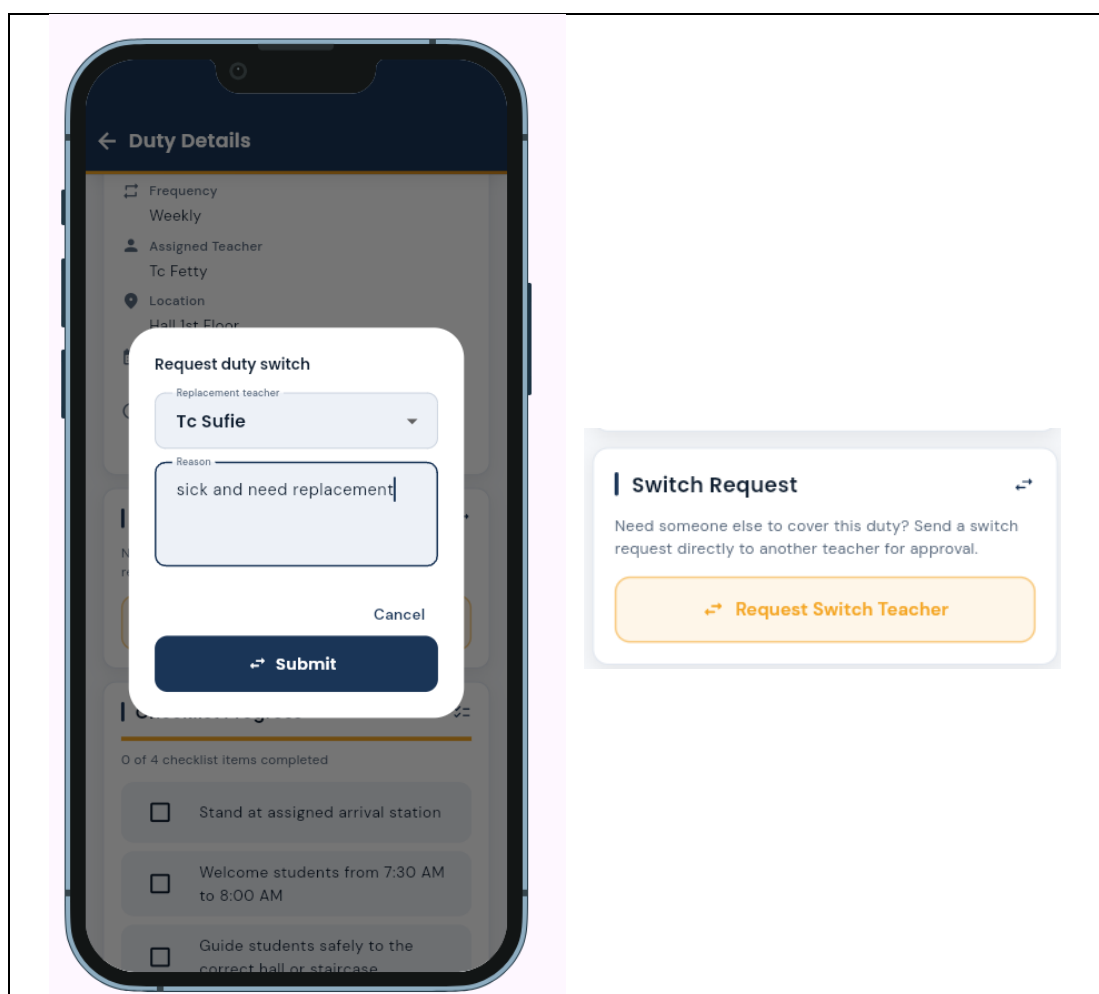


Figure 4.5.5.6.9 Duty Switch Request Form Interface (Teacher Fetty)

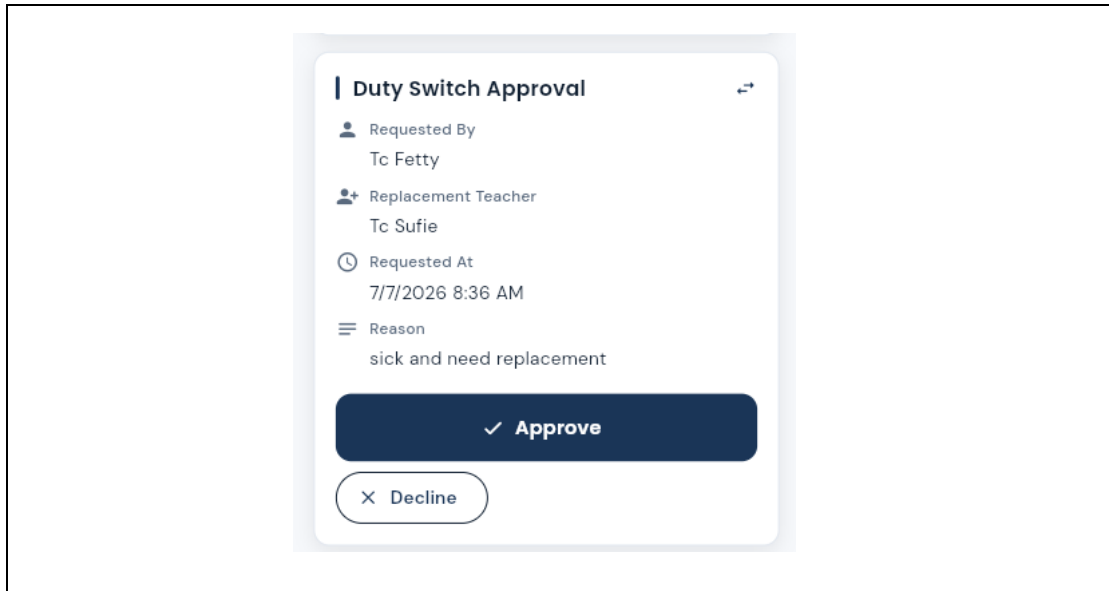


Figure 4.5.5.6.10 Duty Switch Request Review Interface (Teacher Sufie)

4.5.5.7 Teacher Training Tracker Module

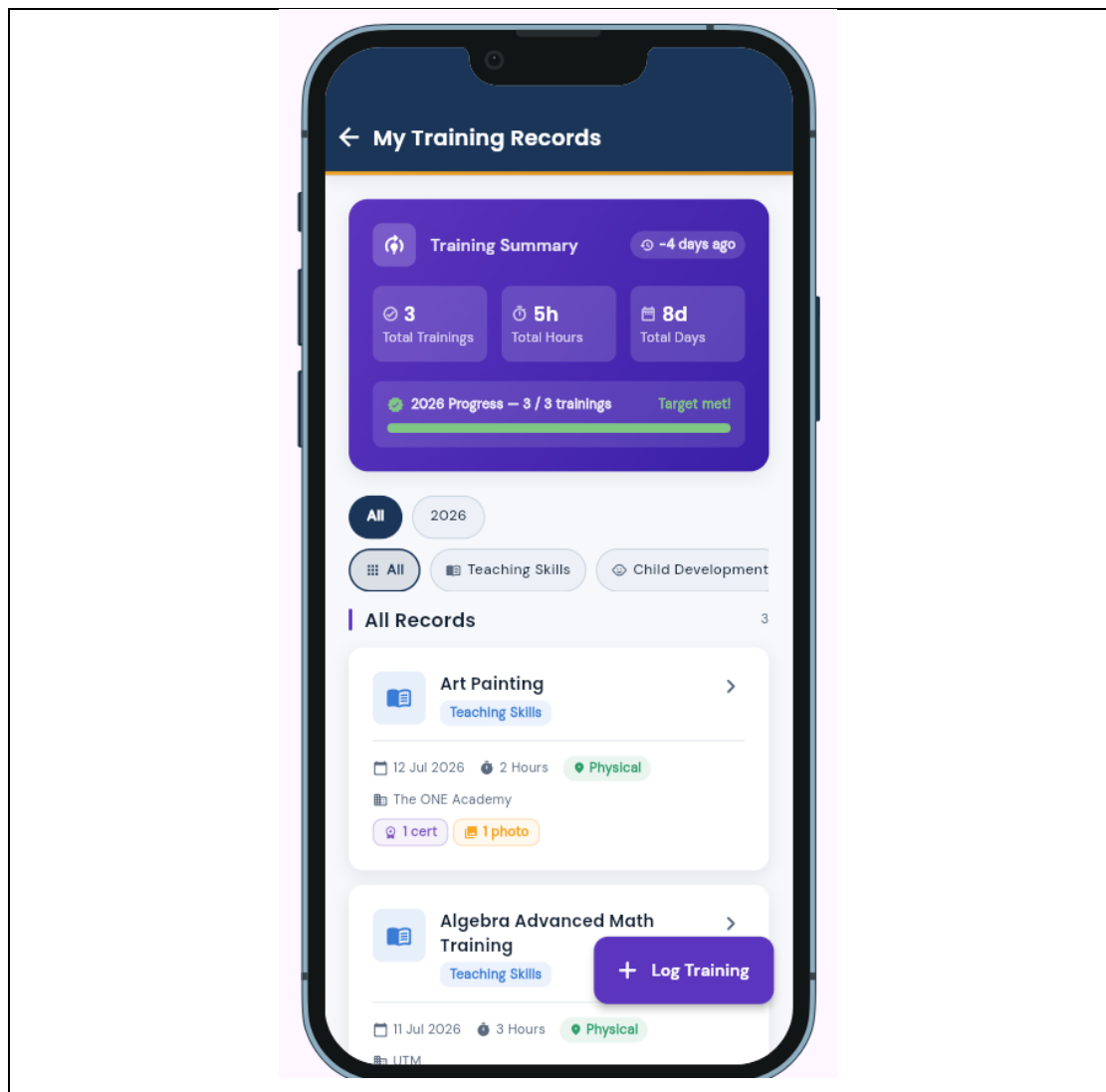


Figure 4.5.5.7.1 My Training Records Interface

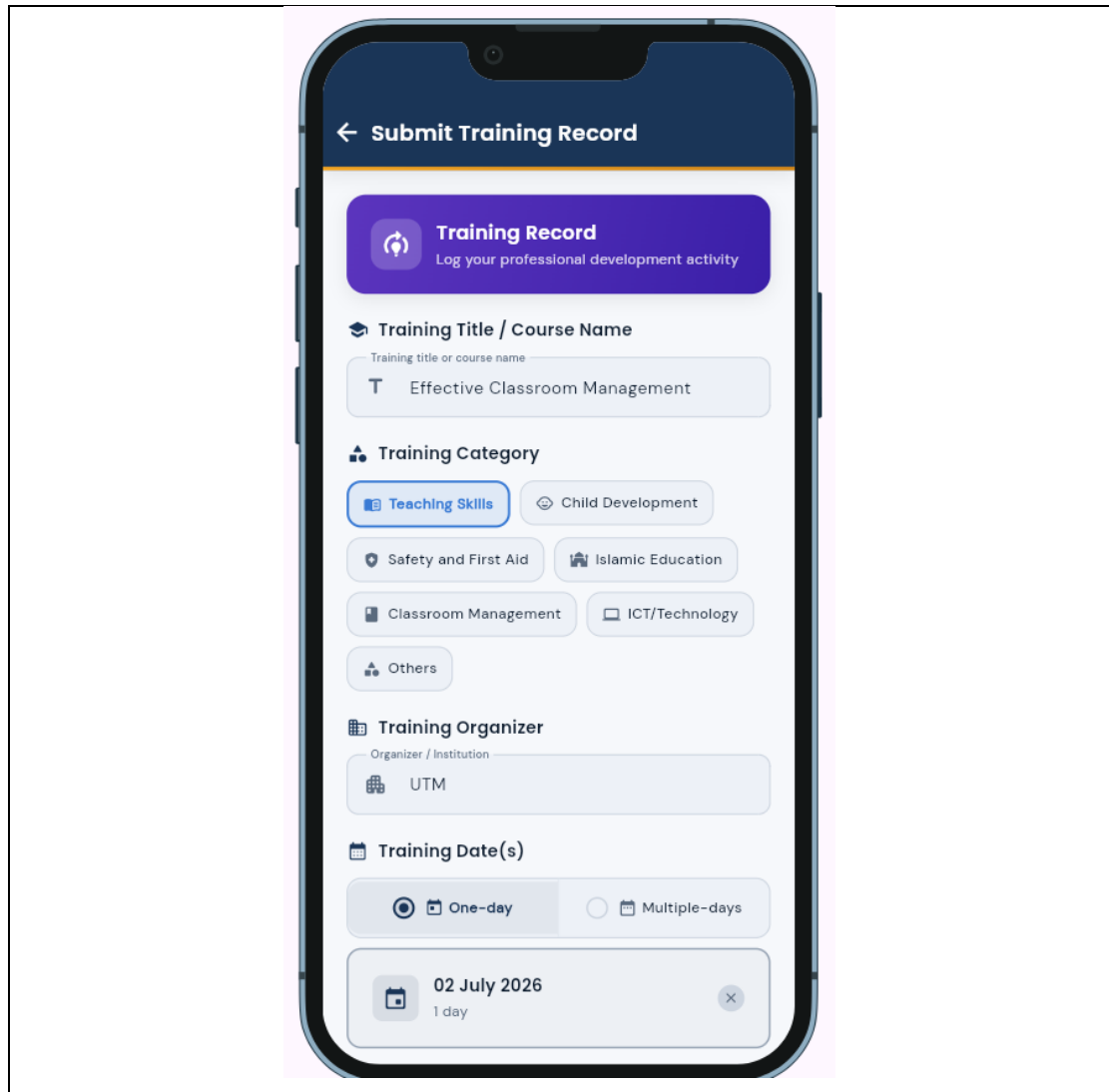


Figure 4.5.5.7.2 Add / Submit Training Record Interface

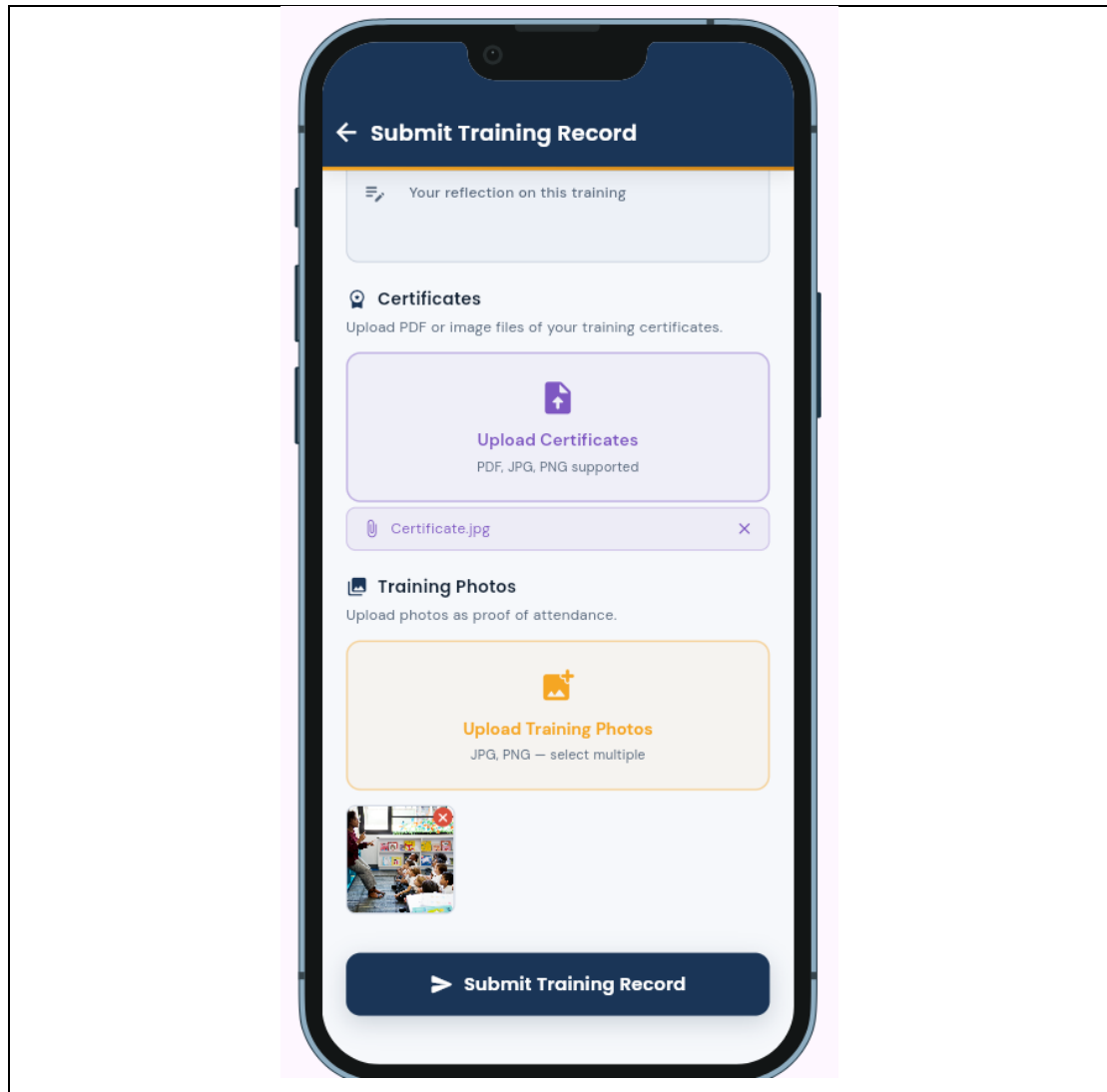


Figure 4.5.5.7.3 Upload Training Evidence Interface

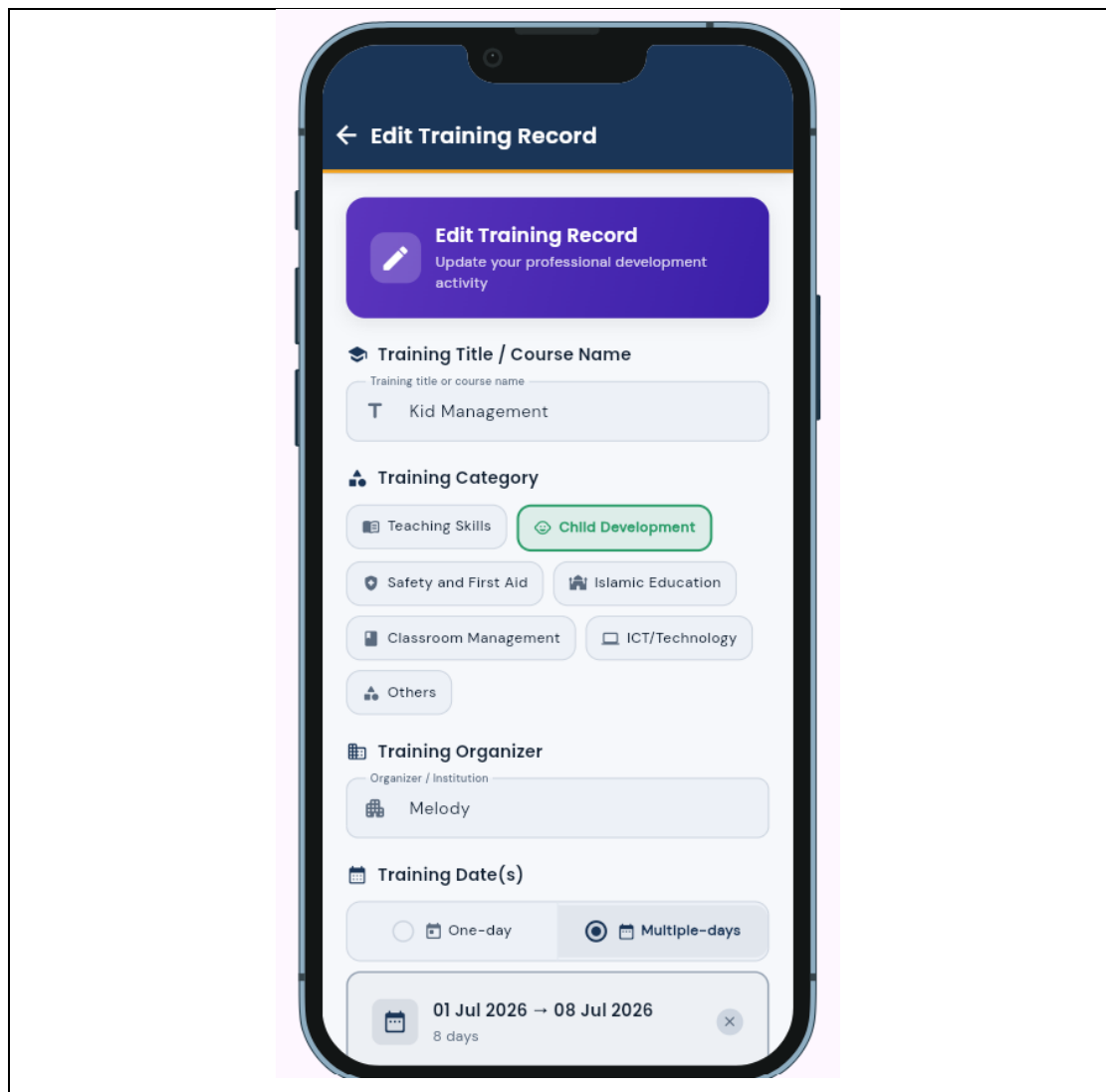


Figure 4.5.5.7.4 Edit Training Record Interface

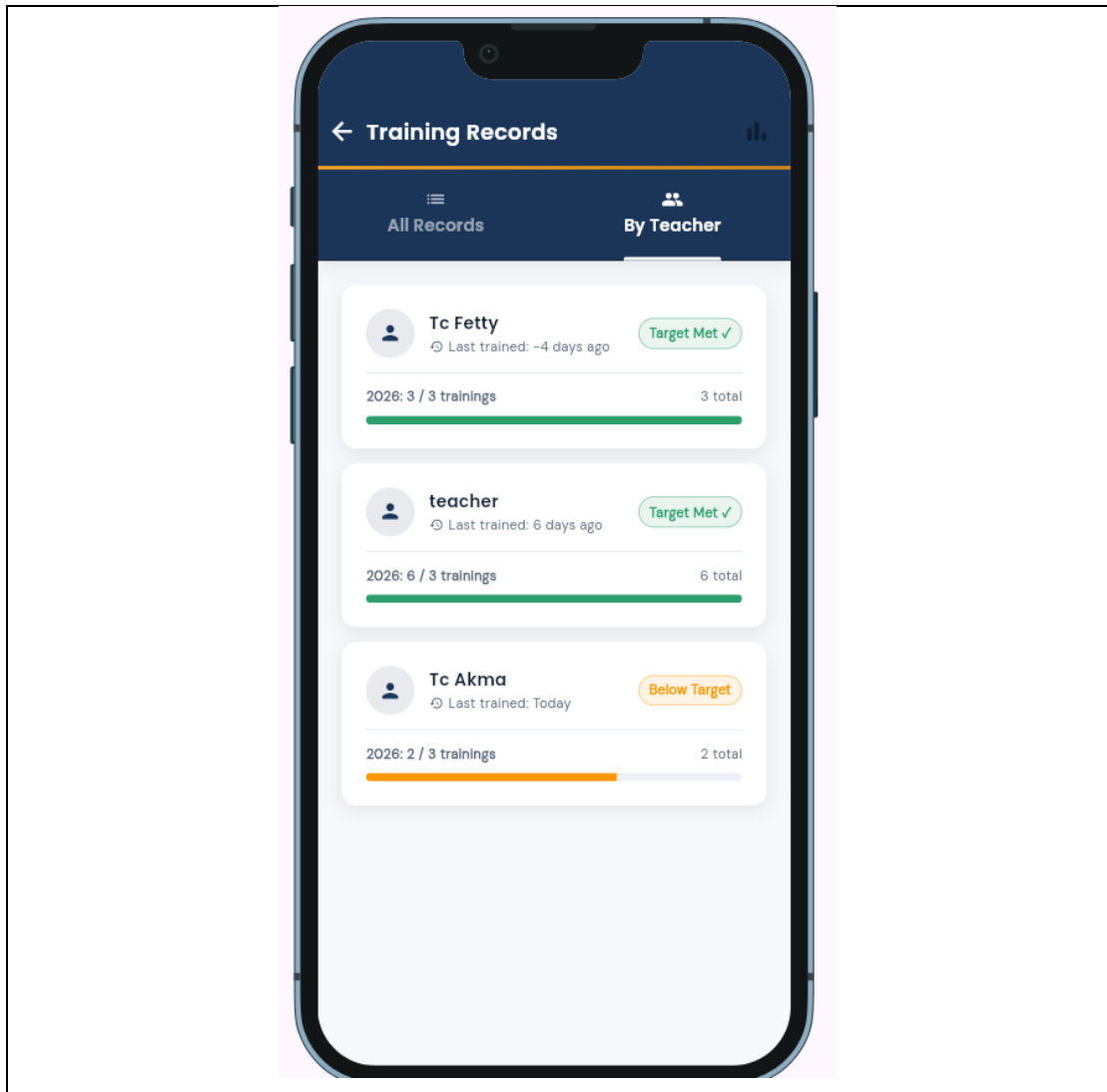


Figure 4.5.5.7.5 Training Records Monitoring Interface (Admin)

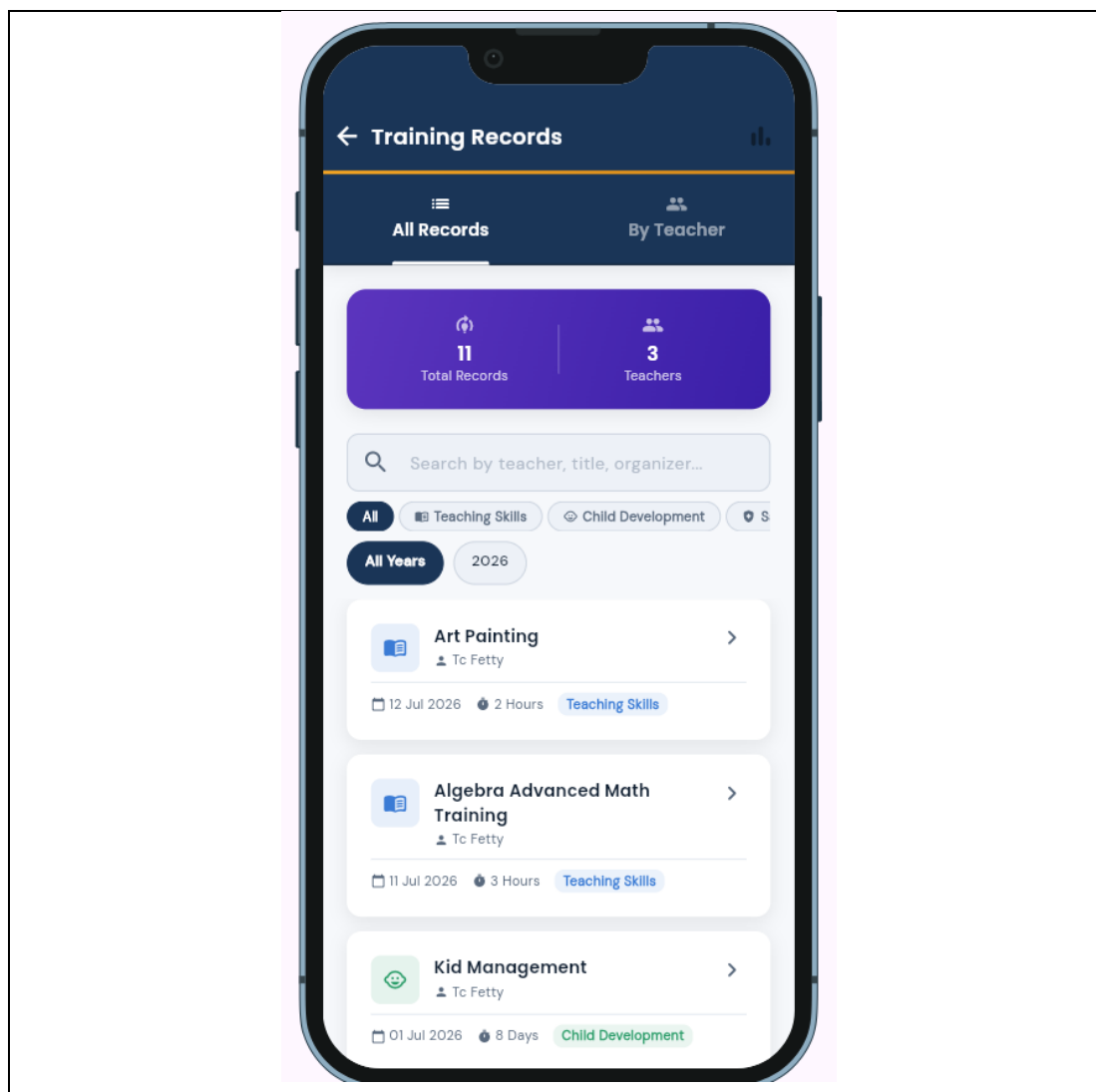


Figure 4.5.5.7.6 Training Summary Interface (Admin)

4.5.5.8 Teacher Performance Tracker Module

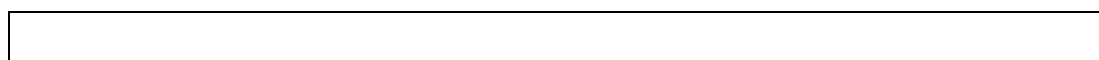


Figure 4.5.5.8.1 Teacher Evaluation List Interface

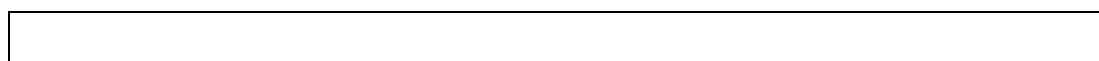


Figure 4.5.5.8.2 Create Performance Evaluation Interface

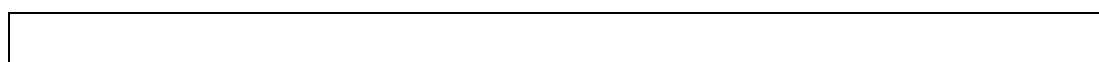


Figure 4.5.5.8.3 KPI Score Entry Interface



Figure 4.5.5.8.4 Save Evaluation Draft Interface



Figure 4.5.5.8.5 Edit Performance Evaluation Interface



Figure 4.5.5.8.6 Release Evaluation Interface



Figure 4.5.5.8.7 Released Feedback Interface

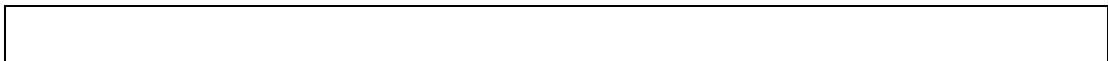


Figure 4.5.5.8.8 KPI Score Summary Interface

4.5.5.9 Teacher Reporting System Module

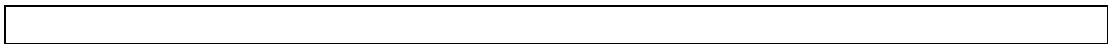


Figure 4.5.5.9.1 Report Submission Form Interface

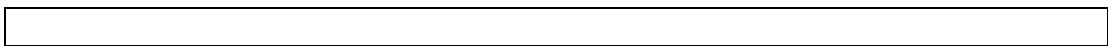


Figure 4.5.5.9.2 Anonymous Report Submission Interface

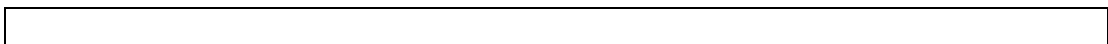


Figure 4.5.5.9.3 Upload Report Evidence Interface

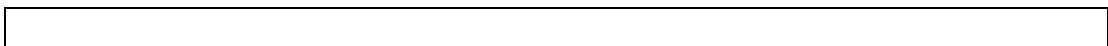


Figure 4.5.5.9.4 My Report History Interface

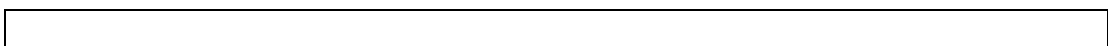


Figure 4.5.5.9.5 All Teacher Reports Interface

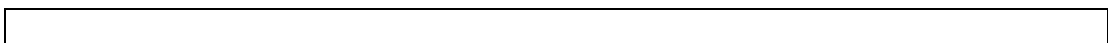


Figure 4.5.5.9.6 Report Details Interface



Figure 4.5.5.9.7 Update Report Status Interface



Figure 4.5.5.9.8 Assign Technician Interface



Figure 4.5.5.9.9 Admin Remarks Interface



Figure 4.5.5.9.10 Assigned Operational Reports Interface

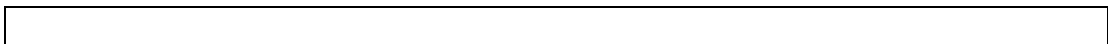


Figure 4.5.5.9.11 Technician Action Update Interface

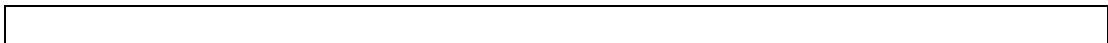


Figure 4.5.5.9.12 Maintenance Remarks Interface

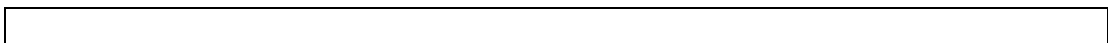


Figure 4.5.5.9.13 Mark Report as Resolved Interface

4.5.6 Chapter Summary

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter explains the implementation and testing of the GENIUSAQILOS Teacher Management System. After completing the analysis and design phase in Chapter 4, the system was developed based on the functional requirements, non-functional requirements, database design, role-based access control, and interface design prepared earlier. The main purpose of this chapter is to describe how the proposed system was implemented into a working mobile application and how the completed functions were tested to ensure that they meet the needs of Tadika Aqil Miqail.

The implementation of GENIUSAQILOS focuses on transforming the manual and fragmented teacher management process into a centralized digital system. The system was developed to support three main roles, namely Principal or Admin, Teacher, and Technician. Each role is given different access rights based on their responsibilities. The principal can manage users, teacher records, leave applications, duty assignments, training submissions, performance evaluation, and reports. Teachers can update their own profiles, apply for leave, view duties, submit training records, and report issues. Technicians can view and handle assigned operational reports.

This chapter also presents the system development process, main coding functions, important system interfaces, and testing activities. The testing section includes black box testing, white box testing, and user testing to evaluate whether the system functions correctly, handles user input properly, and supports the daily administrative workflow of the kindergarten.

5.2 System development

The GENIUSAQILOS Teacher Management System was developed as a mobile-based application to support the daily teacher management operations of Tadika Aqil Miqail. The development process focused on converting the system design from Chapter 4 into a working application that supports role-based access, centralized data storage, and module-

based teacher administration. The system was implemented using Flutter, Dart, Firebase Authentication, Cloud Firestore, Firebase Storage, and GitHub.

5.2.1 Development Environment and Framework

The system was developed using Flutter as the main application framework and Dart as the programming language. Flutter was selected because it supports cross-platform mobile application development and provides reusable widgets for building a consistent user interface. Dart was used to implement the system logic, form validation, navigation control, and communication between the application and Firebase services.

The project structure was organized based on feature modules. Each major function was separated into its own folder, such as authentication, registration, user management, teacher records, leave management, task and duty manager, training tracker, performance tracker, and reporting system. This structure makes the system easier to maintain because each module contains its own models, providers, and screens.

5.2.2 Backend and Database Development

Firebase was used as the backend platform for GENIUSAQILOS. Firebase Authentication was implemented to manage user login and authentication. Users log in using registered email and password credentials, and the system identifies their role after successful authentication.

Cloud Firestore was used as the main database to store system data. The system stores user profiles, teacher records, leave applications, duty assignments, training submissions, performance evaluation records, and teacher reports in different Firestore collections. This allows system data to be accessed and updated in real time.

Firebase Storage was also used to store uploaded files and images. These include teacher documents, training evidence, report evidence, and duty proof photos. By using Firebase Storage, the system can support document and image uploads while keeping file records connected to the related Firestore data.

5.2.3 Authentication and Role-Based Access Control Development

The authentication module was developed to ensure that only registered users can access the system. After login, the system retrieves the user profile from Firestore and checks the user role. Based on the role, users are redirected to the correct dashboard.

Role-based access control was implemented for three main user roles: Principal or Admin, Teacher, and Technician. The admin has access to management functions such as user management, teacher records, leave approval, duty assignment, training review, performance evaluation, and report monitoring. Teachers can access their own profile, leave applications, assigned duties, training submissions, and report functions. Technicians can only access assigned operational reports.

This role-based structure helps prevent unauthorized access. For example, teachers are not allowed to access admin-only pages such as manage user, all teacher records, and administrative approval pages. This improves system security and ensures that each user can only perform tasks related to their responsibility.

5.2.4 Main Module Development

Several main modules were developed to meet the system requirements. The registration module allows new users to register and submit account details. The manage user module allows the admin to view registered users, update user roles, manage account status, and remove user profiles when necessary. Unique account IDs such as A01, T01, and TC01 were also added to make user identification clearer and easier to read.

The teacher records module allows teacher profiles and documents to be stored digitally. Teachers can view and update their own information, while the admin can review teacher records and check document completeness. This helps replace scattered manual records with a more organized digital profile system.

The leave management module allows teachers to apply for leave and allows the admin to approve or reject leave applications. The system also records leave history and supports clearer tracking of leave requests.

The task and duty manager module allows the admin to assign duties to teachers based on duty type, location, date, time slot, and checklist. Teachers can view their assigned duties, update completion progress, submit remarks, and upload proof photos. A duty switch request function was also implemented, allowing a teacher to request another teacher to take over a duty. When the target teacher approves the request, the duty is reassigned automatically.

The teacher training tracker module allows teachers to submit training records and upload evidence. The admin can review training submissions and monitor teacher development. The teacher performance tracker module allows the admin to evaluate teachers based on KPI categories and release feedback to teachers. The reporting module allows teachers to submit operational reports, while the Admin and Technician can follow up according to report type and assignment.

5.2.5 Interface Development

The user interface was developed using a consistent visual template across the system. The interface uses a navy-based header, card layout, clear icons, status indicators, and role-based dashboard design. This helps make the application more professional and easier to use.

Each module was designed with simple navigation and clear action buttons. For example, the admin dashboard displays management functions as menu cards, while the Teacher dashboard focuses on personal tasks such as leave, duty, training, and reporting. The manage user interface was improved with summary cards, role chips, status chips, search functions, and filter controls so that user account information can be managed more clearly. The duty manager interface was also designed to support both Admin and Teacher workflows. Admin users can create and edit duties, while teachers can view their own duty list, filter duties, complete checklist items, upload proof photos, and manage duty switch requests. The interface design aims to reduce confusion and support faster daily operation handling.

5.2.6 Version Control and Team Collaboration

GitHub was used as the version control platform during system development. Each team member worked on assigned modules using separate branches. Completed work was committed and pushed to GitHub, then submitted through pull requests before being merged into the main branch.

This workflow helped the team manage code changes more systematically. It also reduced the risk of overwriting other members' work because each module could be developed and reviewed separately. When new changes were merged into the main branch, team members pulled the latest code to keep their local project updated. This supported better collaboration and allowed the system modules to be integrated progressively.

5.3 Coding of system's main functions

.....

5.4 Essential interfaces that show system's results and achievements

5.4.1 Login Module



Figure 5.4.1.1 Login Interface

Figure 5.4.1.1 illustrates the Login Interface of the GeniusAqilOS system. Users are required to enter their registered email address and password before clicking the Sign In button. The system validates the credentials and grants access to the dashboard upon successful authentication.

5.4.2 Registration Module

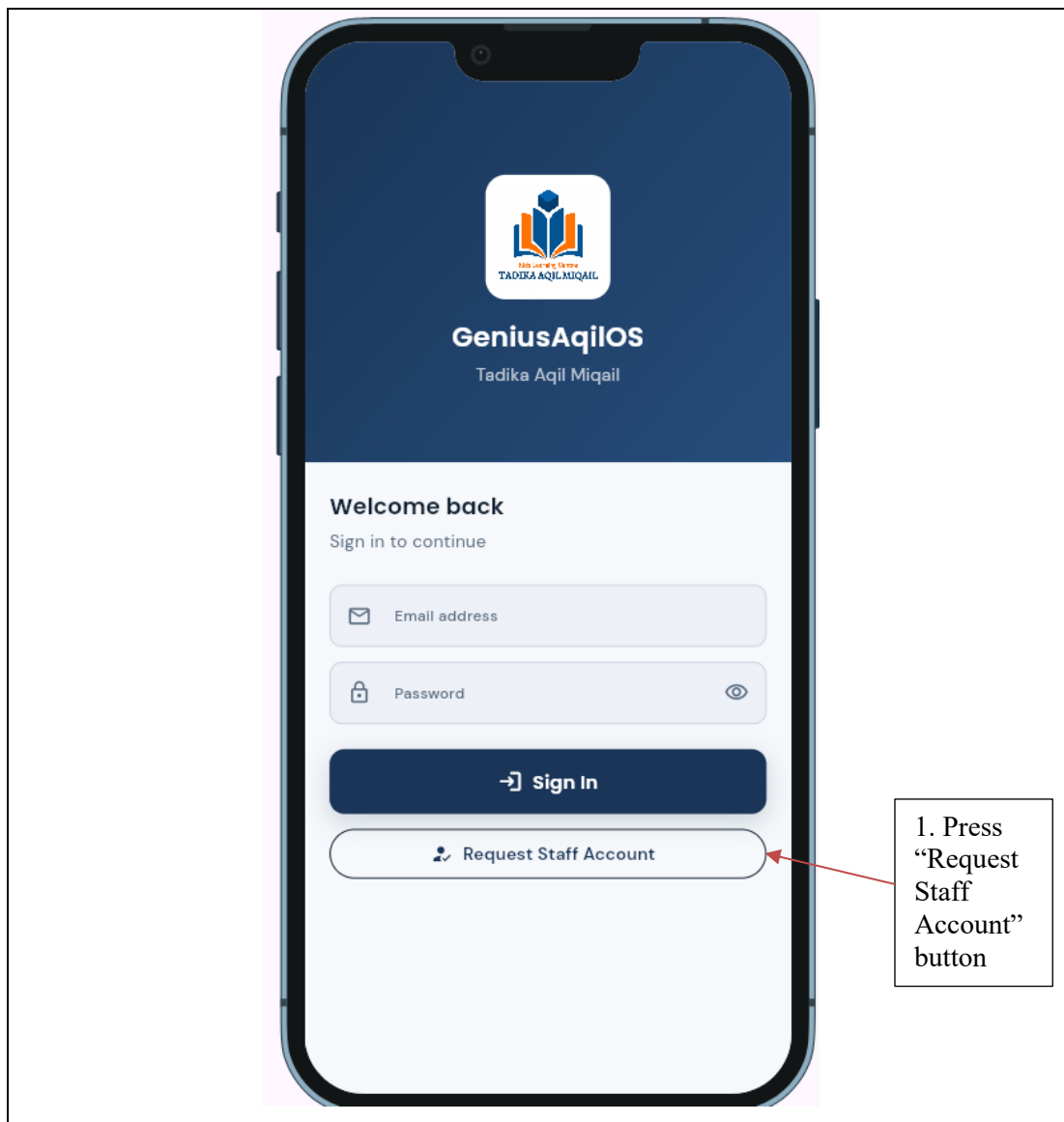


Figure 5.4.2.1 Register Account Interface

Figure 5.4.2.1 illustrates the Registration Module Interface of the GeniusAqilOS system. Users who do not have an existing staff account can click the "Request Staff Account" button to submit a registration request. The system will then direct the user to the staff account registration process, where the required information can be provided for account creation and verification.

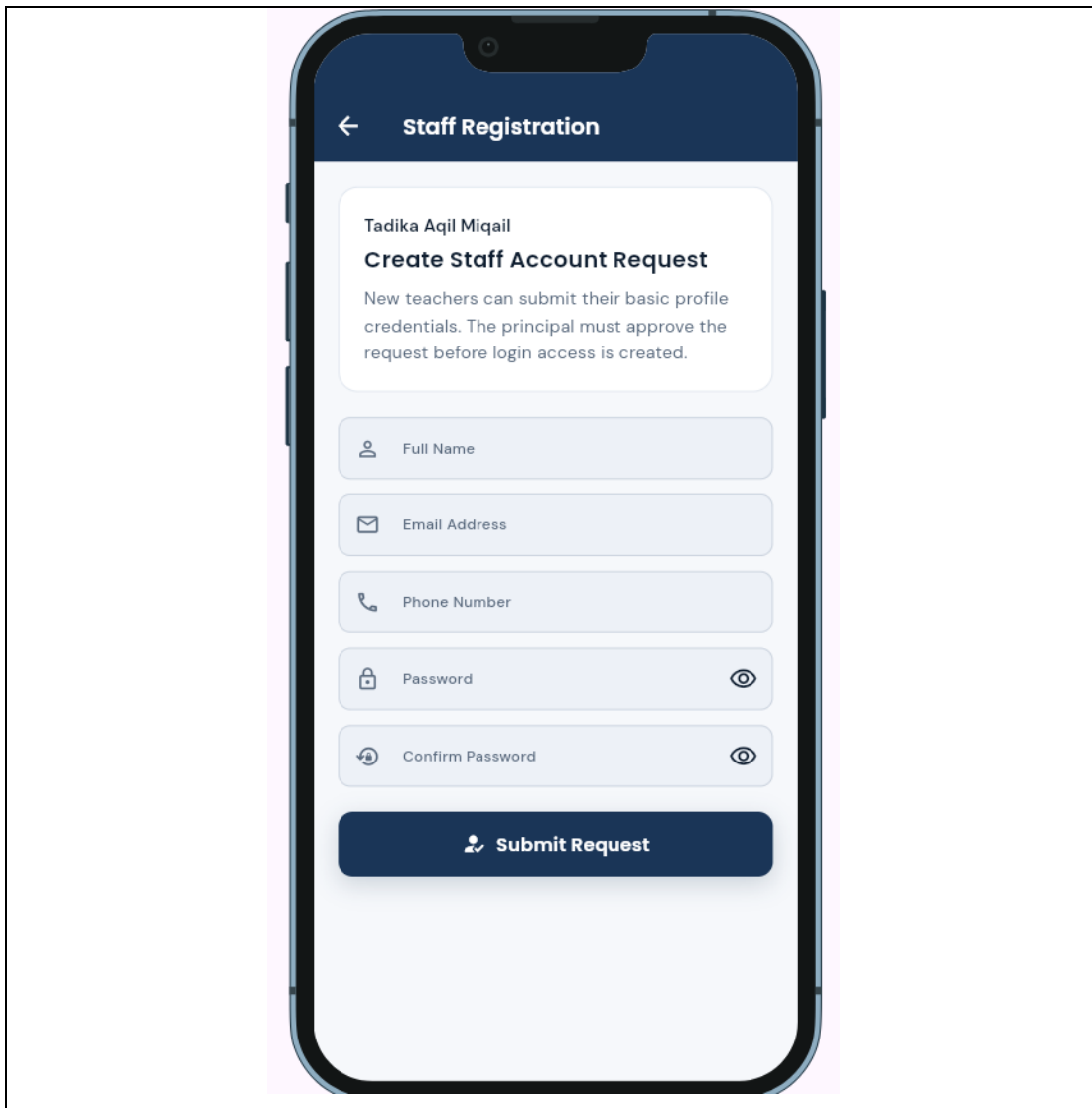


Figure 5.4.2.2 Create Staff Account Interface

5.4.3 Manage User Module

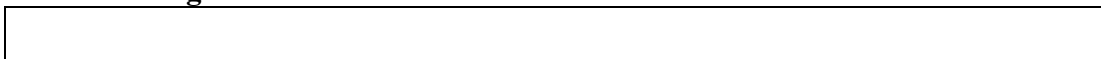


Figure 4.5.5.8.1 Teacher Evaluation List Interface

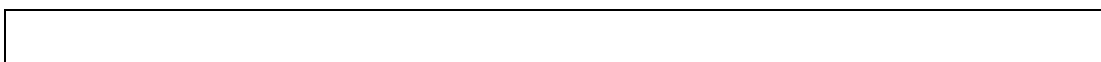


Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.4.4 Teacher Records Module

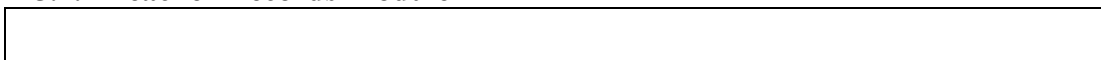


Figure 4.5.5.8.1 Teacher Evaluation List Interface

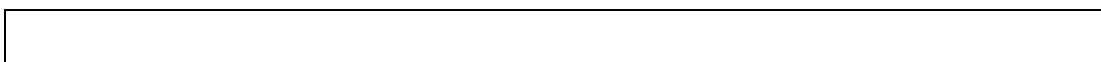


Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.4.5 Leave Management Module



Figure 4.5.5.8.1 Teacher Evaluation List Interface



Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.4.6 Task & Duty Manager Module



Figure 4.5.5.8.1 Teacher Evaluation List Interface

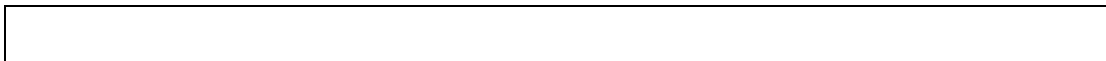


Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.4.7 Teacher Training Tracker Module



Figure 4.5.5.8.1 Teacher Evaluation List Interface



Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.4.8 Teacher Performance Tracker Module



Figure 4.5.5.8.1 Teacher Evaluation List Interface

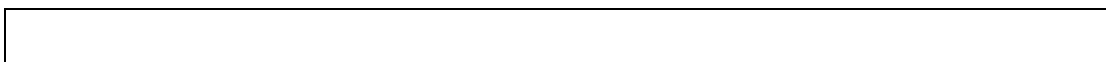


Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.4.9 Teacher Reporting System Module

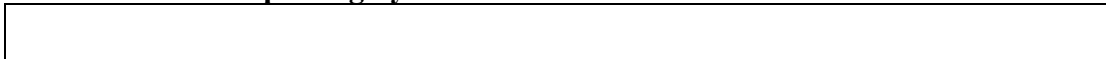


Figure 4.5.5.8.1 Teacher Evaluation List Interface

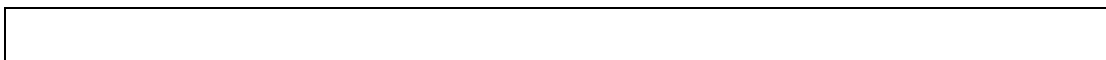


Figure 4.5.5.8.2 Create Performance Evaluation Interface

5.5 Testing

5.5.1 Black box testing

.....

5.5.2 White box testing

.....

5.5.3 User testing

.....

5.6 Chapter Summary

.....

This chapter discusses the process and writing of a very important chapter in the SCSP 4234 Report that is *Implementation and Testing* (Chapter 5). The discussion of implementation and testing is vital as it relate with the importance and achievement of the project. It is expected that student will be able to write the chapter effectively with the guide provided in this chapter.

Implementation, Testing

The implementation chapter will successfully explain all the results achieved through project coding, experiments and testing. The chapter may be expanded into more than one chapter according to the project. The results of the project implementation are discussed in detail here. Documents and proof of correctness of the project must be attached. Testing types and techniques, as well as testing results, must be stated. Table 5.1 and 5.2 shows suggested outline of Chapter 5 of the FYPi report

Table 5.1:Example of Chapter 5 of the FYPi report

System Development Based Project

5.1 Introduction
5.2 System development
5.3 Coding of system's main functions
5.4 Essential interfaces that show system's results and achievements
5.5 Testing 5.4.1 Black box testing • System flow, input/output, error messages 5.4.2 White box testing 5.4.3 User testing
5.6 Chapter summary

Table 5.2: Example of Chapter 5 of the FYPi report

Research Based Project					
Chapter	5:	Results,	Analysis	and	Discussion
5.1 Introduction					
5.2 Research results and analysis					
					• Discussion • Tables, Graph etc.
5.3 Chapter summary					

Additional guideline for system development

Software testing is one of important phases in a system development process. There are two common testing methods, which are White Box Testing and Black Box Testing. Next section briefly describes these two software testing methods, respectively.

White box testing

White-box testing is a method of testing application software. It tests the internal structures or the workings of an application, as opposed to its functionality. White-box testing examines the internal perspective or logic of the system. This method requires programming skills to design test cases. Different input data are chosen to exercise all possible paths through the code and to determine the appropriate outputs.

White-box test design techniques include:

- i. Control Flow testing / Branch Testing
- ii. Data flow testing (Data Tracking)
- iii. Path testing (Logic Testing)

White-box testing can be applied at the module, integration and system levels. It can test paths within a module, paths between modules during integration, and between subsystems during a system level test. Though this method of test design can uncover many errors or problems, it might not detect unimplemented parts of the specification or missing requirements.

Black box testing

Black-box testing is a method of software testing that tests the functionality of an application. Specific knowledge of the application's code/internal structure and programming knowledge in general is not required. Test cases are built around specifications and requirements that is what the application should suppose to do. It uses external descriptions of the software, including specifications, requirements, and designs to derive test cases. The test designer selects valid and invalid inputs and determines the correct output.

Additional guideline for research

Following the research method described in the Methodology chapter, discussion should include the analysis and discussion of the results. It is encouraged that discussions are accompanied with graphs, charts or tables for enhanced clarification. Figure 7.1, figure 7.2, figure 7.3 and figure 7.4 are some examples of result presentations and discussion of the result.

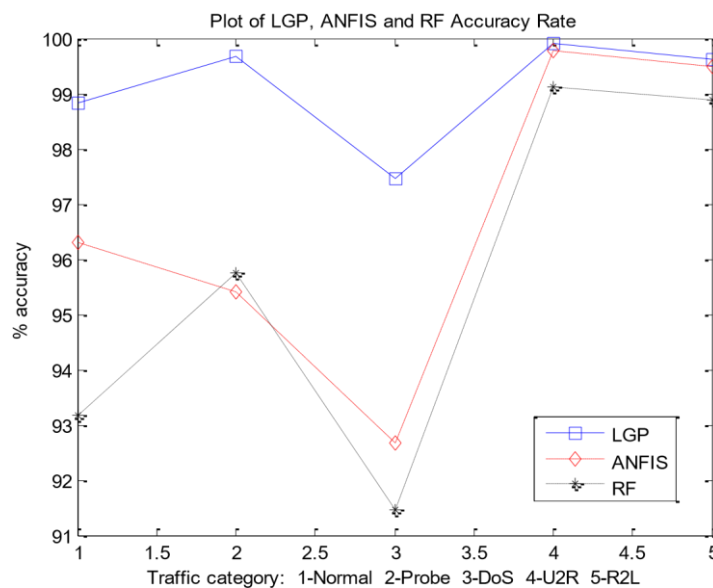


Figure 7.1: Example of a line graph

Performance Measure	Baseline(%)	RRA(%)	EA-IDS(%)
Overall Accuracy	96.764	98.845	99.144
FAR	16.082	5.591	2.453
Hit Rate	99.944	99.948	99.550

Figure 7.2: Example of a table

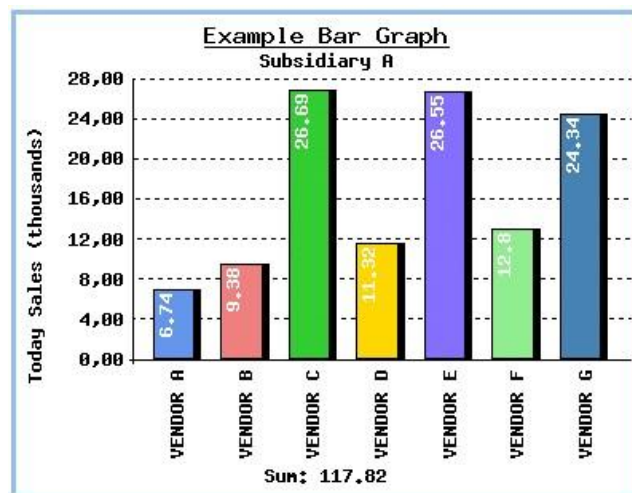


Figure 7.3: Example of a bar chart

Performance Measure	Baseline(%)	RRA(%)	EA-IDS(%)
Overall Accuracy	96.764	98.845	99.144
FAR	16.082	5.591	2.453
Hit Rate	99.944	99.948	99.550
Generally, the overall detection accuracy has improved from 96.764% (Baseline) to 99.144% (EA-IDS) and FAR was greatly reduced from 16.082% (Baseline) to 2.453. The hit rate remains stable which is almost reaching 100% correctly ability to segregate between attacks and normal.			

Figure 7.4: Example of a table with its discussion.

CHAPTER 6

CONCLUSION

6.1 Introduction

.....

6.2 System Contribution/Achievement

.....

6.3 System Constraint

.....

6.4 Future Suggestion

.....

6.5 Chapter Summary

This chapter focuses on the concluding in the FYPi report. Conclusion in SCSP 4233 and SCSP 4234 is handled differently. This guideline presents an effective way of concluding the FYPi report.

FYPi Conclusion Chapter (Chapter 6) summarizes the whole project undertaken by a student. It discusses results and achievements of the project as well as suggestions and future works. Project strength, weaknesses and suggestions to improve must be stated clearly and concisely.

Table 8.1: Example outline of Chapter 6 of SCSP 4233 & SCSP 4234 report

System Development Based Project	
SCSP 4233 Chapter 5: Conclusion 5.1 Introduction ● Restate the project significance and objectives. 5.2 Achievements ● Briefly explain findings based on literature review ● Briefly explain any objectives that has been concluded or partially concluded 5.3 Suggested plan for project implementation/execution (SCSP 4234)	SCSP 4234 Chapter 6: Conclusion 6.1 Introduction 6.2 System Contribution/Achievement 6.3 System Constraint 6.4 Future Suggestion 6.5 Summary

Table 8.1: Example outline of Chapter 6 of SCSP 4233 & SCSP 4234 report

Research Based Project	
-------------------------------	--

SCSP 4233 Chapter 5: Conclusion 5.1 Introduction - Restate the project significance and objectives. 5.2 Achievement/milestone achieved - Briefly explain findings based on literature review - Briefly explain any objectives that has been concluded or partially concluded 5.3 Preliminary results (if any) 5.4 Suggested planning for SCSP 4234	SCSP 4234 Chapter 6: Conclusion 6.1 Introduction 6.2 Achievement of project objectives 6.3 Research Contribution (if any) 6.4 Suggestions for improvement and future works
---	--

6.1 Introduction

- Restate the project significance and objectives.

6.2 Achievement of Project Objectives

- Briefly explain findings based on literature review
- Briefly explain any objectives that has been concluded or partially concluded

6.3 Suggestions for Future Improvement

Suggested plan for project implementation/execution.

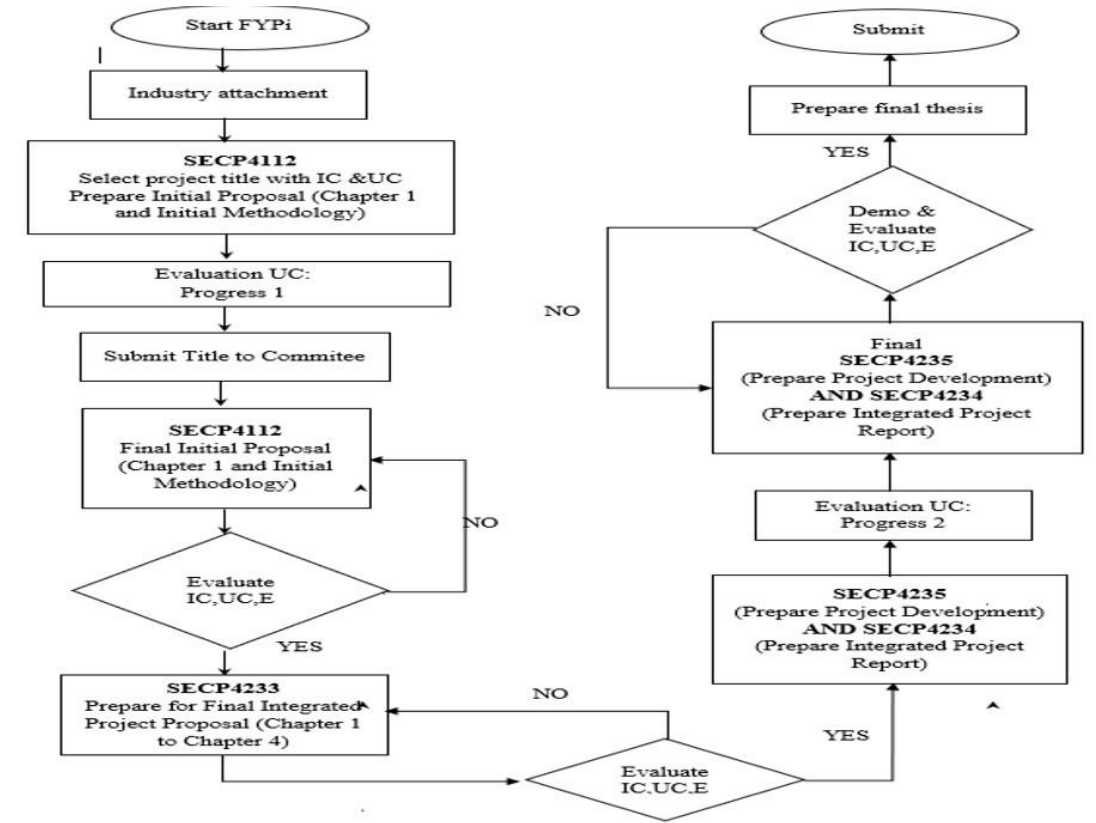
NOTES

When labeling the figures and tables in the appendices, it is recommended that it begins with the name of the appendix. For example, figures in Appendix A will be labeled Figure A1, Figure A2 and so on; while tables in Appendix A will be labeled Table A1,

Table A2 and so on. It is not advisable to continue the labeling from the main text, even if the appendix is an extension of the main text. For example, if Appendix C holds the database tables that are not put into the main text in Chapter 4 (Design), it must not be labeled as Figure 4.xx.

APPENDIX

Appendix A: Flowchart



Appendix B: Pseudo Code

Appendix C: Time-series Results Long

References

- Laudon, K. C., & Laudon, J. P. (2022). *Management Information Systems: Managing the Digital Firm* (17th ed.). Pearson.
- O'Brien, J. A., & Marakas, G. M. (2019). *Management Information Systems* (11th ed.). McGraw-Hill Education.
- Armstrong, M. (2023). *Armstrong's Handbook of Human Resource Management Practice* (16th ed.). Kogan Page.
- Dessler, G. (2020). *Human Resource Management* (16th ed.). Pearson.
- Mojumder, A. H. (2024). *Human Resource Management: Strategies, Challenges, and Evolution*. ISAR Publisher.
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. *IEEE Computer*, 29(2), 38–47.
- Ferraiolo, D. F., Chandramouli, R., & Kuhn, D. R. (2001). *Role-Based Access Control*. Artech House.
- Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing* (Special Publication 800-145). National Institute of Standards and Technology.
- Erl, T., Puttini, R., & Mahmood, Z. (2013). *Cloud Computing: Concepts, Technology & Architecture*. Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- Sommerville, I. (2020). *Software Engineering* (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

2.3

- Runhaar, P. (2017). *How can schools and teachers benefit from Human Resources Management? Educational Management Administration & Leadership*, 45(4), 639–656.
- Shaikh, H. (2024). *E-HRM Systems as Strategic Enablers of Teachers' Performance in Higher Education Institutions*.
- Al-Hassan, A., et al. (2022). *Teacher Management Information Systems for Educational Administration: A Review. International Journal of Educational Management*.
- Shafie, N., et al. (2023). *Development of a Web-Based Teacher Management System for Educational Institutions. Journal of Information Systems and Digital Technologies*.
- Damayanto, A., Hardiansyah, A., & Mayasari, A. R. (2025). *School Management Automation: Analyzing the Impact of Management Information Systems on the Effectiveness*

of the Learning Process. Indonesian Journal of Technology and Education.

Grepon, B. G. S., Baran, N. T., Gumonan, K. M. V. C., Martinez, A. L. M., & Lacs, M. L. E. (2021). *Designing and Implementing e-School Systems: An Information Systems Approach to School Management of a Community College in Northern Mindanao, Philippines*. ArXiv.

Almubarak, A., et al. (2024). *An analytical approach for an AI-based teacher performance evaluation system in Saudi Arabia's schools*. *SN Applied Sciences*.

Kardianto, T., et al. (2023). *Designing Key Performance Indicators (KPI) for Teachers to Improve the Performance*. In *Proceedings of INSYMA 2022*.

Ali, A., et al. (2022). *Development of an Online Leave Management System for Organizational Administration*. *International Journal of Computer Applications*.

Ibrahim, M., et al. (2023). *Digital Leave Management System: Improving Administrative Efficiency in Organizations*. *International Journal of Advanced Computer Science and Applications*.

2.4

Google. (2025). *Flutter Documentation*. <https://docs.flutter.dev>

Zammetti, F. (2020). *Practical Flutter: Improve Your Mobile Development with Google's Latest Open-Source SDK*. Apress.

Google. (2025). *Dart Documentation*. <https://dart.dev>

Buckett, C. (2021). *Dart in Action*. Manning Publications.

Google. (2025). *Firestore Authentication Documentation*.
<https://firebase.google.com/docs/auth>

Google. (2025). *Cloud Firestore Documentation*. <https://firebase.google.com/docs/firestore>

Google. (2025). *Firestore Storage Documentation*. <https://firebase.google.com/docs/storage>

Loeliger, J., & McCullough, M. (2022). *Version Control with Git* (3rd ed.). O'Reilly Media.

GitHub. (2025). *GitHub Documentation*. <https://docs.github.com>